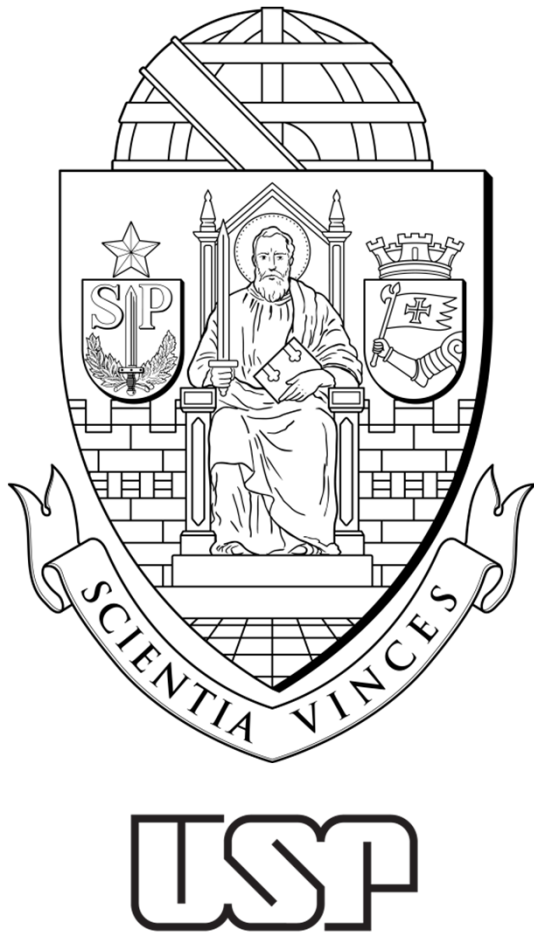


Estratégias de Busca Informada



- ❑ Nesta aula são descritas algumas estratégias de busca informada em espaços de estados que são usadas quando há informações específicas sobre o problema
- ❑ Em geral, as informações específicas tendem a melhorar o processo de busca

Busca Informada

- ❑ A busca em grafos pode atingir uma complexidade elevada devido ao número de alternativas
- ❑ Estratégias de busca informada utilizam conhecimento específico do problema (além da definição do próprio problema) para encontrar soluções de forma mais eficiente do que a busca sem informações
- ❑ A abordagem geral que utilizaremos é denominada **busca pela melhor escolha** (**best-first** search)
- ❑ O termo “busca pela melhor escolha” tem seu uso consagrado mas ele é inexato
 - Se fosse realmente possível expandir o melhor nó primeiro, não haveria a necessidade de se realizar uma busca; seria um caminho direto ao objetivo
 - O que podemos fazer é escolher o nó que *parece* ser o melhor de acordo com a função de avaliação
 - Se a função de avaliação for precisa, esse será de fato o melhor nó; caso contrário, a busca pode se perder

Busca Informada

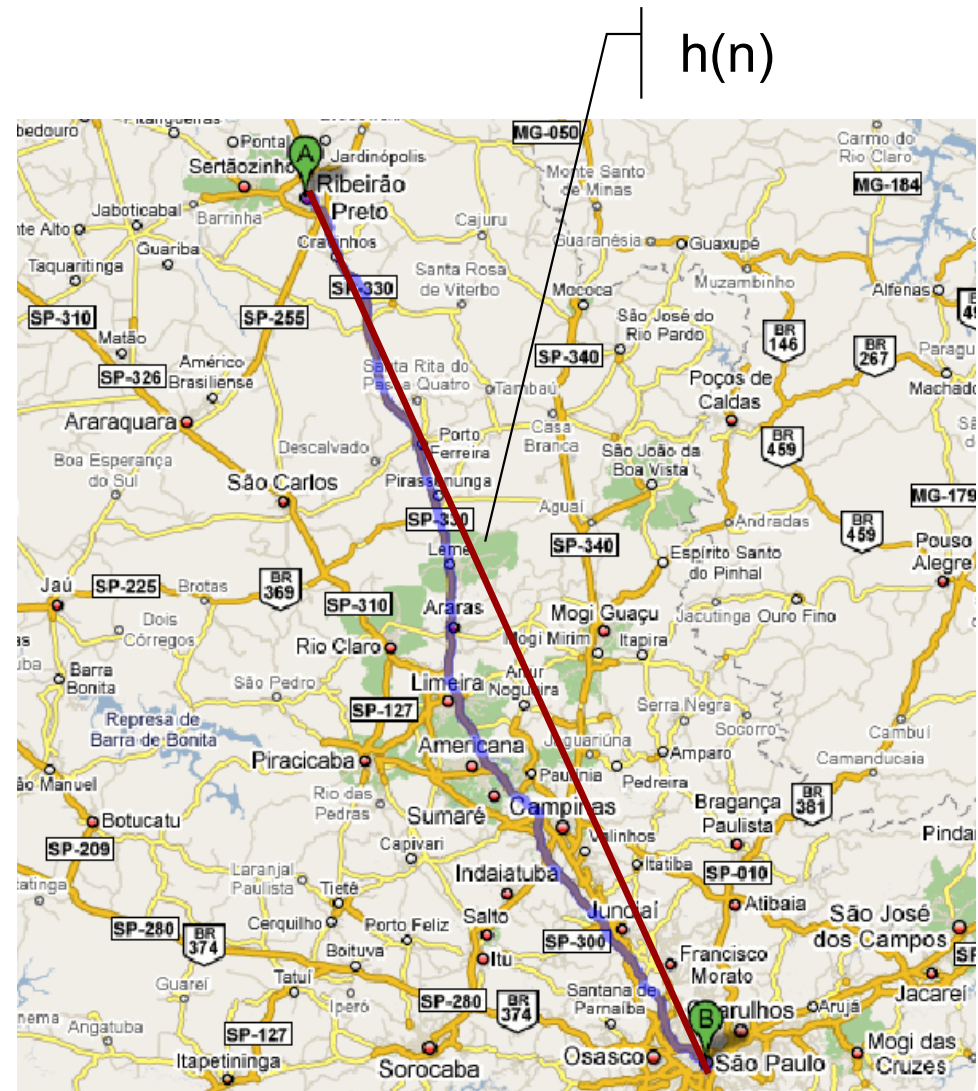
- ❑ Na busca best-first, a escolha do nó **n** a ser expandido é efetuada com base em uma função de avaliação **f(n)**
 - Em geral, o nó **n** com o menor valor **f(n)** é selecionado para a expansão, uma vez que a função de avaliação mede a “distância” de **n** até o objetivo, ou seja, até um nó final
- ❑ A forma mais comum de adicionar **conhecimento** do problema ao algoritmo de busca (daí o nome **busca informada**) e, portanto, um componente fundamental dos algoritmos de busca best-first é uma **função heurística**, denotada por **h(n)**
 - **h(n)** = custo **estimado** do caminho mais econômico partindo do nó **n** e chegando a um nó final
- ❑ As funções heurísticas são específicas do problema, exceto pela seguinte restrição que se aplica a todos
 - **h(n)=0** se **n** é um nó final

Função Heurística

- ❑ **Heurística** (arte de descobrir): conhecimentos que permitem uma solução rápida para algum problema
 - Heureka (“Eu encontrei”, Arquimedes)
- ❑ Uma **heurística** é uma função que quando aplicada a um estado retorna um número que corresponde a uma **estimativa** da qualidade (mérito) do estado atual em relação a um estado final
- ❑ Em outras palavras, a heurística nos informa **aproximadamente** quão longe o estado atual está de um estado final
 - Heurísticas podem subestimar ou superestimar a qualidade de um estado
 - Heurísticas que apenas subestimam são altamente desejáveis e são chamadas **admissíveis**
 - ❖ i.e. número menores são melhores
 - Heurísticas admissíveis são **otimistas**, pois estimam que o custo da solução de um problema seja menor do que é na realidade

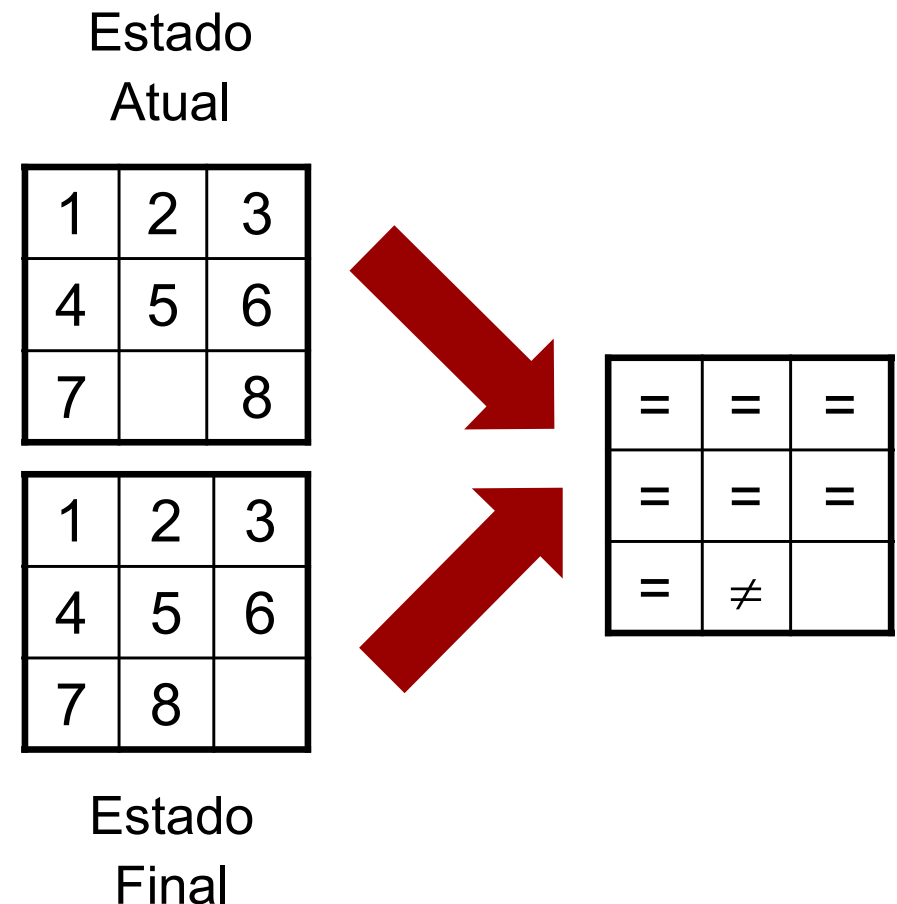
Função Heurística: Distância em Linha Reta

- ❑ Por exemplo, em um mapa contendo cidades e estradas que as interconectam, poderíamos estimar o custo do caminho mais curto (econômico) entre duas cidades como sendo a **distância em linha reta**
- ❑ Esta heurística é admissível pois o caminho mais curto entre dois pontos quaisquer no plano é uma linha reta e, portanto, não pode ser uma superestimativa



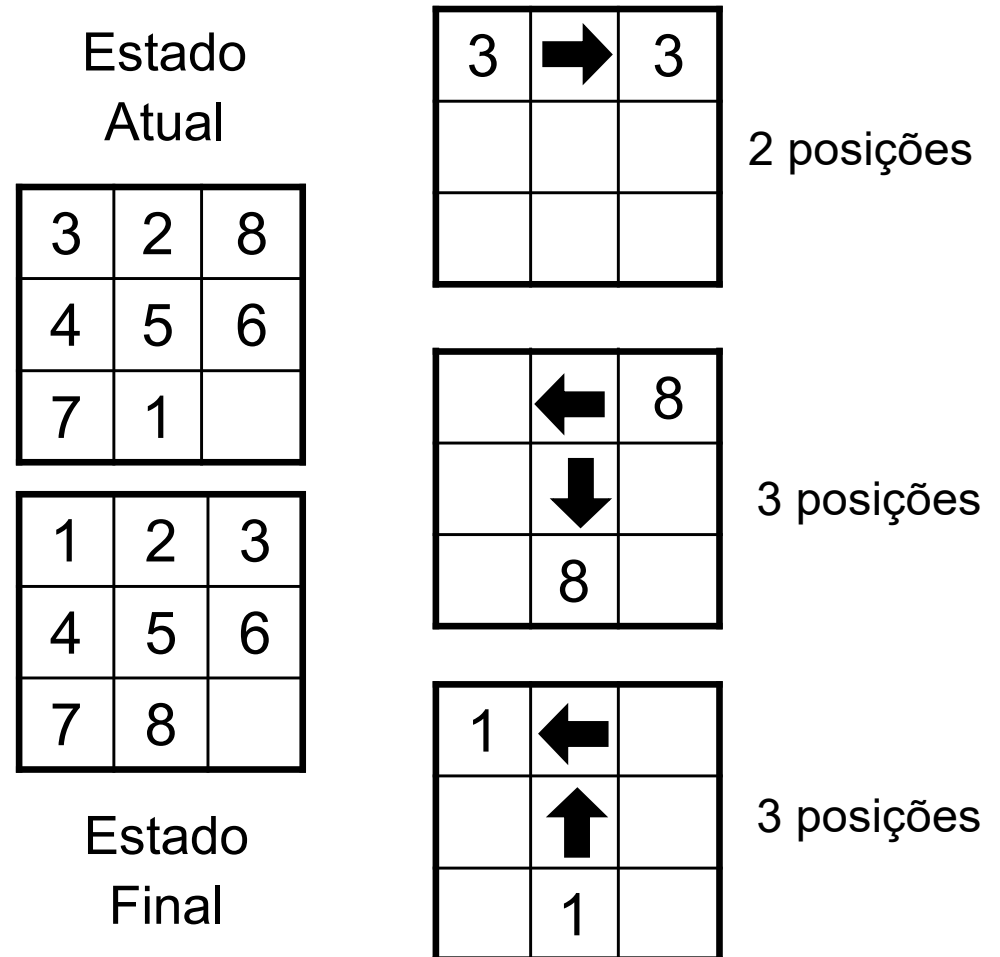
Função Heurística I para o Quebra-Cabeça de 8 Peças

- ❑ Número de peças fora do lugar (sem incluir o espaço vazio)
- ❑ Nesta situação, apenas a peça “8” está fora do lugar; portanto a função heurística é igual a 1
- ❑ Em outras palavras, a heurística nos informa que ela **estima** que uma solução pode estar disponível em apenas mais um movimento
 - $h(\text{estado atual}) = 1$



Função Heurística II para o Quebra-Cabeça de 8 Peças

- ❑ Distância Manhattan (sem incluir o espaço vazio)
- ❑ Nesta situação, apenas as peças “1”, “3” e “8” estão fora do lugar por 2, 3 e 3 posições, respectivamente; portanto a função heurística é igual a 8
- ❑ Em outras palavras, a heurística nos informa que ela **estima** que uma solução pode estar disponível em apenas mais 8 movimentos
 - $h(\text{estado atual}) = 8$

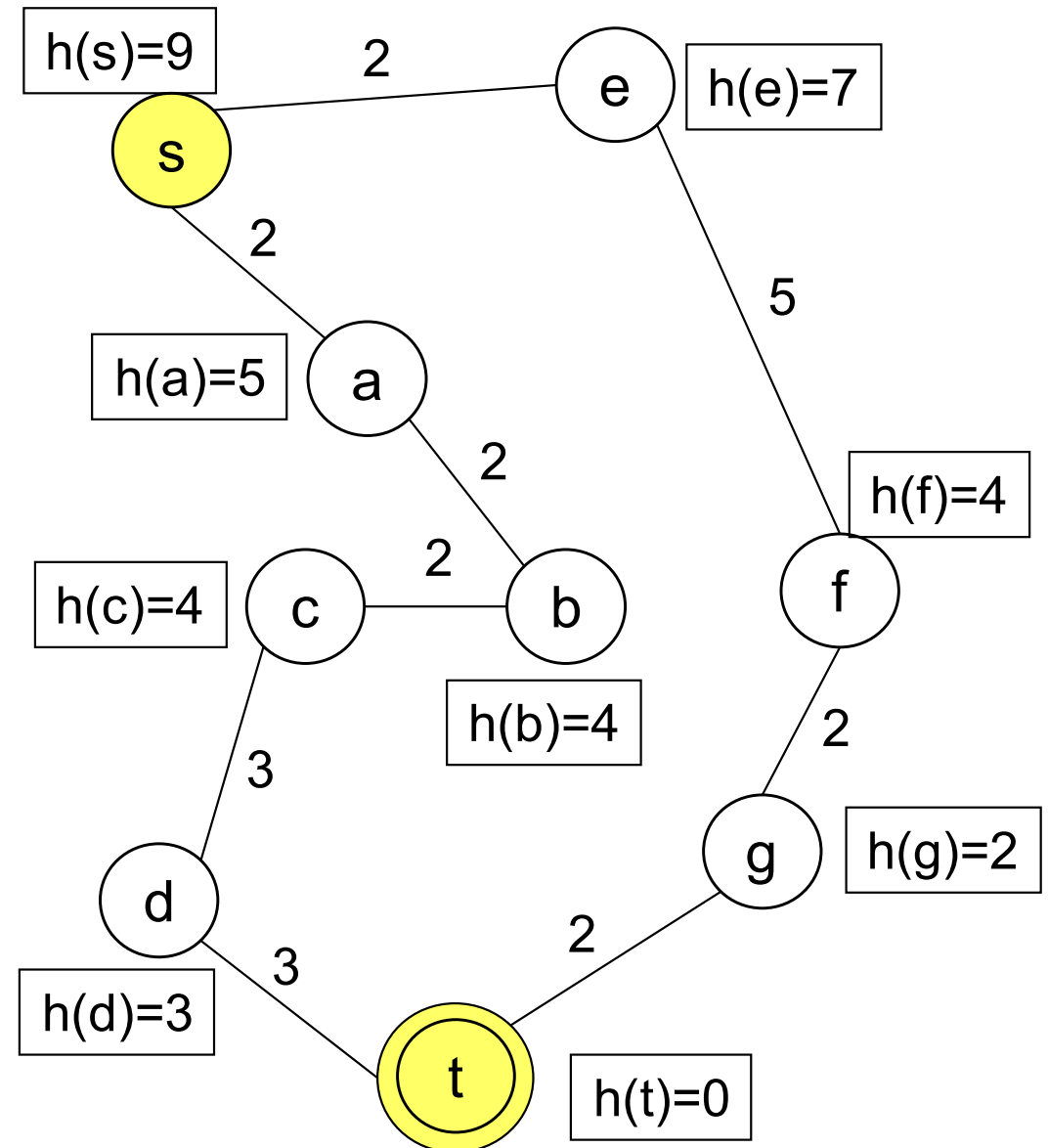


Busca Gulosa pela Melhor Escolha

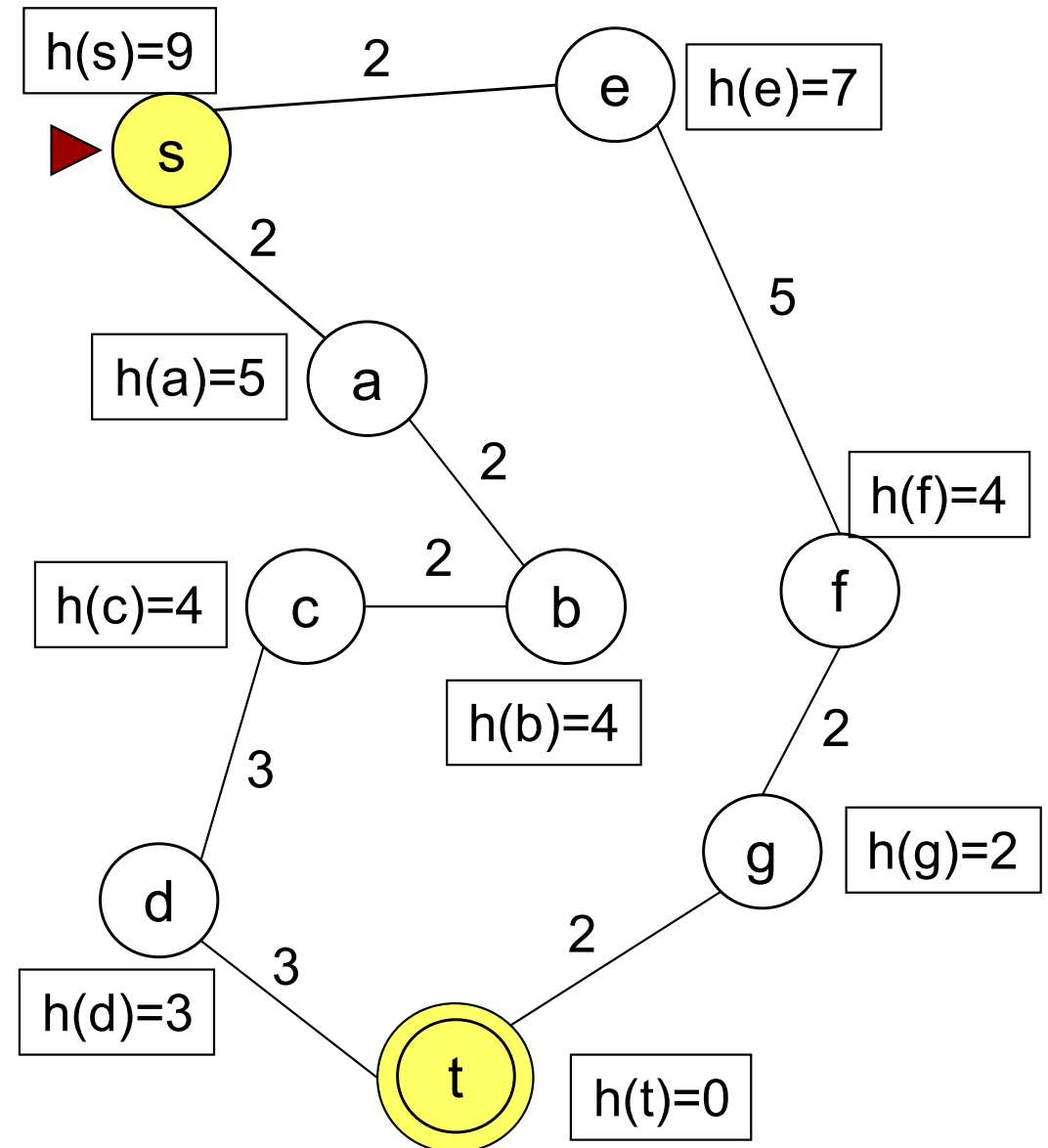
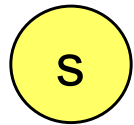
- ❑ A **busca gulosa pela melhor escolha** (greedy best-first search) tenta expandir o nó mais próximo ao nó final, assumindo que isso provavelmente levará a uma solução rápida
- ❑ Dessa forma, a busca avalia os nós utilizando apenas a função heurística
 - $f(n) = h(n)$
- ❑ Greedy best-first é semelhante à busca em profundidade
 - Prefere seguir um único caminho até o objetivo, retrocedendo somente se encontrar um beco sem saída
 - Mesmos defeitos da busca em profundidade: não é ótima, nem completa (pode entrar em um caminho infinito e nunca retornar para testar outras possibilidades)
 - Complexidade de tempo e espaço no pior caso são iguais a $O(b^m)$, onde **m** é a profundidade máxima do espaço de busca

Exercício

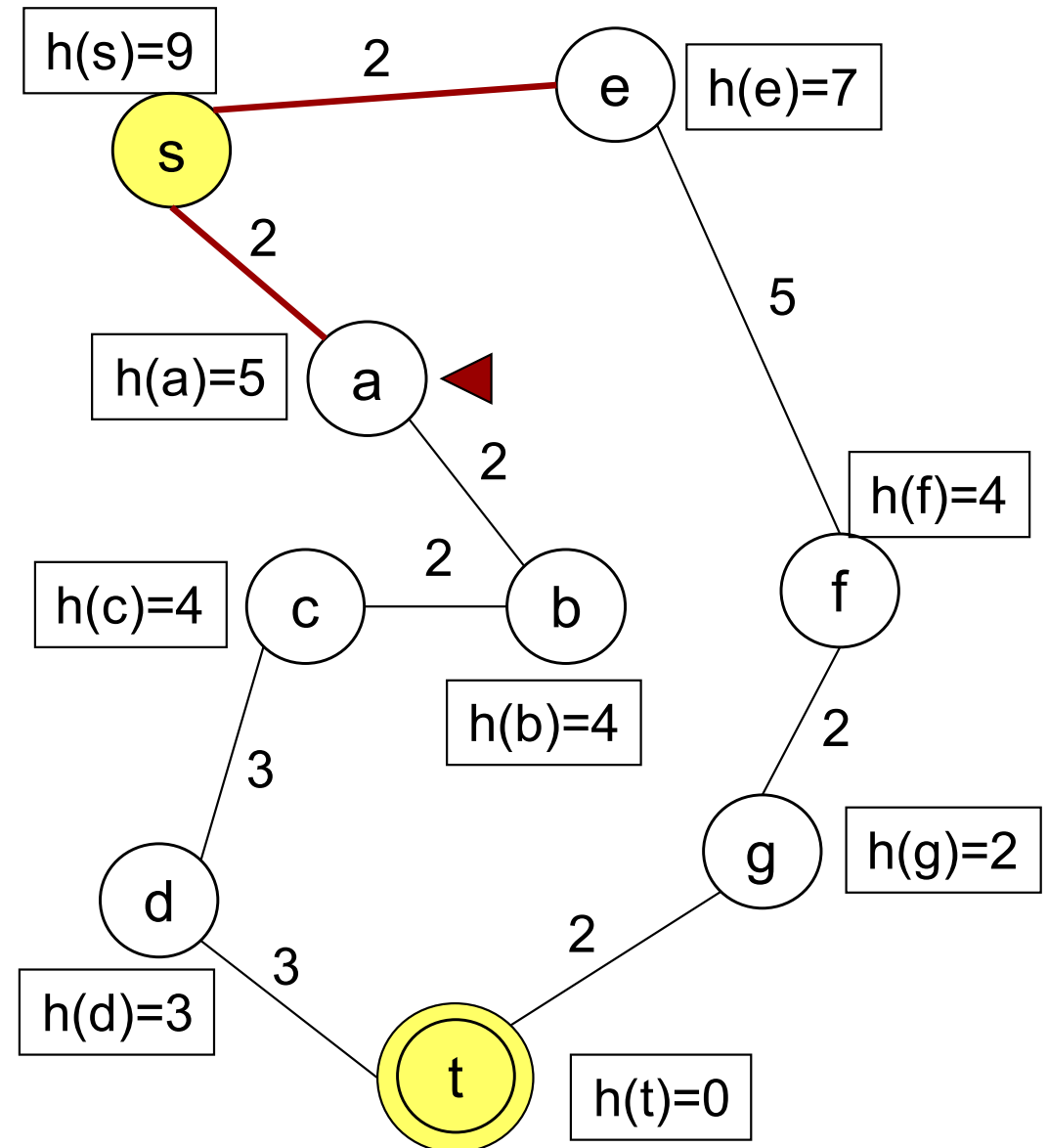
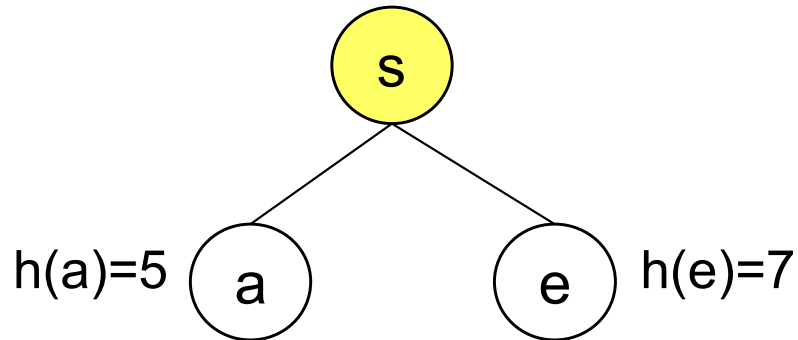
- Encontre o caminho de **s** até **t** usando busca gulosa pela melhor escolha



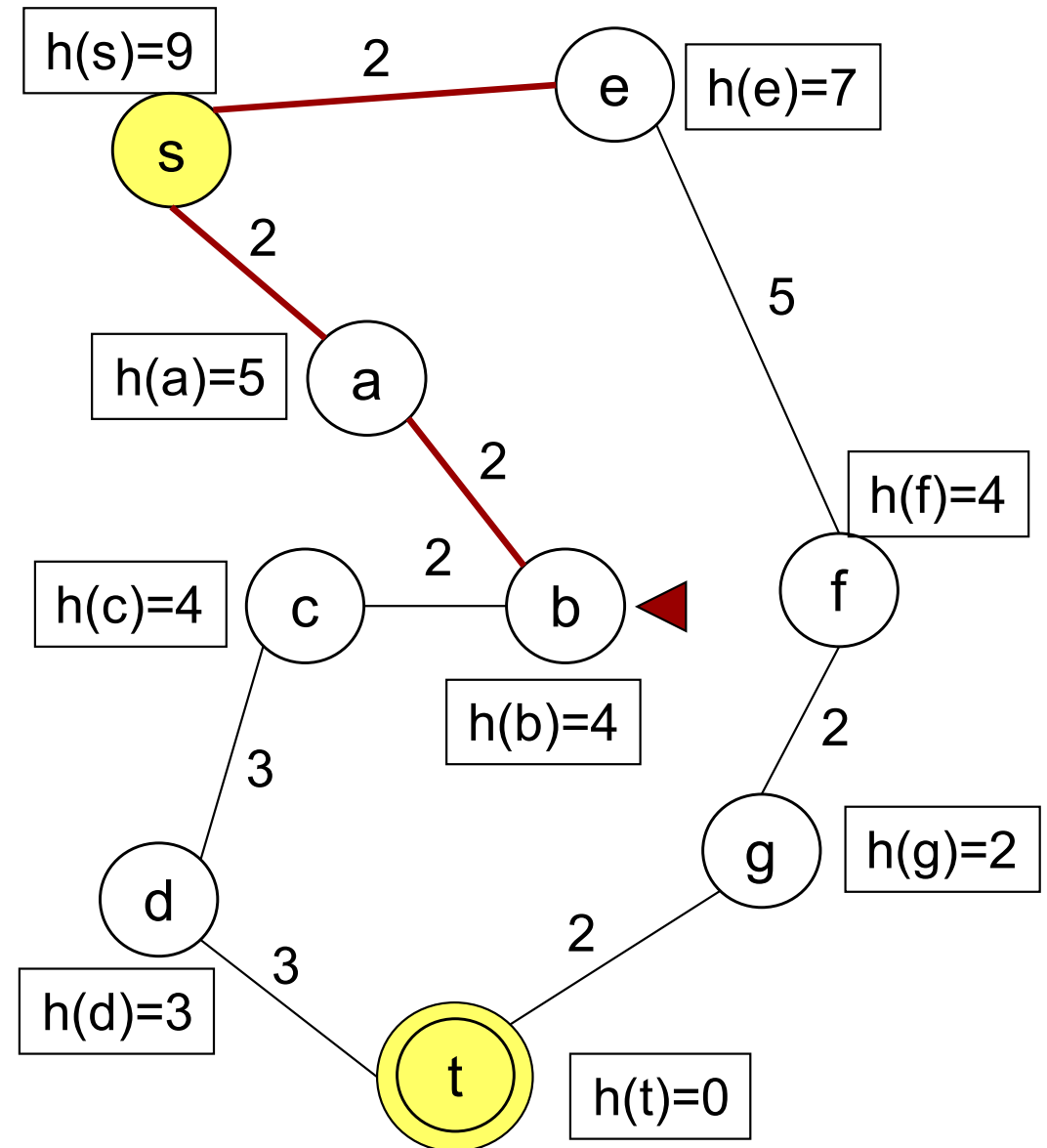
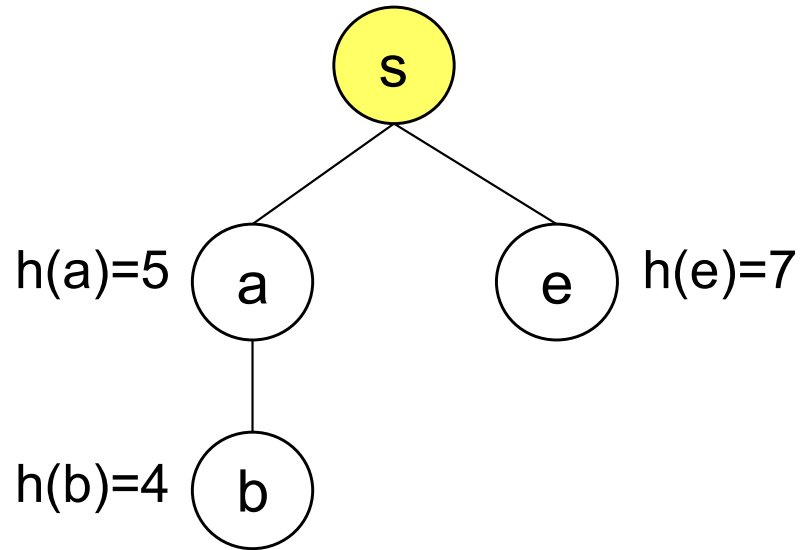
Busca Gulosa pela Melhor Escolha



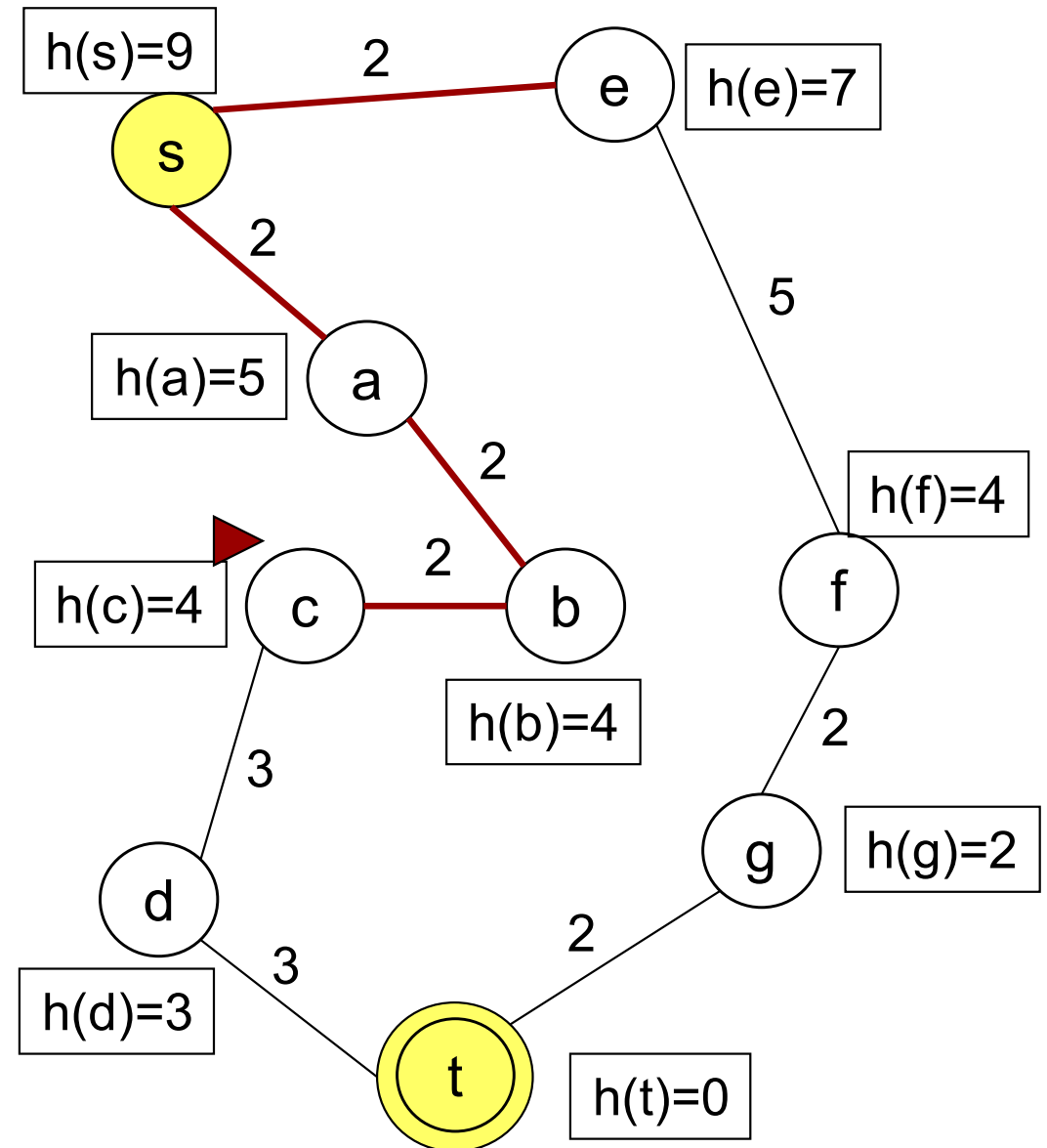
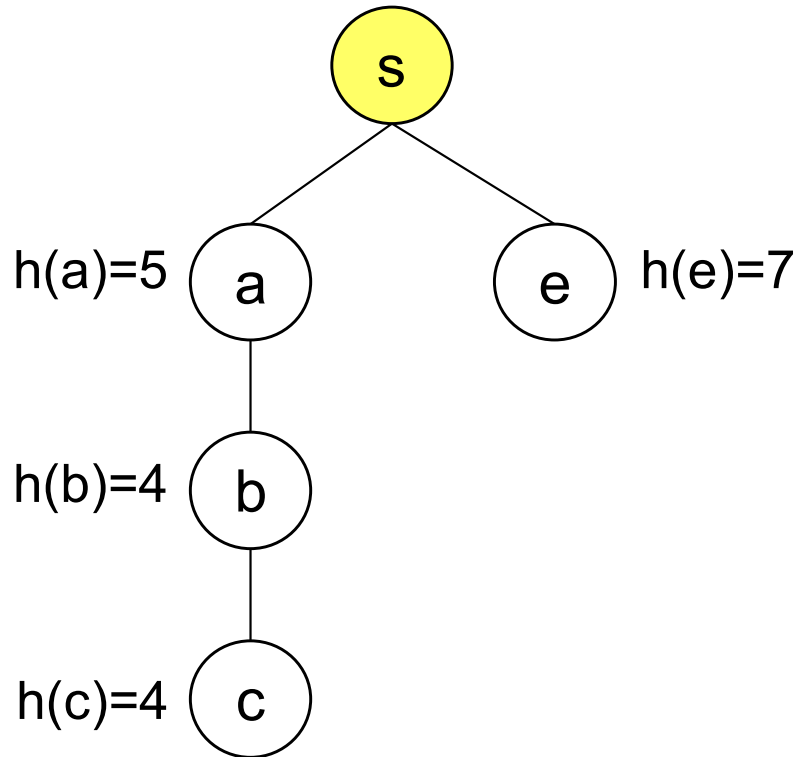
Busca Gulosa pela Melhor Escolha



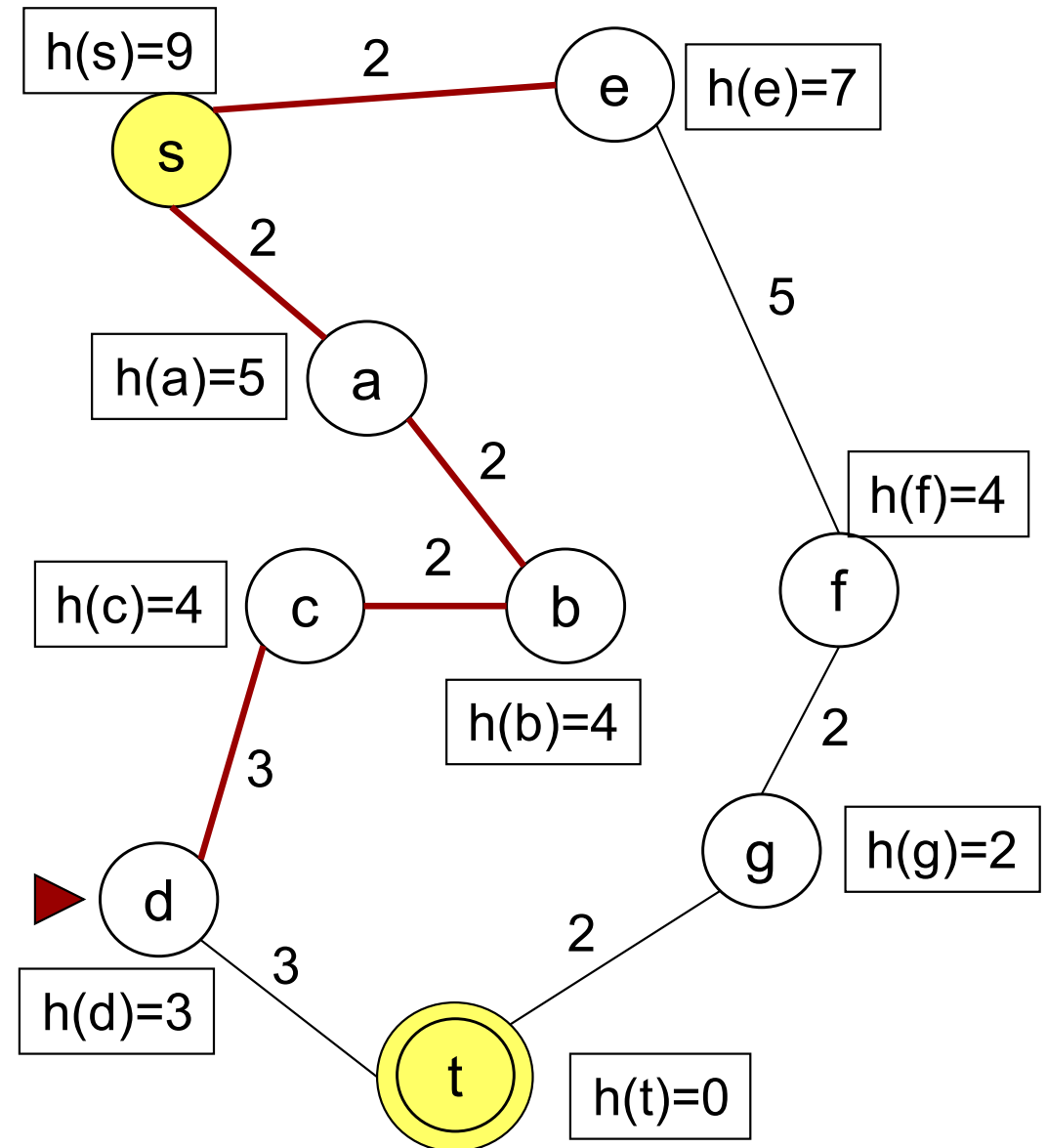
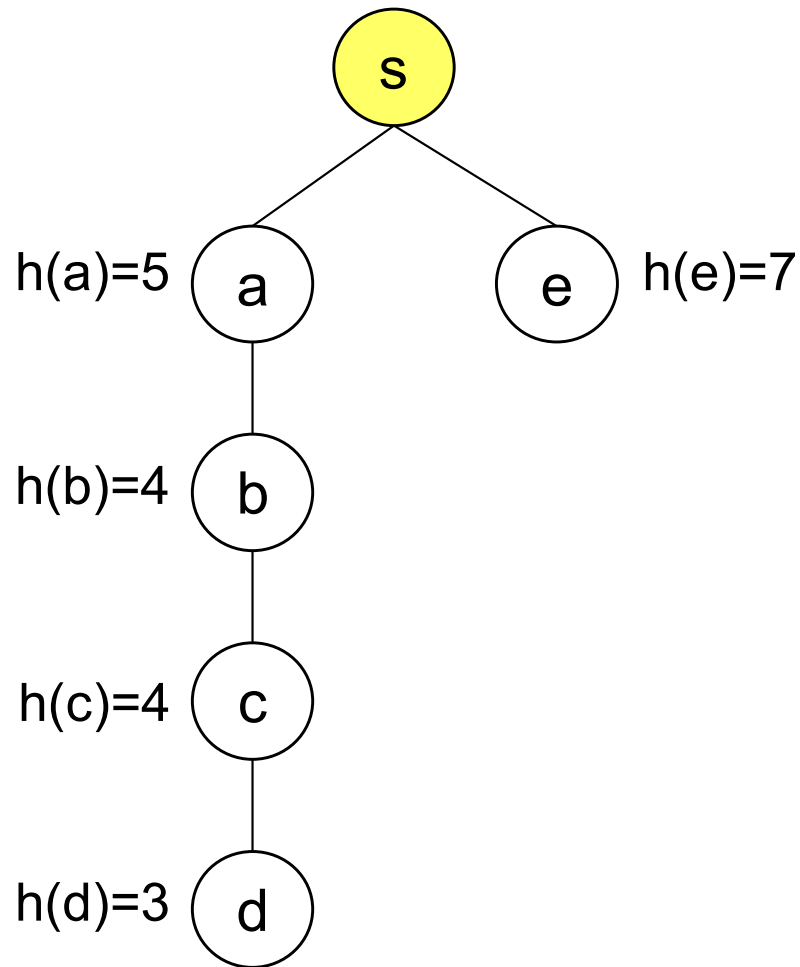
Busca Gulosa pela Melhor Escolha



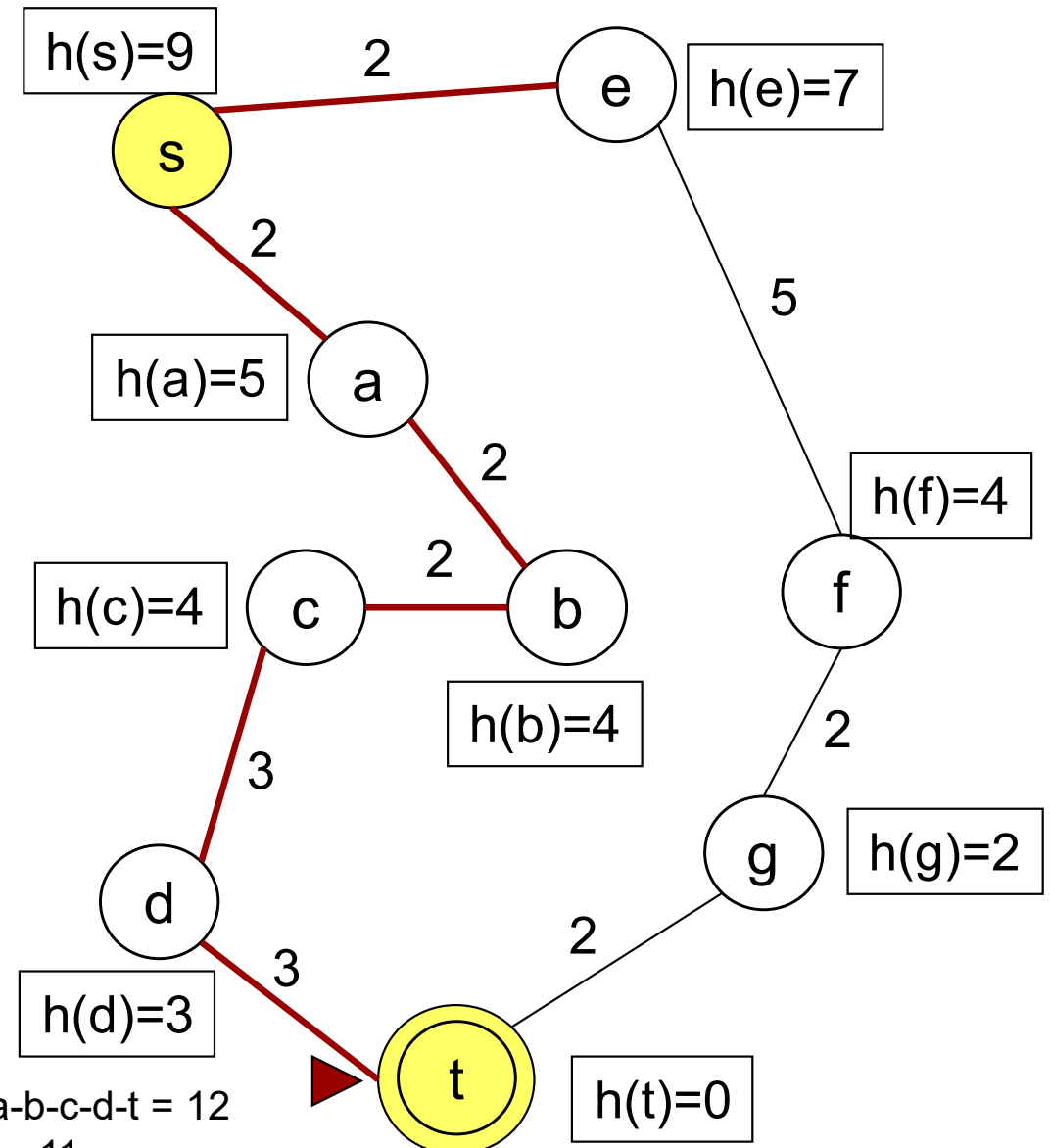
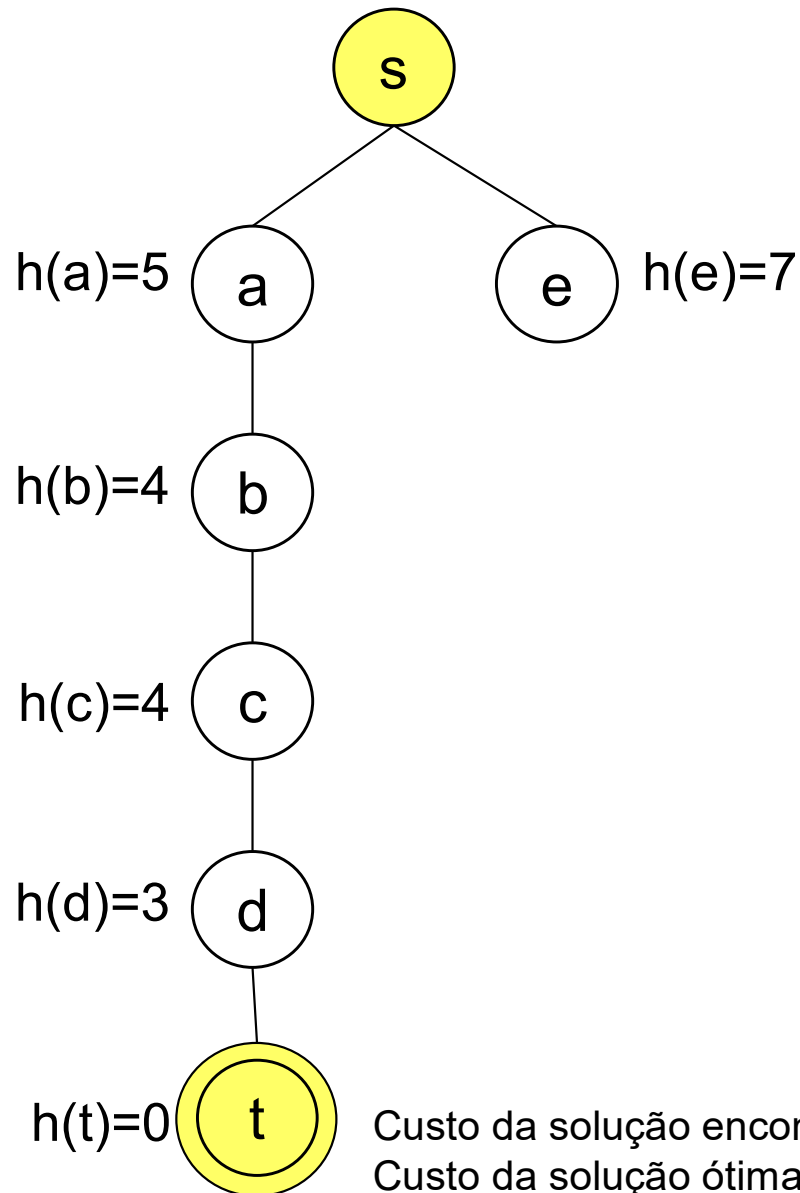
Busca Gulosa pela Melhor Escolha



Busca Gulosa pela Melhor Escolha

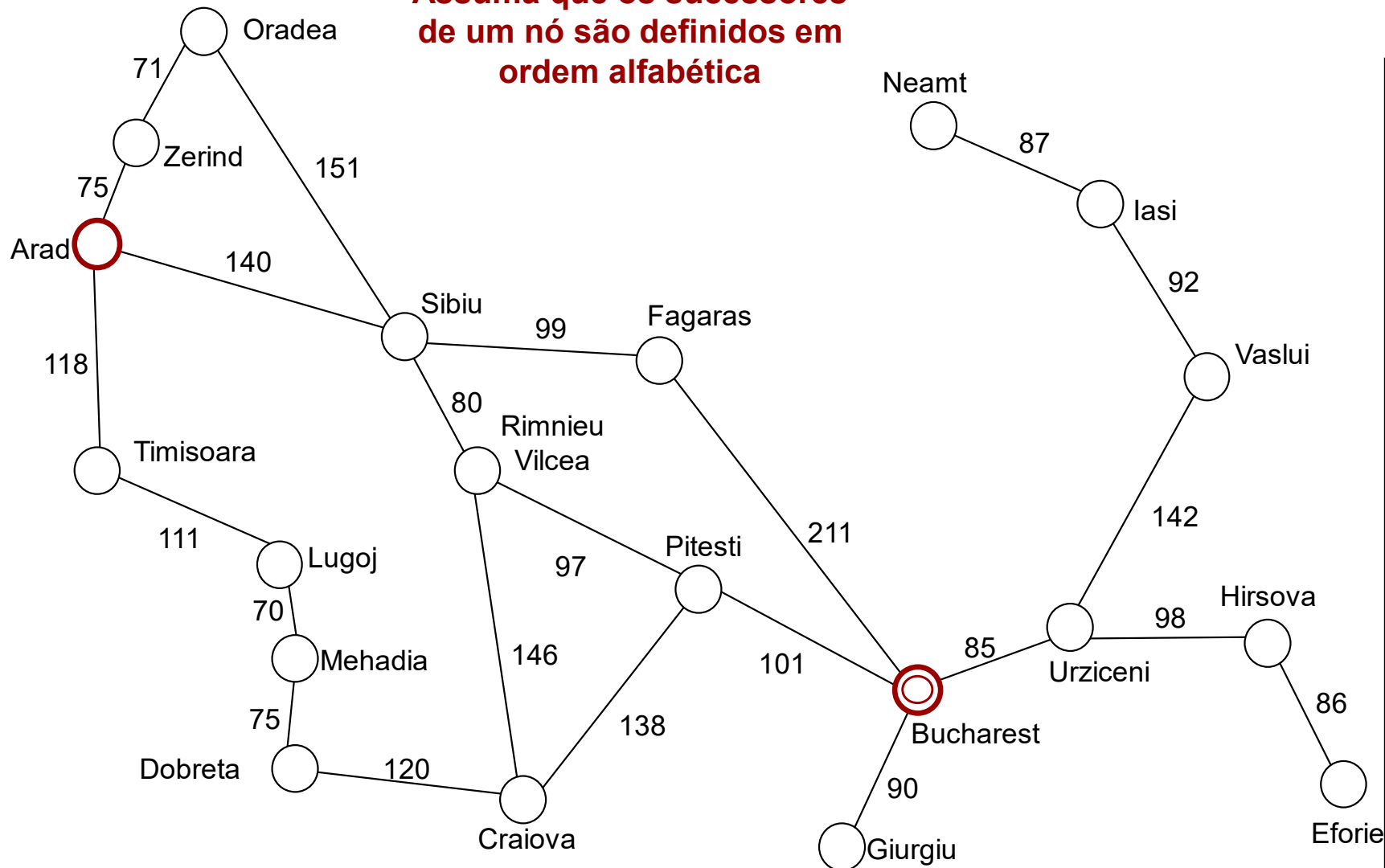


Busca Gulosa pela Melhor Escolha



Busca Gulosa pela Melhor Escolha: Encontre o caminho de Arad até Bucharest

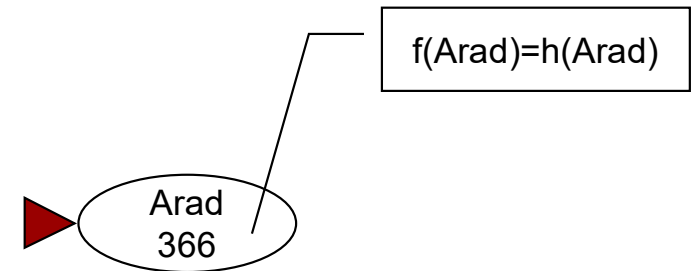
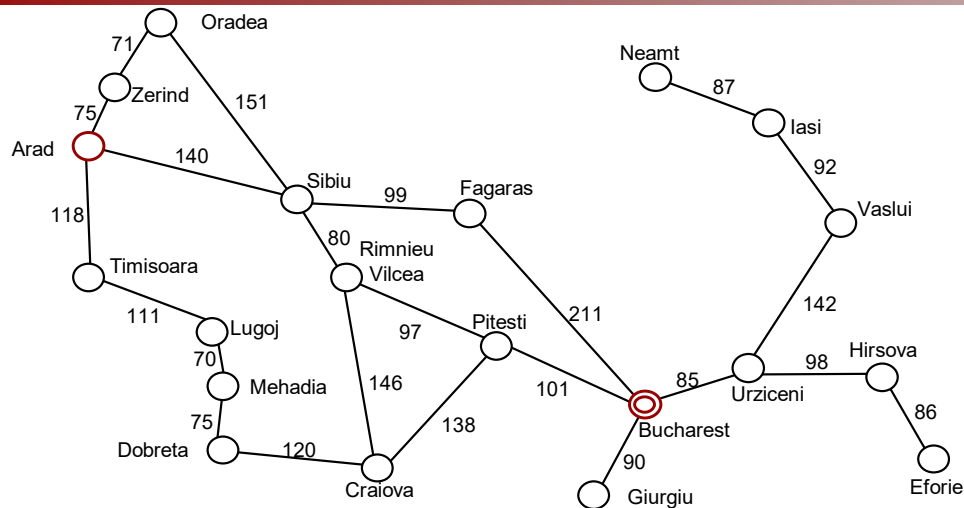
Assuma que os sucessores de um nó são definidos em ordem alfabética



Distância em linha reta até Bucharest

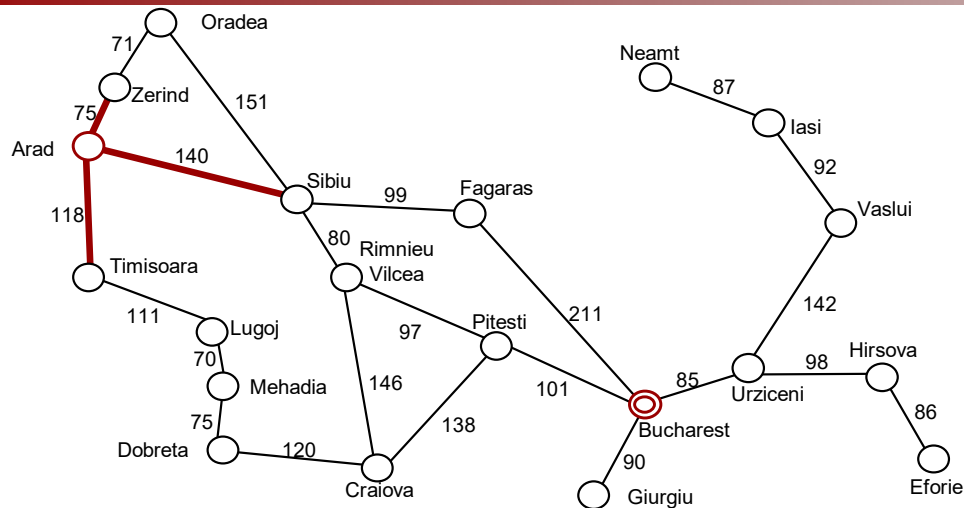
h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimniew Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Encontre o caminho de Arad até Bucharest

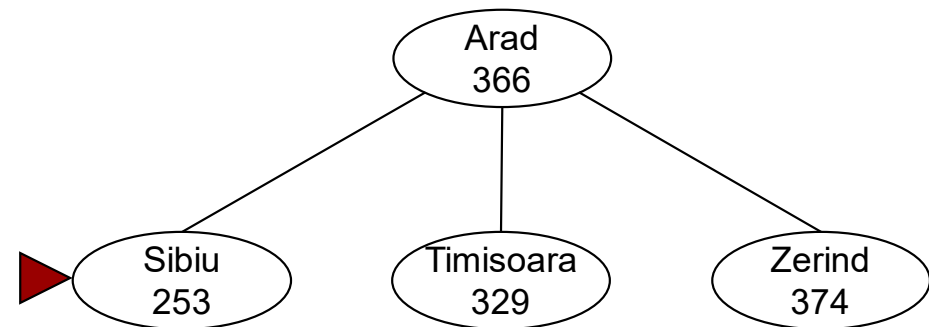


h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnieniu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

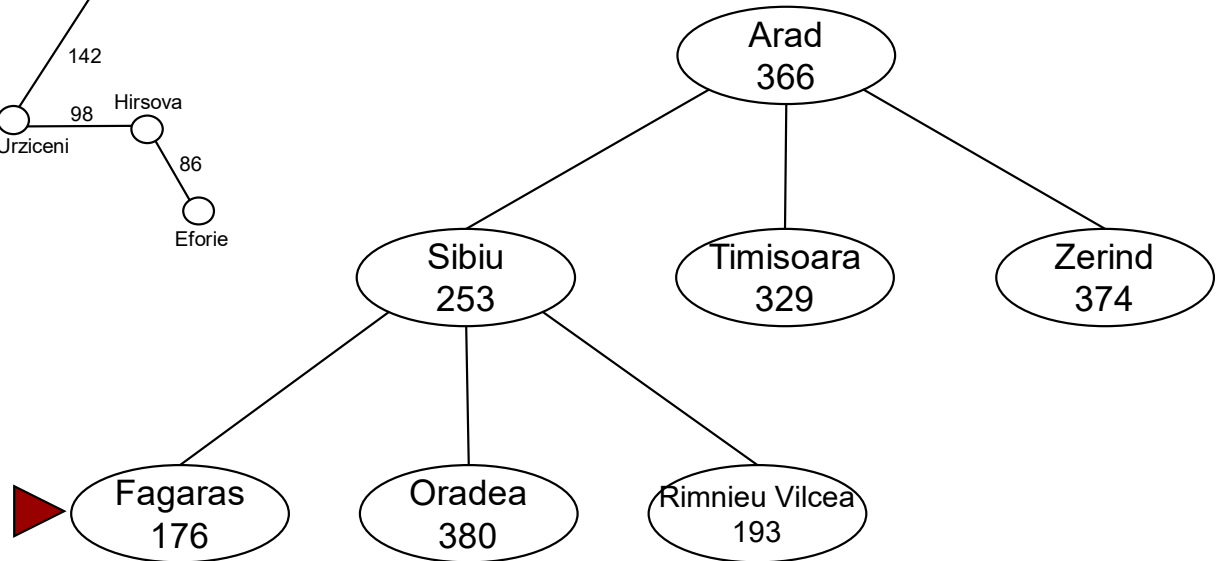
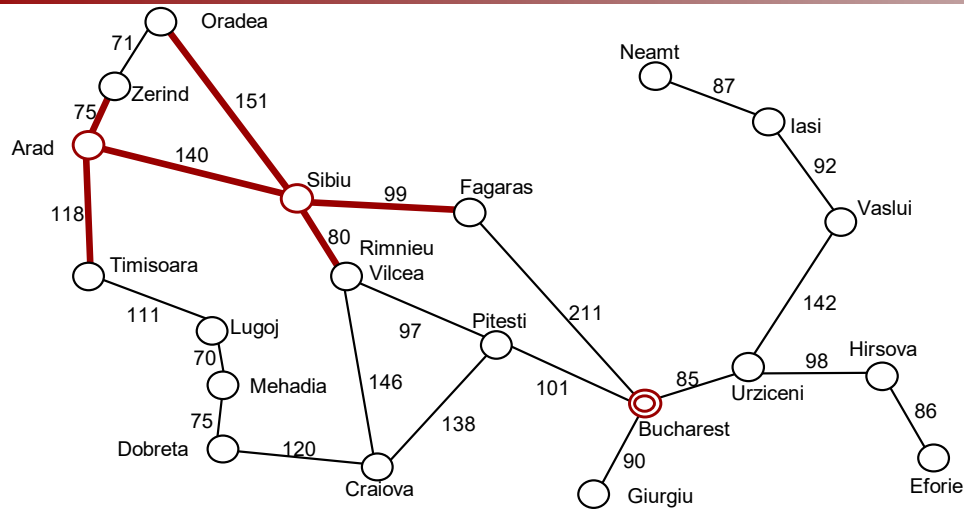
Encontre o caminho de Arad até Bucharest



h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnieniu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

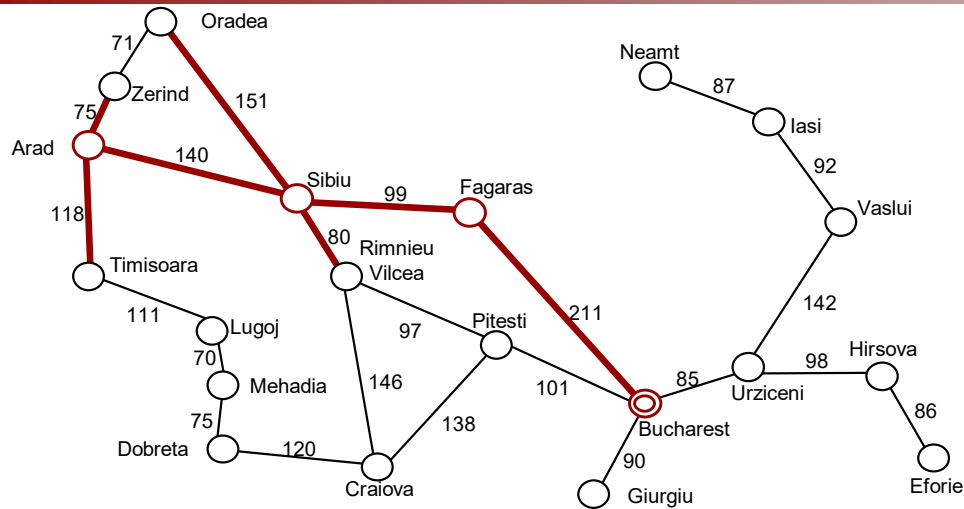


Encontre o caminho de Arad até Bucharest

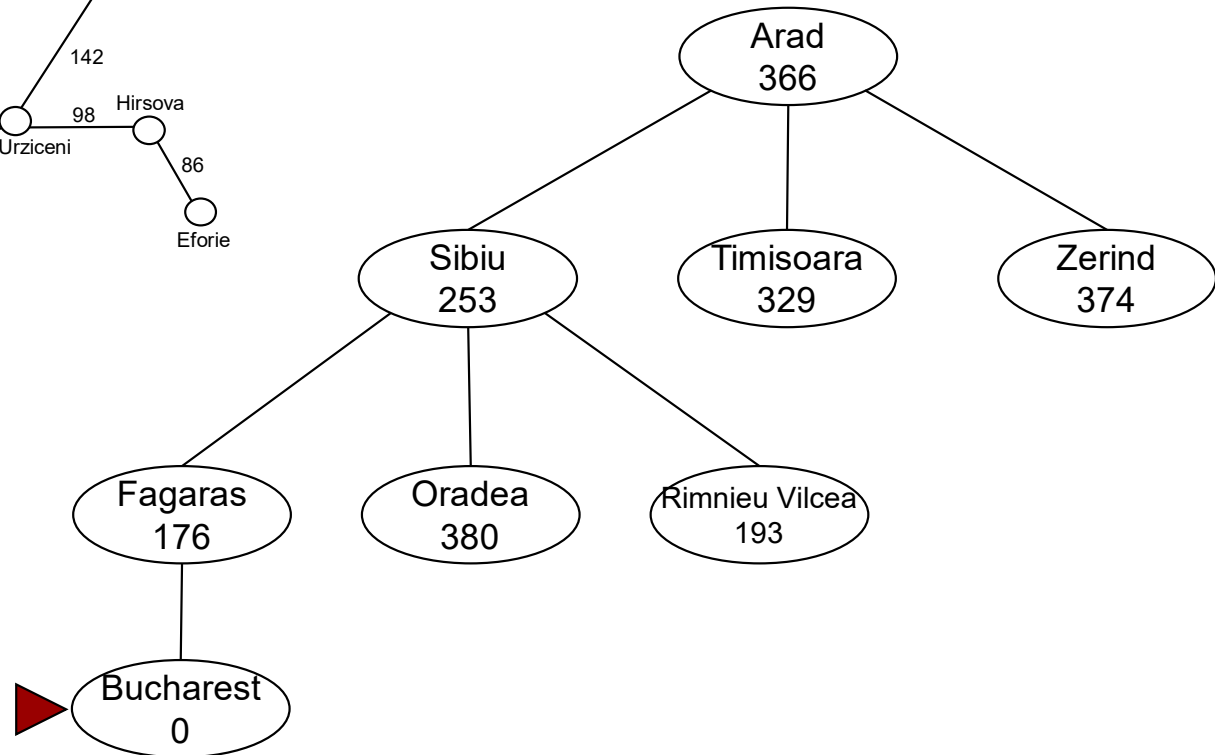


h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnien Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Encontre o caminho de Arad até Bucharest



h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnien Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Custo da solução encontrada A-S-F-B = 450
Custo da solução ótima A-S-RV-P-B = 418

Greedy Best-First

```
resolvaGreedy(No, Solucao) :-
    h(No, H),
    greedy([[No]:H], Solucao). % Fila armazena apenas [Caminho]:H

greedy([[No|Caminho]:_|_], [No|Caminho]) :-
    final(No).
greedy([Caminho:H|Caminhos], Solucao) :-
    estenderGreedy(Caminho:H, NovosCaminhos),
    filaPrioridadeGreedy(NovosCaminhos, Caminhos, Caminhos1),
    greedy(Caminhos1, Solucao).

% Gera vizinhos calculando apenas o H (heuristica)
estenderGreedy([No|Caminho]:_, NovosCaminhos) :-
    findall([NovoNo, No|Caminho]:NovoH,
        ( s(No, NovoNo, _), % O custo da aresta eh ignorado
          \+ member(NovoNo, [No|Caminho]),
          h(NovoNo, NovoH) % A prioridade eh puramente o H
        ),
        NovosCaminhos).

% A fila de prioridade eh idêntica a UCS mas compara o H em vez do G
filaPrioridadeGreedy([], L, L).
filaPrioridadeGreedy([C|Cs], Existente, Final) :-
    insertGreedy(C, Existente, ExistAumentado),
    filaPrioridadeGreedy(Cs, ExistAumentado, Final).

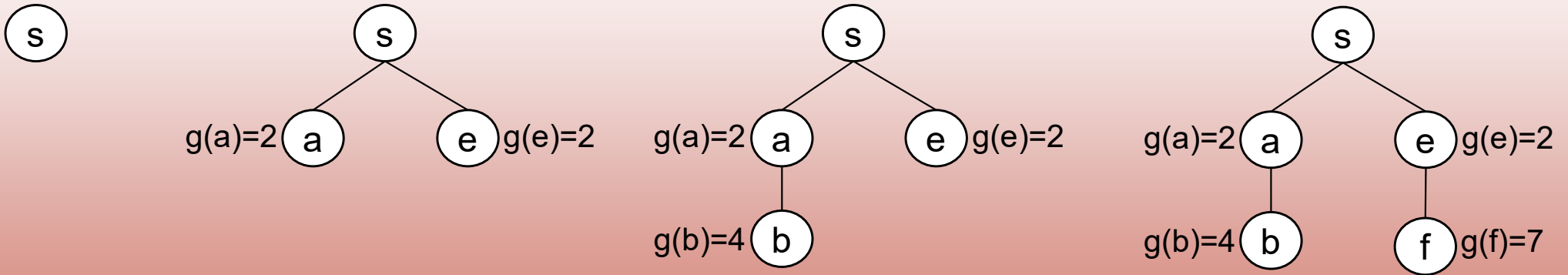
insertGreedy(Cam:H, [CamX:HX|Paths], [CamX:HX|NewPaths]) :-
    H > HX, !,
    insertGreedy(Cam:H, Paths, NewPaths).
insertGreedy(C, Paths, [C|Paths]).
```

O que sabemos?

- ❑ Busca de custo uniforme (uniform cost)
 - Similar à busca em largura
 - Mede o custo de cada nó, desde a raiz da busca
 - É ótima e completa
 - Pode ser muito lenta
- ❑ Busca Gulosa pela Melhor Escolha (greedy best-first)
 - Similar à busca em profundidade
 - Mede o custo estimado de cada nó até um nó final
 - Não é ótima nem completa
 - Pode ser muito rápida, dependendo da qualidade da heurística e do problema específico
- ❑ Seria possível combinar estas estratégias para criar um algoritmo ótimo e completo que seja também muito rápido?

Busca de Custo Uniforme

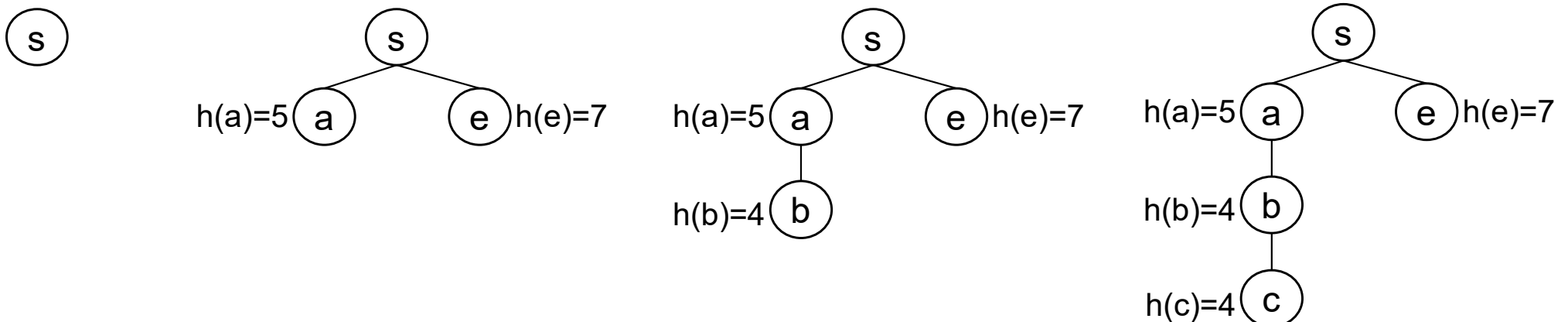
Coloque os estados na fila em ordem de custo



Intuição: Expanda o nó mais barato, onde o custo é o custo do caminho $g(n)$

Busca Gulosa pela Melhor Escolha

Coloque os estados na fila em ordem de distância estimada até o objetivo



Intuição: Expanda o nó que aparenta ser o mais próximo do objetivo, onde a estimativa da distância até o objetivo é $h(n)$

Busca A*

Coloque os estados na fila em ordem de custo total até o objetivo, $f(n)$

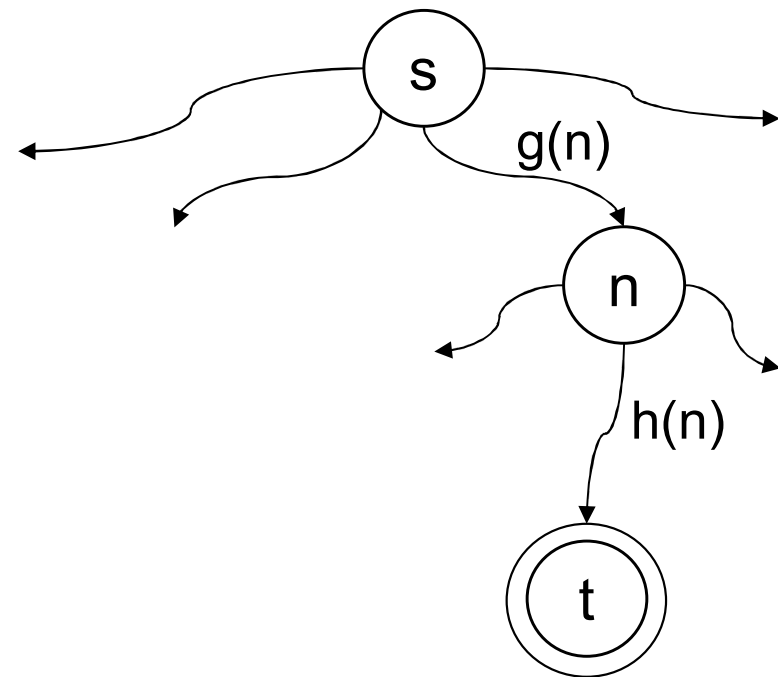
- ❑ Resumidamente:
 - $g(n)$ é o custo até chegar ao nó n
 - $h(n)$ é a distância estimada do nó n até um nó final
 - $f(n) = g(n) + h(n)$
- ❑ Podemos imaginar $f(n)$ como uma estimativa de custo da solução mais barata que passa pelo nó n
- ❑ Note que é possível usar os algoritmos de busca em largura ou de custo uniforme alterando apenas a estratégia na qual os nós são colocados na fila
- ❑ Se a heurística utilizada nunca superestima a distância até o nó final (heurística otimista), então o algoritmo A* é ótimo e completo

Busca A*

- ❑ É conveniente lembrar que uma estratégia de busca é definida por meio da ordem de expansão dos nós
- ❑ Na estratégia de busca *best-first* a idéia básica é prosseguir com a busca sempre a partir do nó mais promissor
- ❑ A **Busca pela Melhor Escolha** (best-first) é um refinamento da busca em largura, que em sua forma completa também é conhecido como **busca A***
 - Ambas estratégias começam pelo nó inicial e mantêm um conjunto de caminhos candidatos
 - Busca em largura expande o caminho candidato mais curto
 - Best-First refina este princípio calculando uma estimativa heurística para cada candidato e escolhe expandir o melhor candidato segundo esta estimativa

A*

- ❑ Vamos assumir que há um custo envolvido entre cada arco:
 - $s(X,Y,C)$ que é verdadeira se há um movimento permitido no espaço de estados do nó X para o nó Y ao custo C ; neste caso, Y é um **sucessor** de X
- ❑ Sejam dados um nó inicial **s** e um nó final **t**
- ❑ Seja o estimador heurístico a função **f** tal que para cada nó **n** no espaço, **f(n)** estima a dificuldade de **n**, ou seja, **f(n)** é o custo do caminho mais barato de **s** até **t** via **n**
- ❑ A função $f(n)$ será construída como: **$f(n) = g(n) + h(n)$**
 - **$g(n)$** é uma estimativa do custo do caminho ótimo de **s** até **n**
 - **$h(n)$** é uma estimativa do custo do caminho ótimo de **n** até **t**



A*

- ❑ Quando um nó **n** é encontrado pelo processo de busca temos a seguinte situação
 - Um caminho de **s** até **n** já foi encontrado e seu custo pode ser calculado como a soma dos custos dos arcos no caminho
 - ❖ Este caminho não é necessariamente um caminho ótimo de **s** até **n** (pode existir um caminho melhor de **s** até **n** ainda não encontrado pela busca) mas seu custo serve como uma estimativa **g(n)** do custo mínimo de **s** até **n**
 - O outro termo, **h(n)** é mais problemático pois o “mundo” entre **n** e **t** não foi ainda explorado
 - ❖ Portanto, **h(n)** é tipicamente uma heurística, baseada no conhecimento geral do algoritmo sobre o problema em questão
 - ❖ Como **h** depende do domínio do problema, não há um método universal para construir **h**

A*

- ❑ Vamos estudar o algoritmo best-first em sua forma completa A* que minimiza o custo total estimado da solução
- ❑ O processo de busca pode ser visto como um conjunto de sub-processos, cada um explorando sua própria alternativa, ou seja, sua própria sub-árvore
- ❑ Sub-árvores têm sub-árvores que são exploradas por sub-processos dos sub-processos, etc
- ❑ Dentre todos os processos apenas um encontra-se ativo a cada momento: aquele que lida com a alternativa atual mais promissora (aquela com menor valor f)
- ❑ Os processos restantes aguardam silenciosamente até que a estimativa f atual se altere e alguma outra alternativa se torne mais promissora
- ❑ Então, a atividade é comutada para esta alternativa

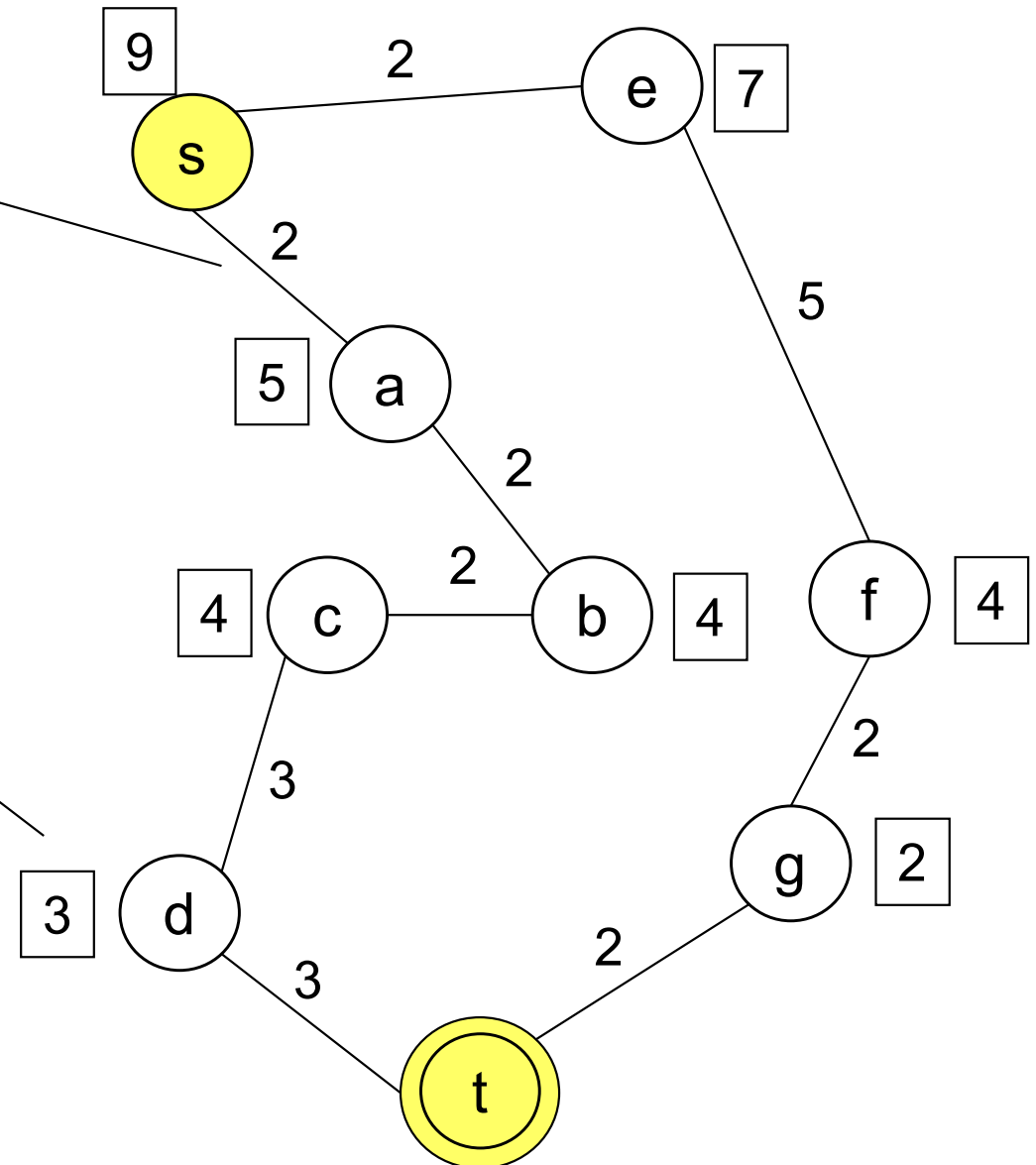
A*

- ❑ Podemos imaginar o mecanismo de ativação-desativação da seguinte forma
 - O processo trabalhando na alternativa atual recebe um orçamento limite
 - Ele permanece ativo até que o orçamento seja exaurido
 - Durante o período em que está ativo, o processo continua expandindo sua sub-árvore e relata uma solução caso um nó final seja encontrado
 - O orçamento limite para essa execução é definido pela estimativa heurística da alternativa competidora mais próxima

A*

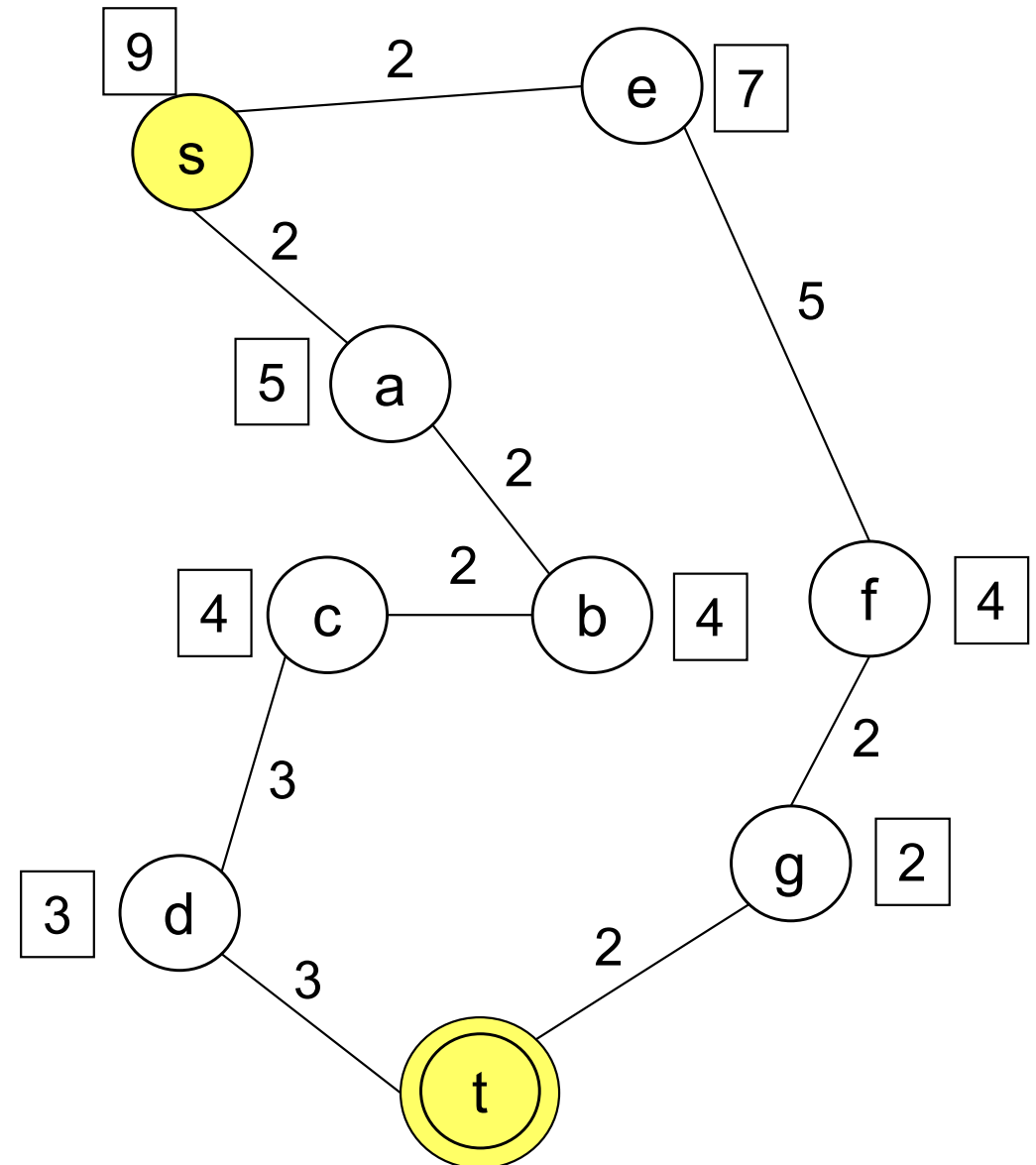
Distância entre duas cidades através de um caminho (rodovia)

Distância entre a cidade em questão e a cidade destino (t) em linha reta



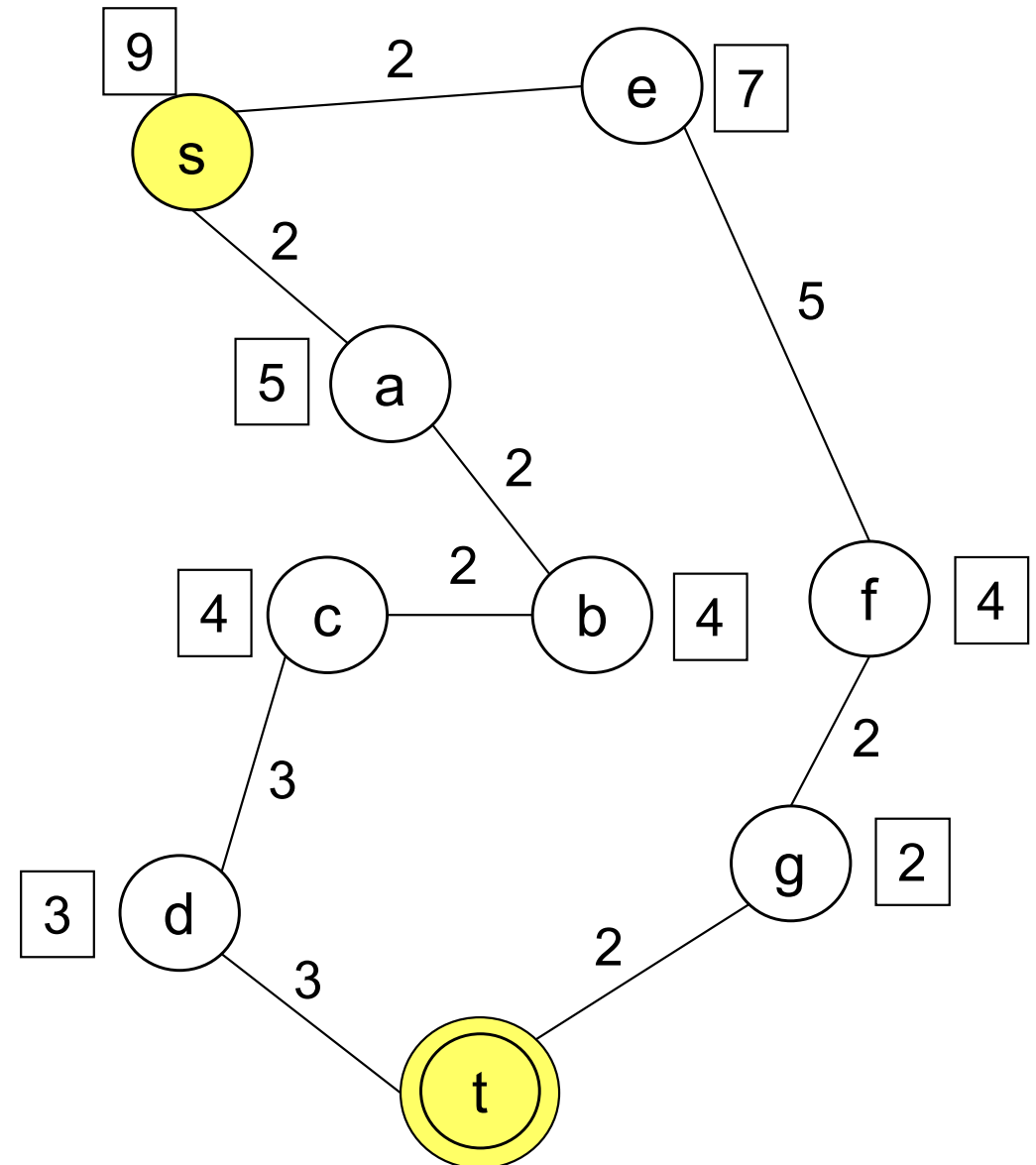
A*

- ❑ Dado um mapa, o objetivo é encontrar o caminho mais curto entre a cidade inicial **s** e a cidade destino **t**
- ❑ Para estimar o custo do caminho restante da cidade X até a cidade **t** utilizaremos a distância em linha reta denotada por **dist(X,t)**
- ❑ $f(X) = g(X) + h(X) =$
 $= g(X) + \text{dist}(X,t)$
- ❑ Note que quando X é a cidade inicial ($X=\mathbf{s}$) então $g(\mathbf{s})=0$
 - $f(\mathbf{s})=0+9=9$
- ❑ O primeiro nó a ser expandido é o nó inicial (raiz da busca)



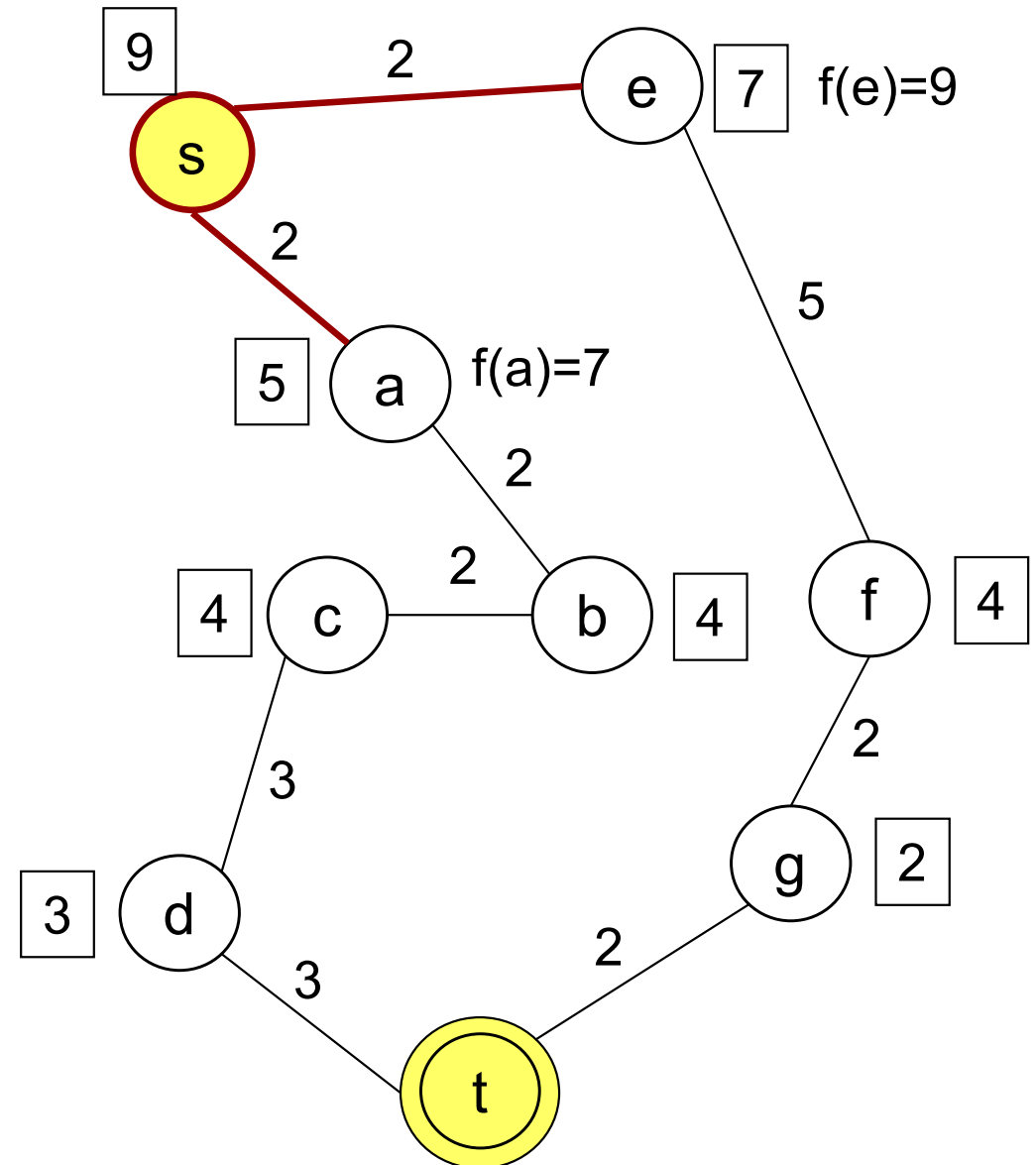
A*

- ❑ Neste exemplo, podemos imaginar a busca A* consistindo em dois processos, cada um explorando um dos caminhos alternativos
- ❑ Processo 1 explora o caminho via **a**
- ❑ Processo 2 explora o caminho via **e**



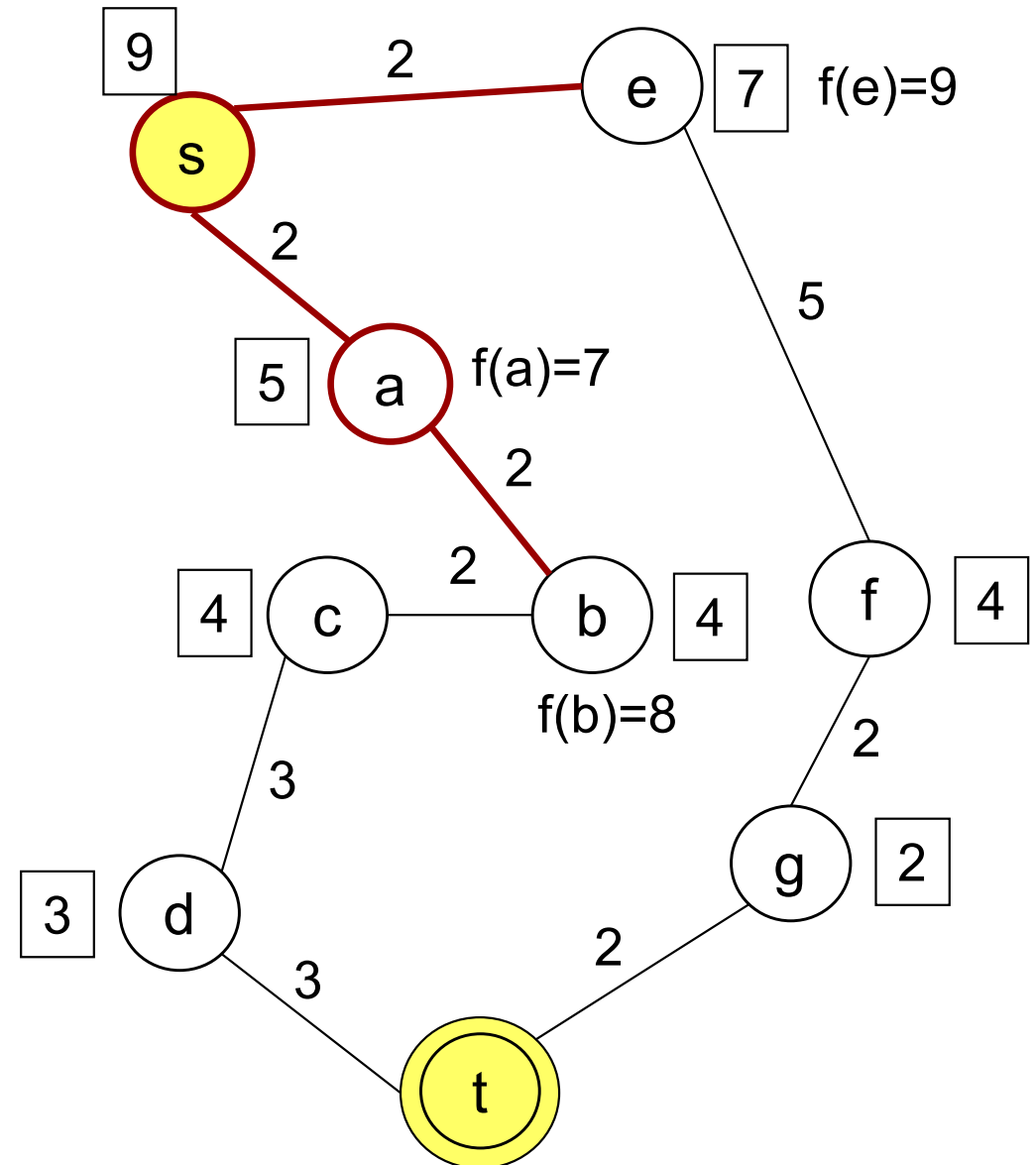
A*

- ❑ $f(a) = g(a) + \text{dist}(a, t) = 2 + 5 = 7$
- ❑ $f(e) = g(e) + \text{dist}(e, t) = 2 + 7 = 9$
- ❑ Como o valor-f de **a** é menor do que de **e**, o processo 1 (busca via **a**) permanece ativo enquanto o processo 2 (busca via **e**) fica em estado de espera



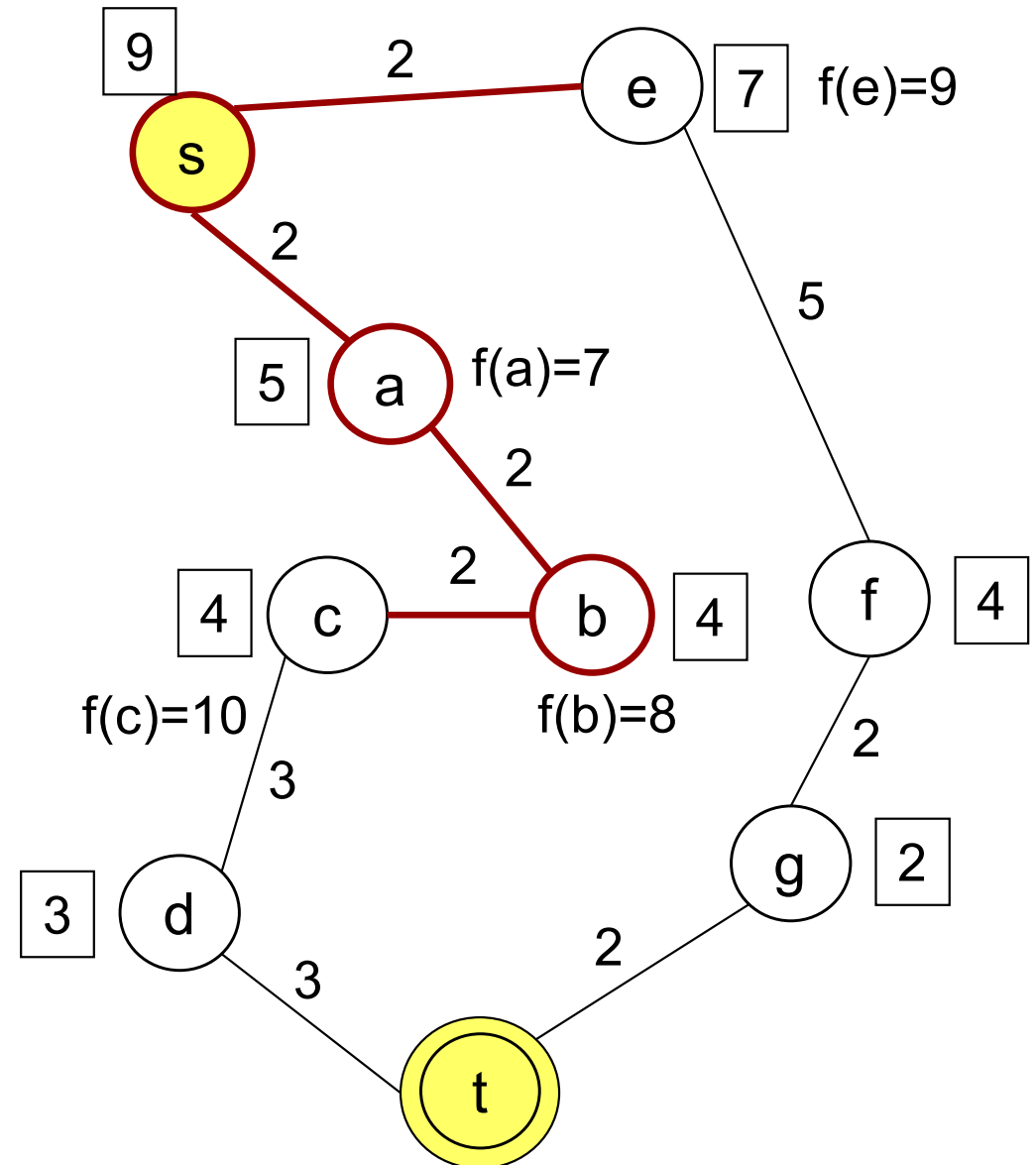
A*

- ❑ $f(a) = g(a) + \text{dist}(a, t) = 2 + 5 = 7$
- ❑ $f(e) = g(e) + \text{dist}(e, t) = 2 + 7 = 9$
- ❑ Como o valor-f de **a** é menor do que de **e**, o processo 1 (busca via **a**) permanece ativo enquanto o processo 2 (busca via **e**) fica em estado de espera
- ❑ $f(b) = g(b) + \text{dist}(b, t) = 4 + 4 = 8$



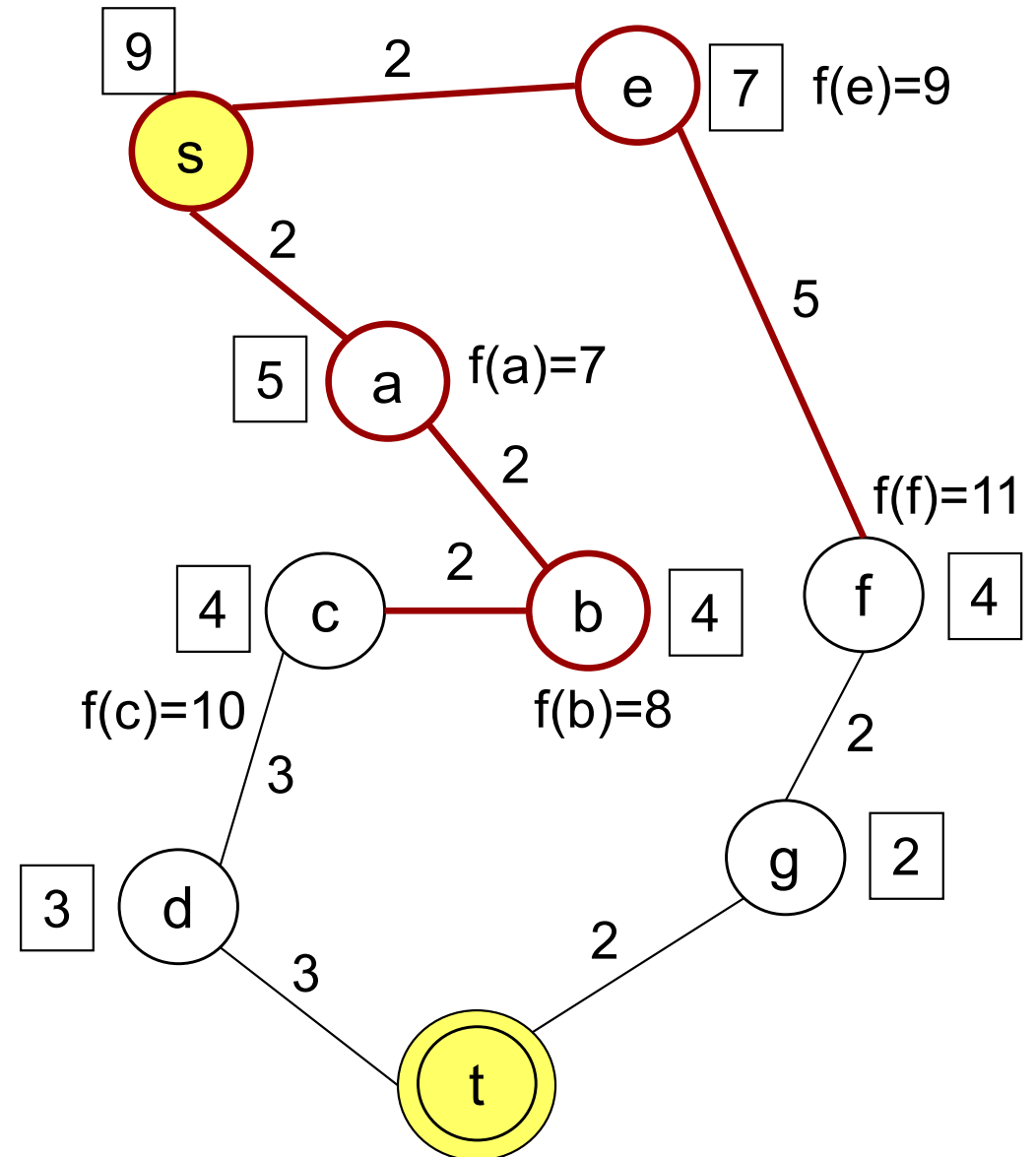
A*

- ❑ $f(a) = g(a) + \text{dist}(a, t) = 2 + 5 = 7$
- ❑ $f(e) = g(e) + \text{dist}(e, t) = 2 + 7 = 9$
- ❑ Como o valor- f de **a** é menor do que de **e**, o processo 1 (busca via **a**) permanece ativo enquanto o processo 2 (busca via **e**) fica em estado de espera
- ❑ $f(b) = g(b) + \text{dist}(b, t) = 4 + 4 = 8$
- ❑ $f(c) = g(c) + \text{dist}(c, t) = 6 + 4 = 10$
- ❑ Como $f(e) < f(c)$ agora o processo 2 prossegue para a cidade **f**



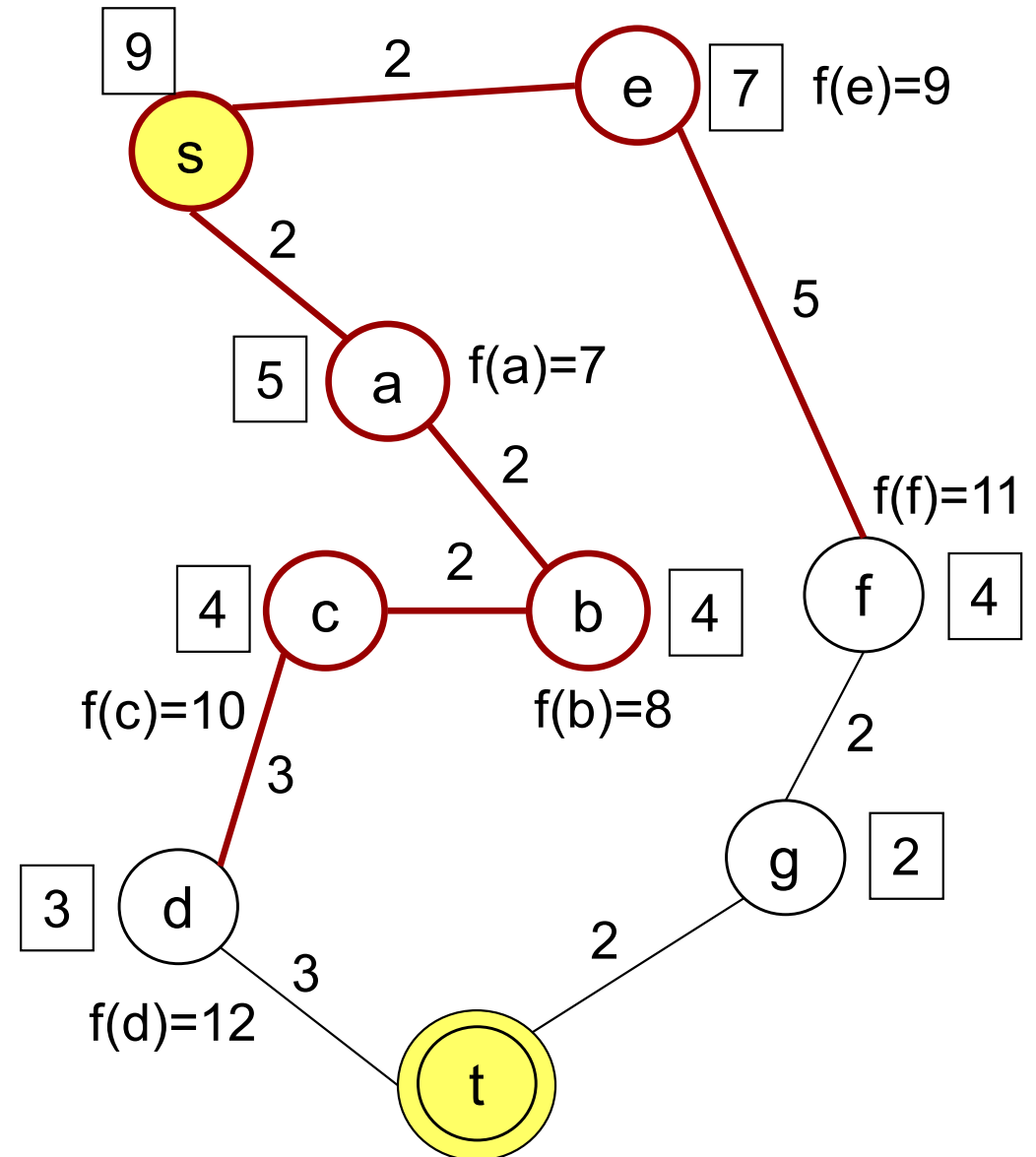
A*

- ❑ $f(f) = g(f) + \text{dist}(f, t) = 7 + 4 = 11$
- ❑ Como $f(f) > f(c)$ agora o processo 2 espera e o processo 1 prossegue



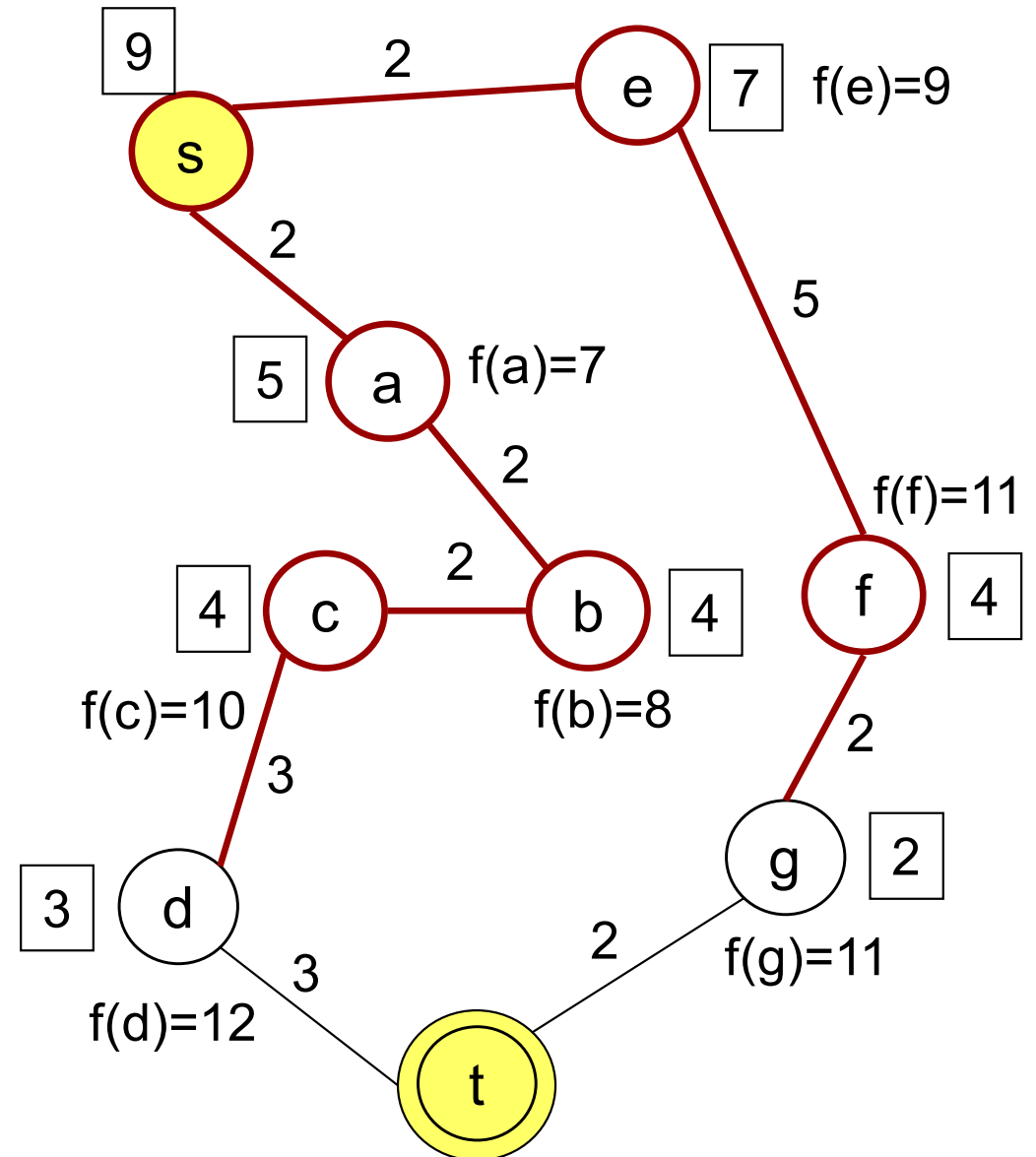
A*

- ❑ $f(f) = g(f) + \text{dist}(f, t) = 7 + 4 = 11$
- ❑ Como $f(f) > f(c)$ agora o processo 2 espera e o processo 1 prossegue
- ❑ $f(d) = g(d) + \text{dist}(d, t) = 9 + 3 = 12$
- ❑ Como $f(d) > f(f)$ o processo 2 reinicia



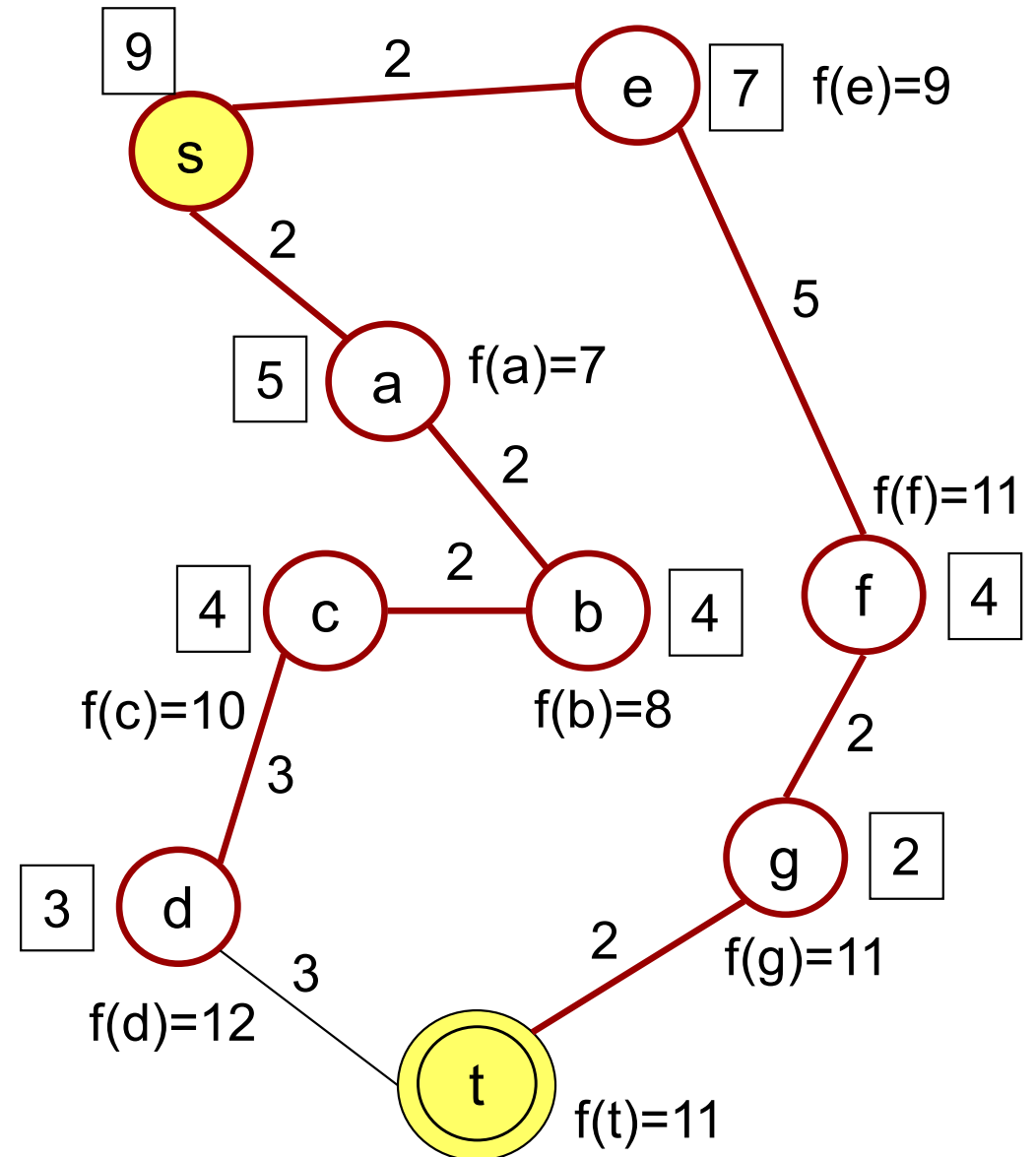
A*

- ❑ $f(f) = g(f) + \text{dist}(f, t) = 7 + 4 = 11$
- ❑ Como $f(f) > f(c)$ agora o processo 2 espera e o processo 1 prossegue
- ❑ $f(d) = g(d) + \text{dist}(d, t) = 9 + 3 = 12$
- ❑ Como $f(d) > f(f)$ o processo 2 reinicia chegando até o destino **t**
- ❑ $f(g) = g(g) + \text{dist}(g, t) = 9 + 2 = 11$



A*

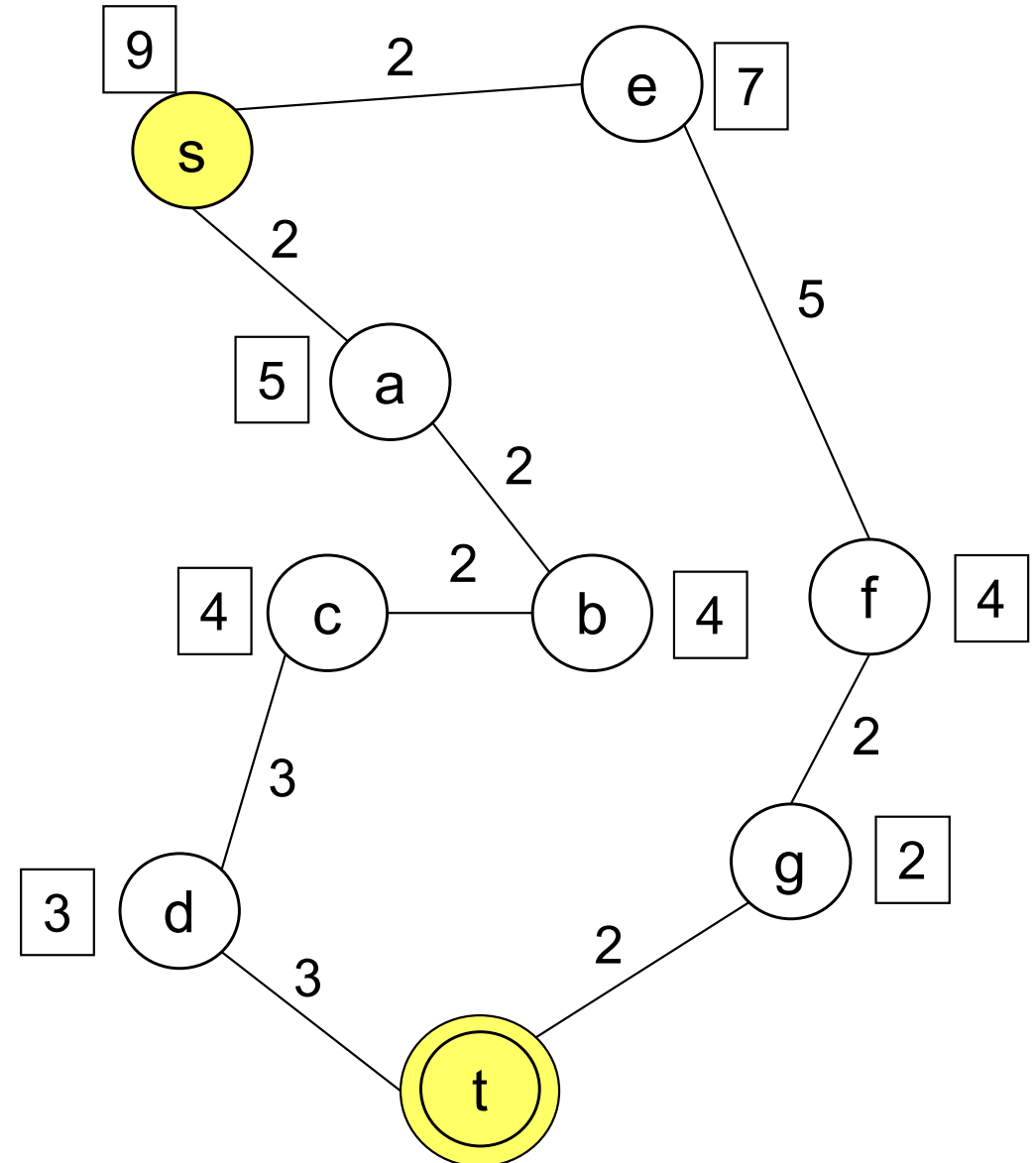
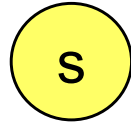
- ❑ $f(f) = g(f) + \text{dist}(f, t) = 7 + 4 = 11$
- ❑ Como $f(f) > f(c)$ agora o processo 2 espera e o processo 1 prossegue
- ❑ $f(d) = g(d) + \text{dist}(d, t) = 9 + 3 = 12$
- ❑ Como $f(d) > f(f)$ o processo 2 reinicia chegando até o destino **t**
- ❑ $f(g) = g(g) + \text{dist}(g, t) = 9 + 2 = 11$
- ❑ $f(t) = g(t) + \text{dist}(t, t) = 11 + 0 = 11$



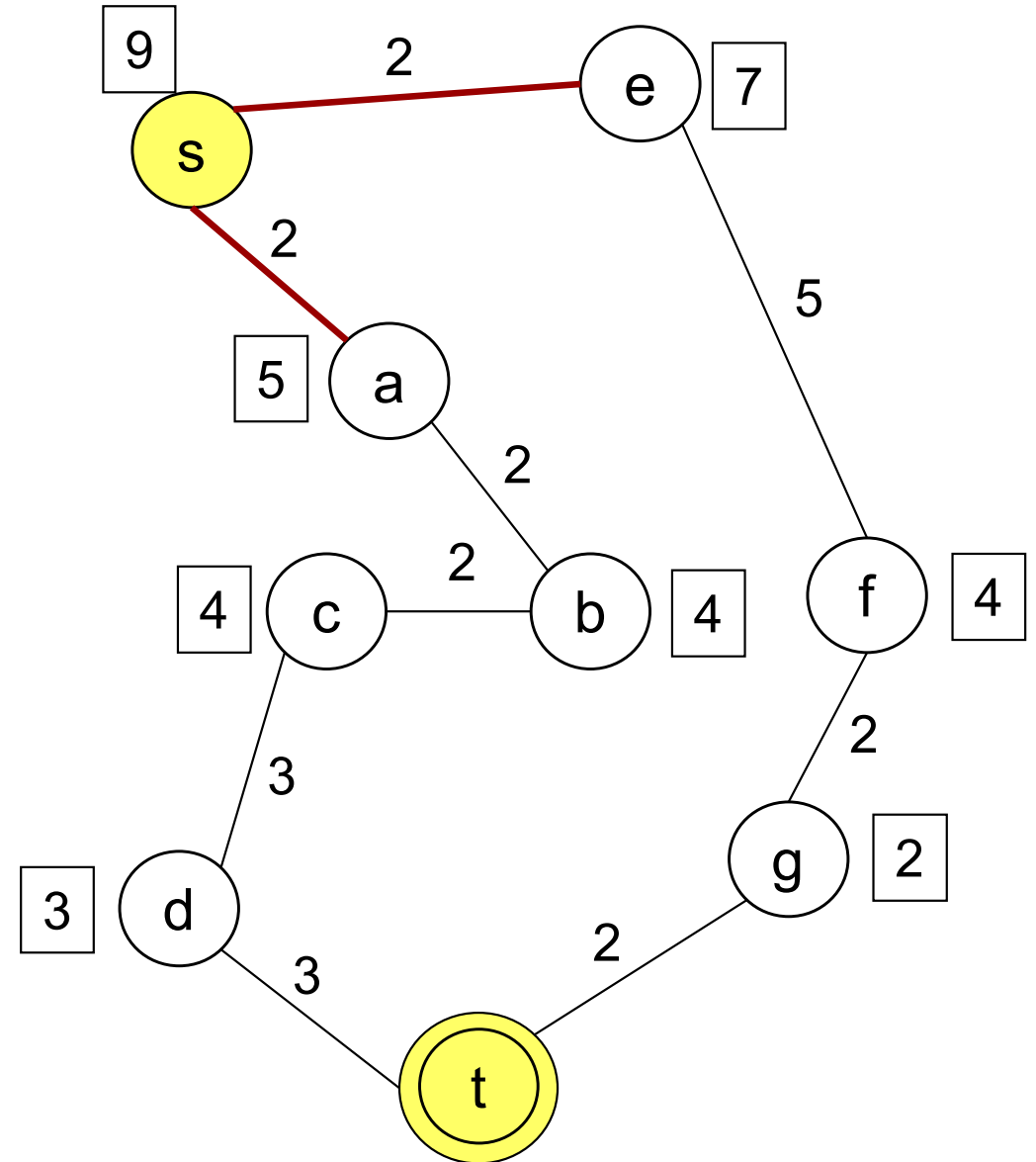
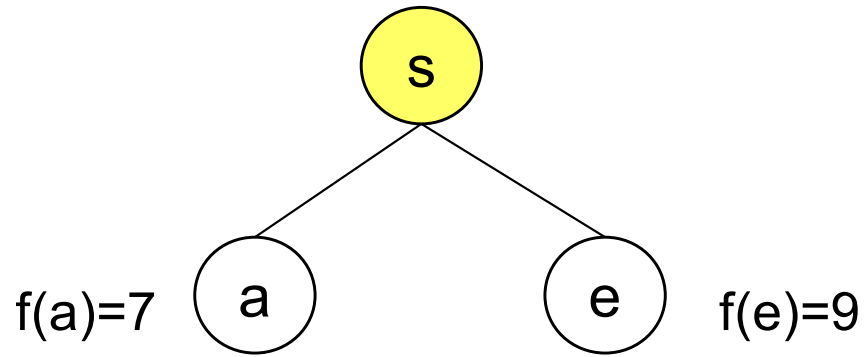
A*

- ❑ A busca, começando pelo nó inicial continua gerando novos nós sucessores, sempre expandindo na direção mais promissora de acordo com os valores-f
- ❑ Durante este processo, uma árvore de busca é gerada tendo como raiz o nó inicial e o algoritmo A* continua expandindo a árvore de busca até que uma solução seja encontrada

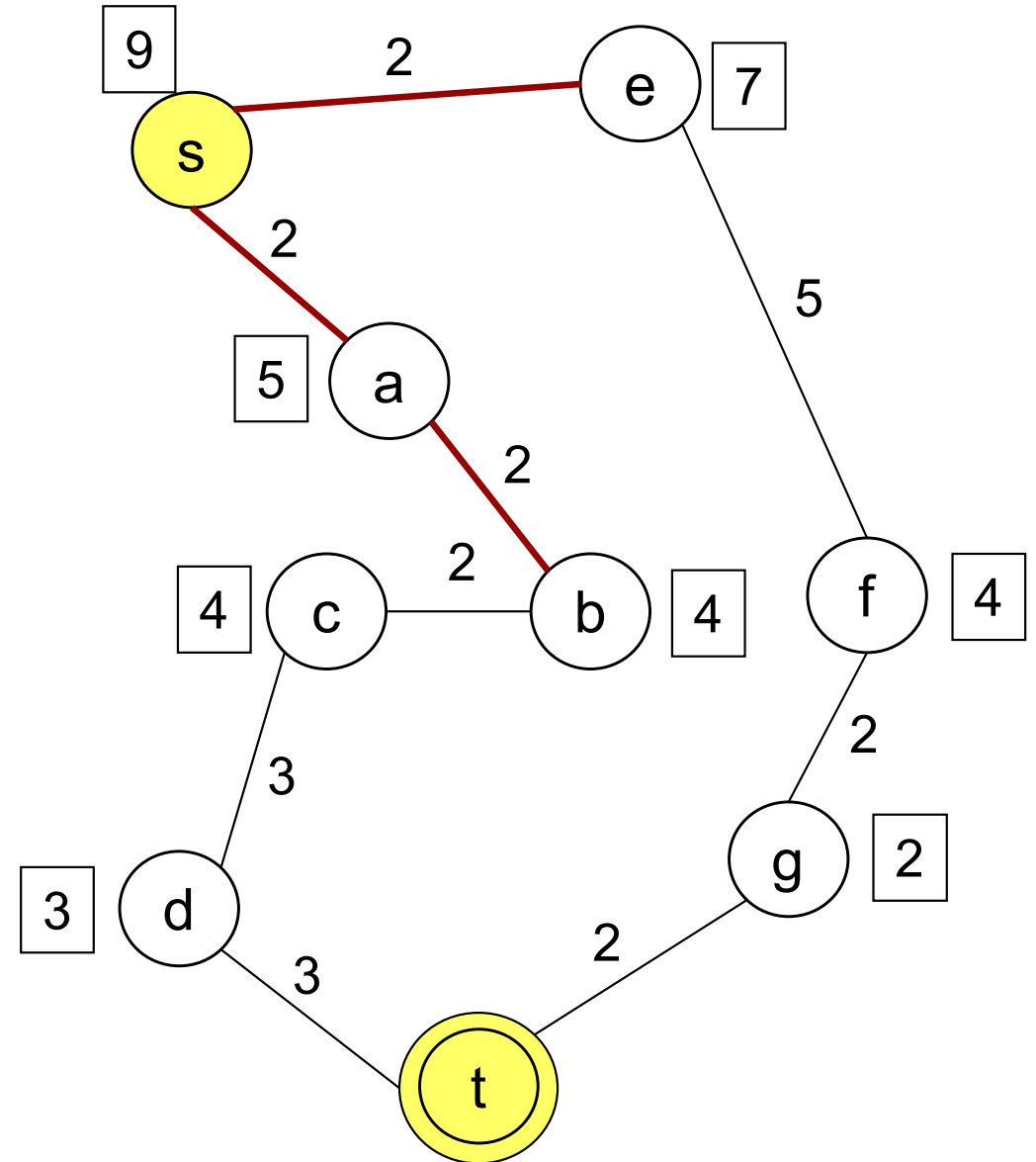
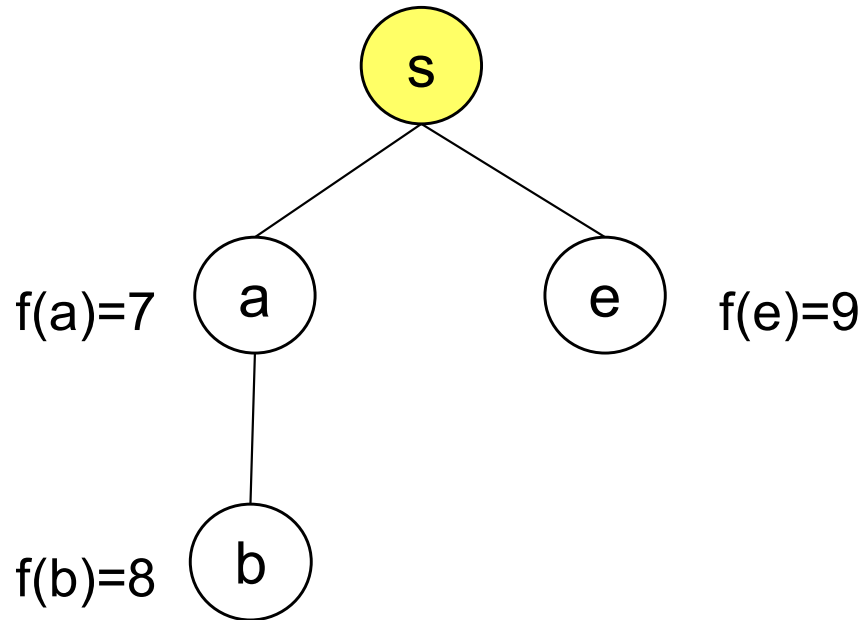
A^*



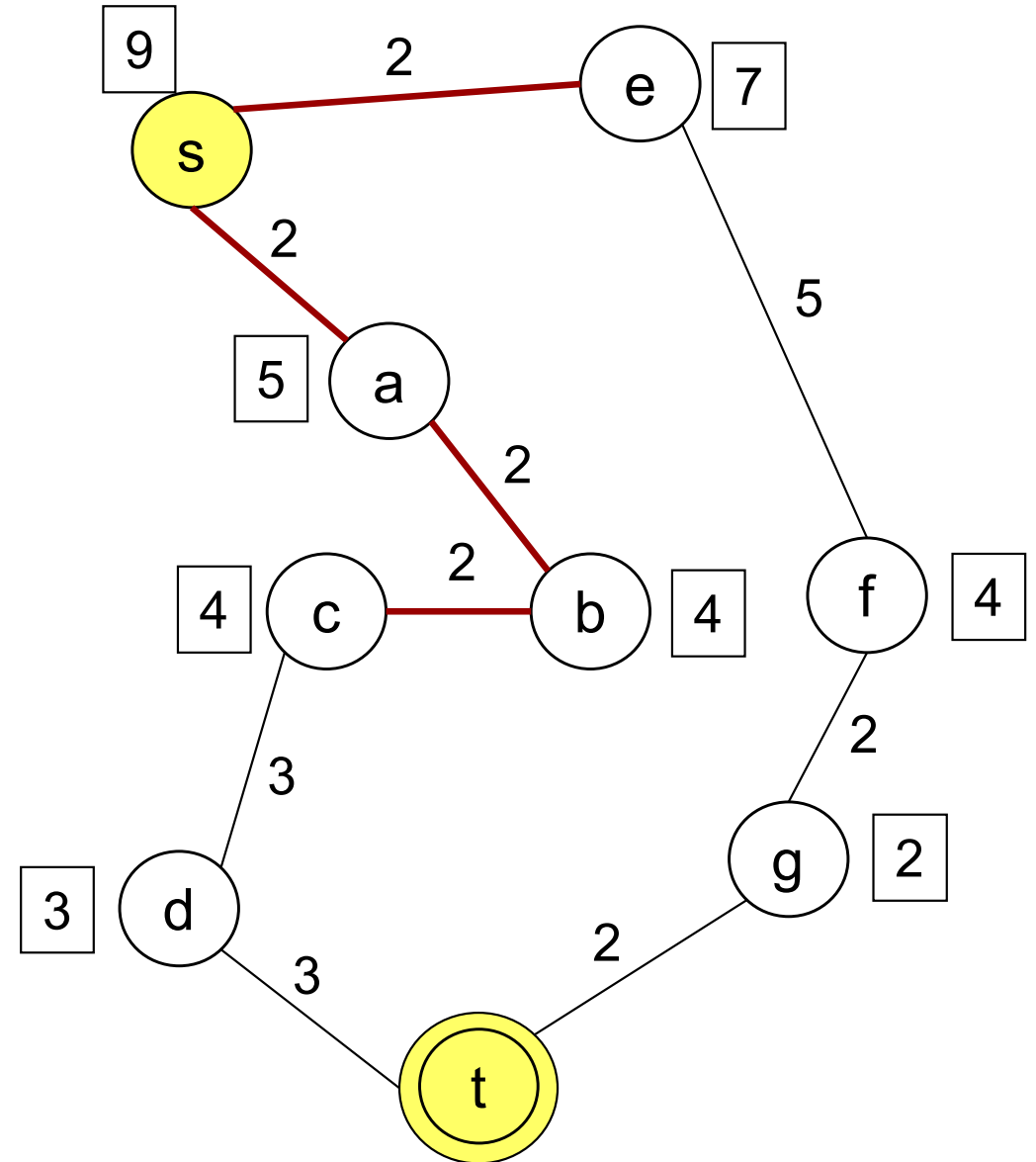
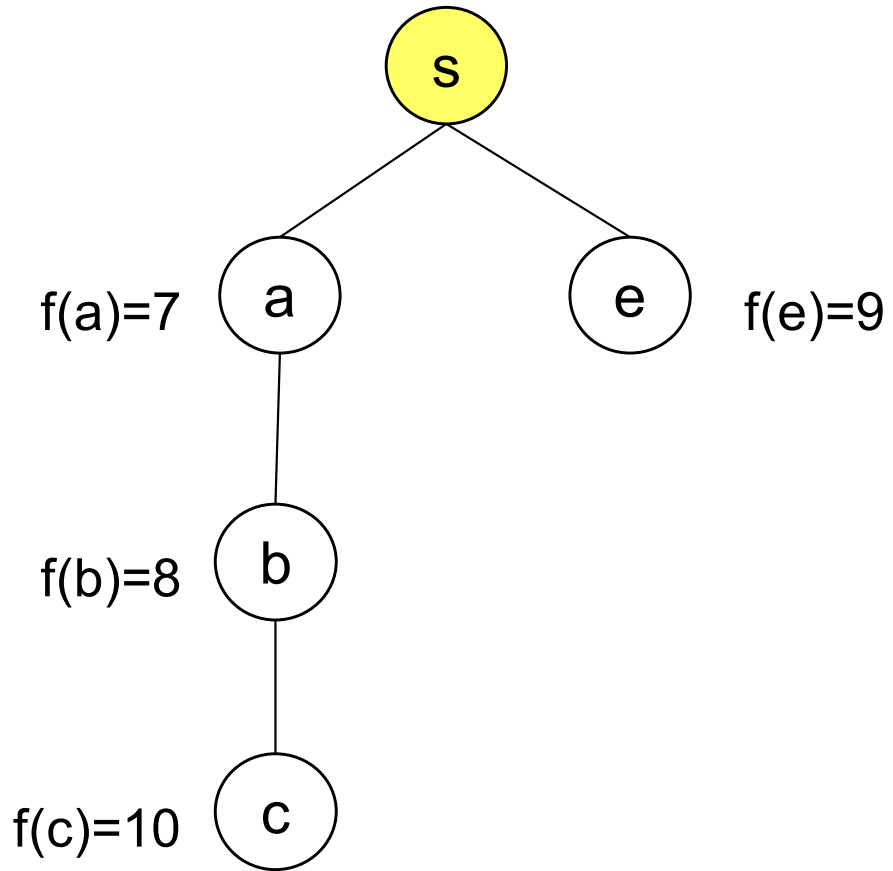
A*



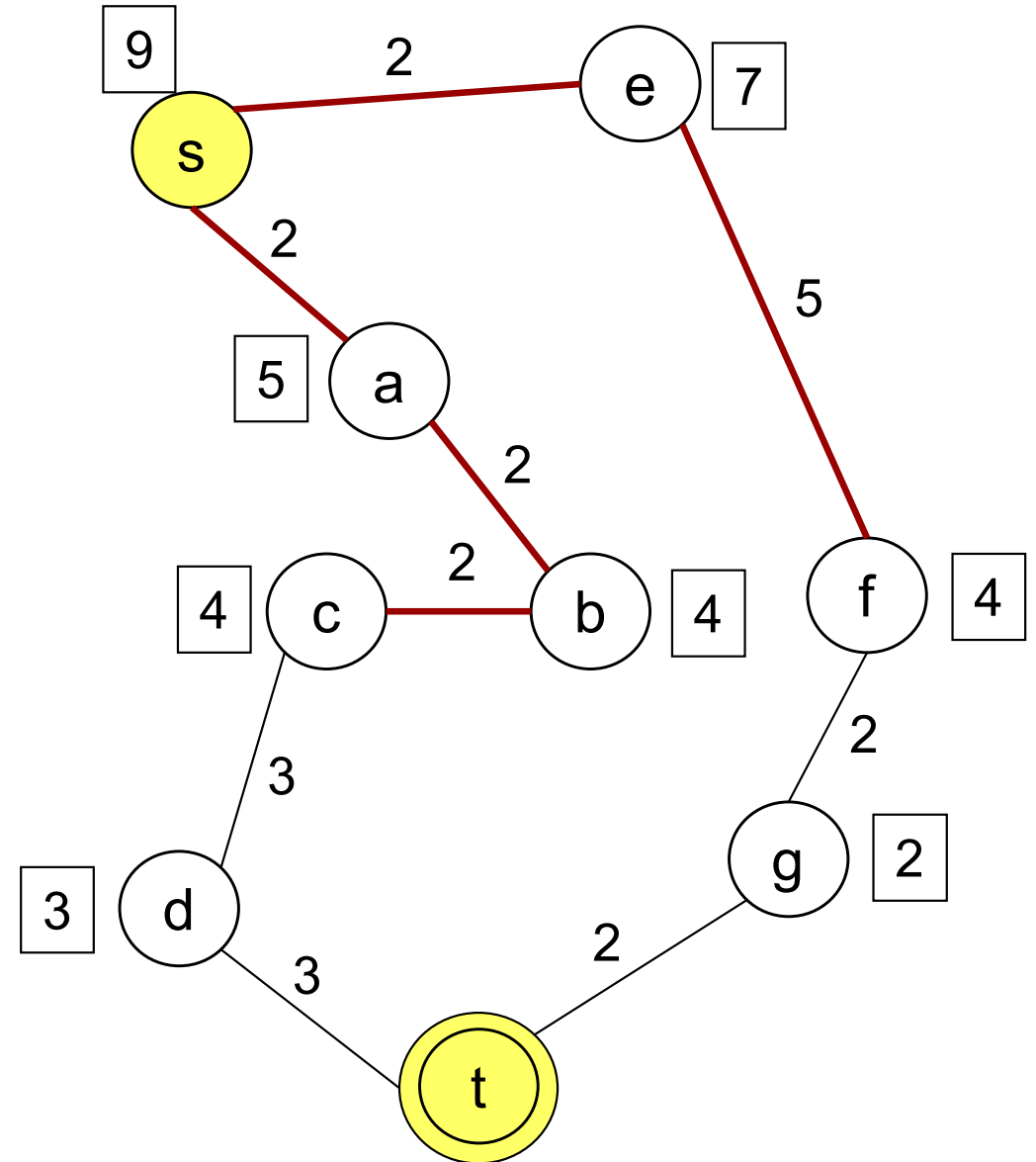
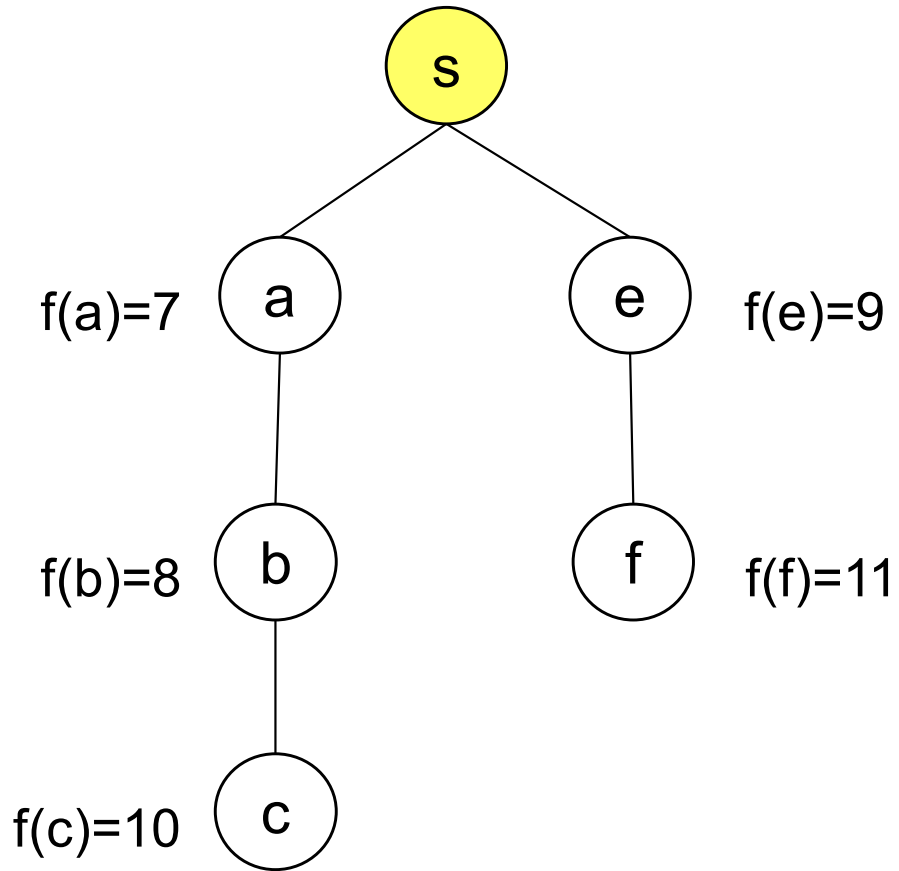
A*



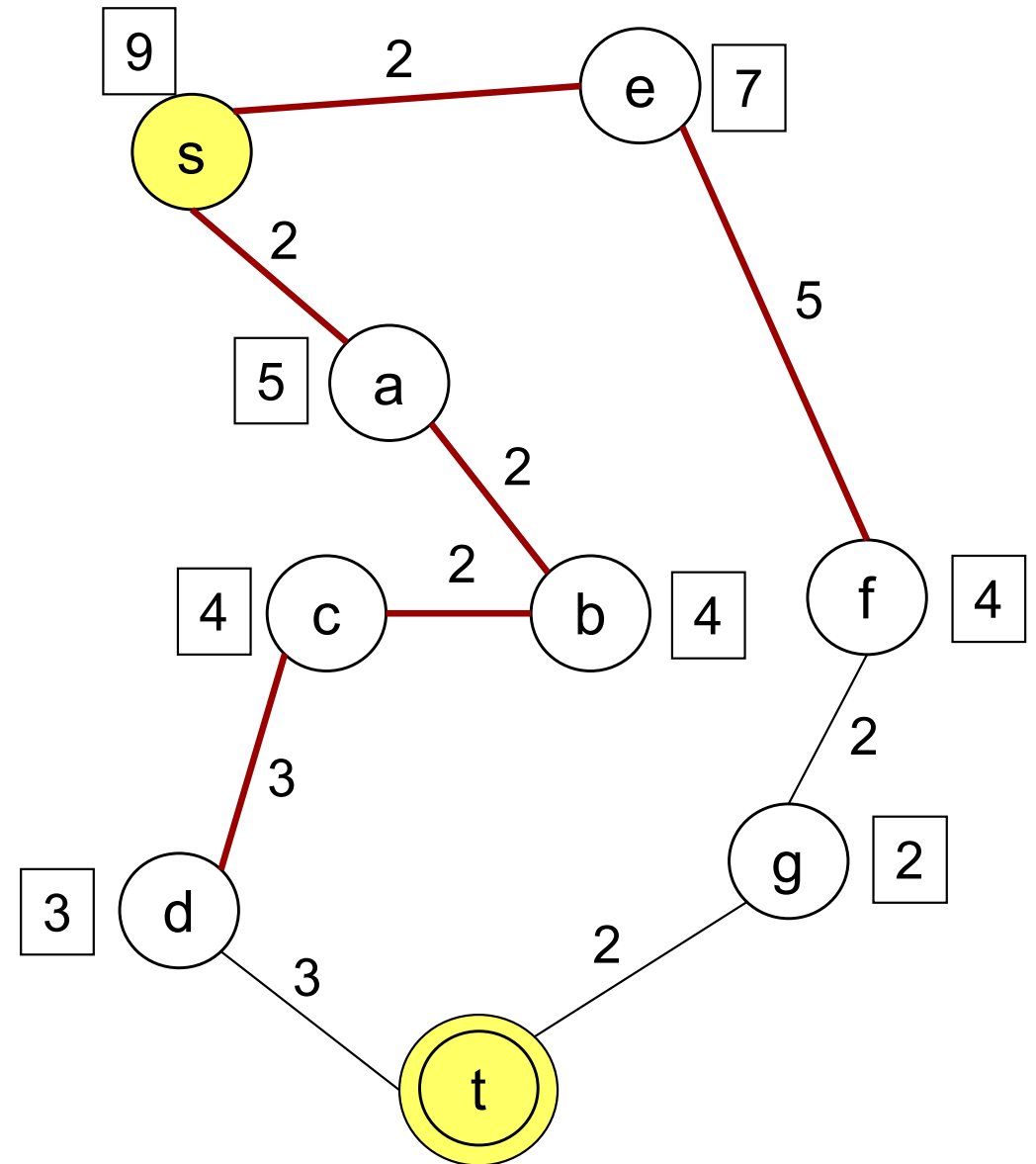
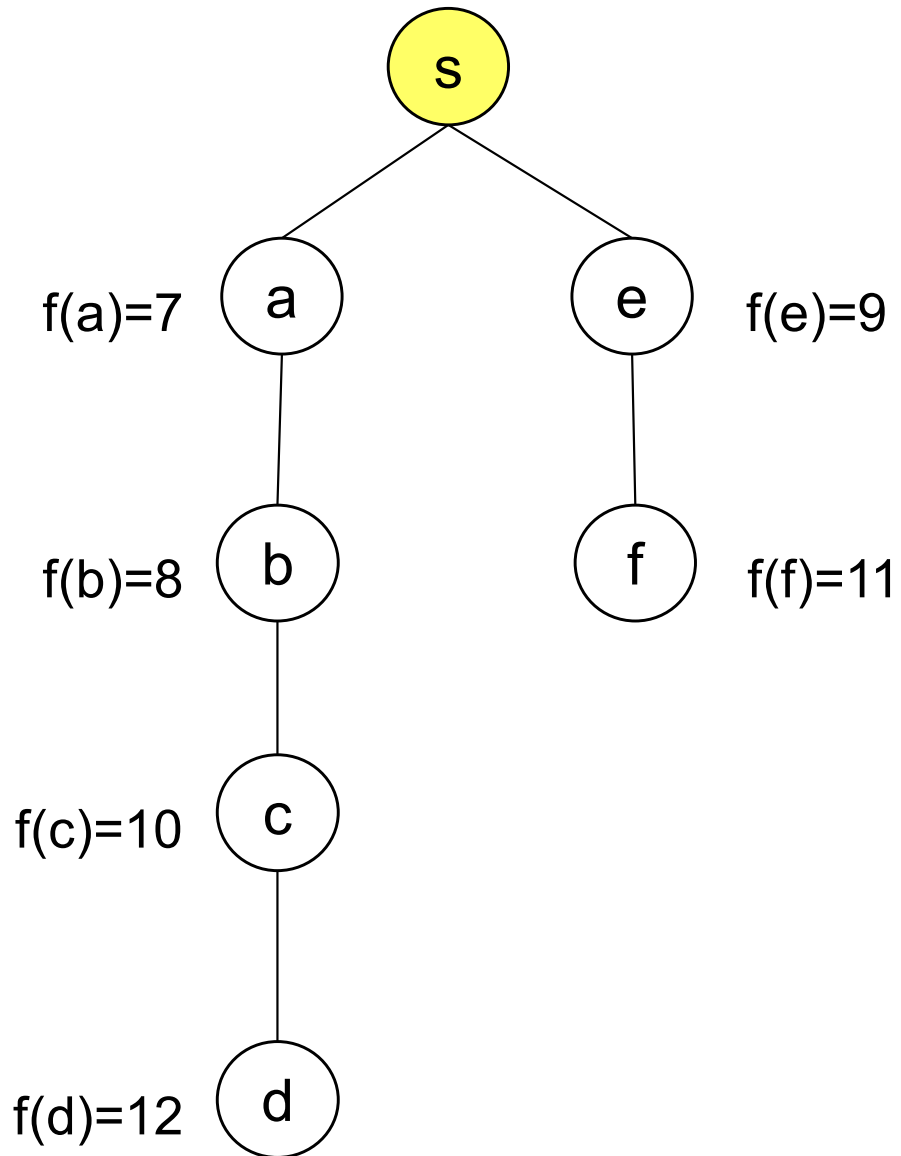
A*



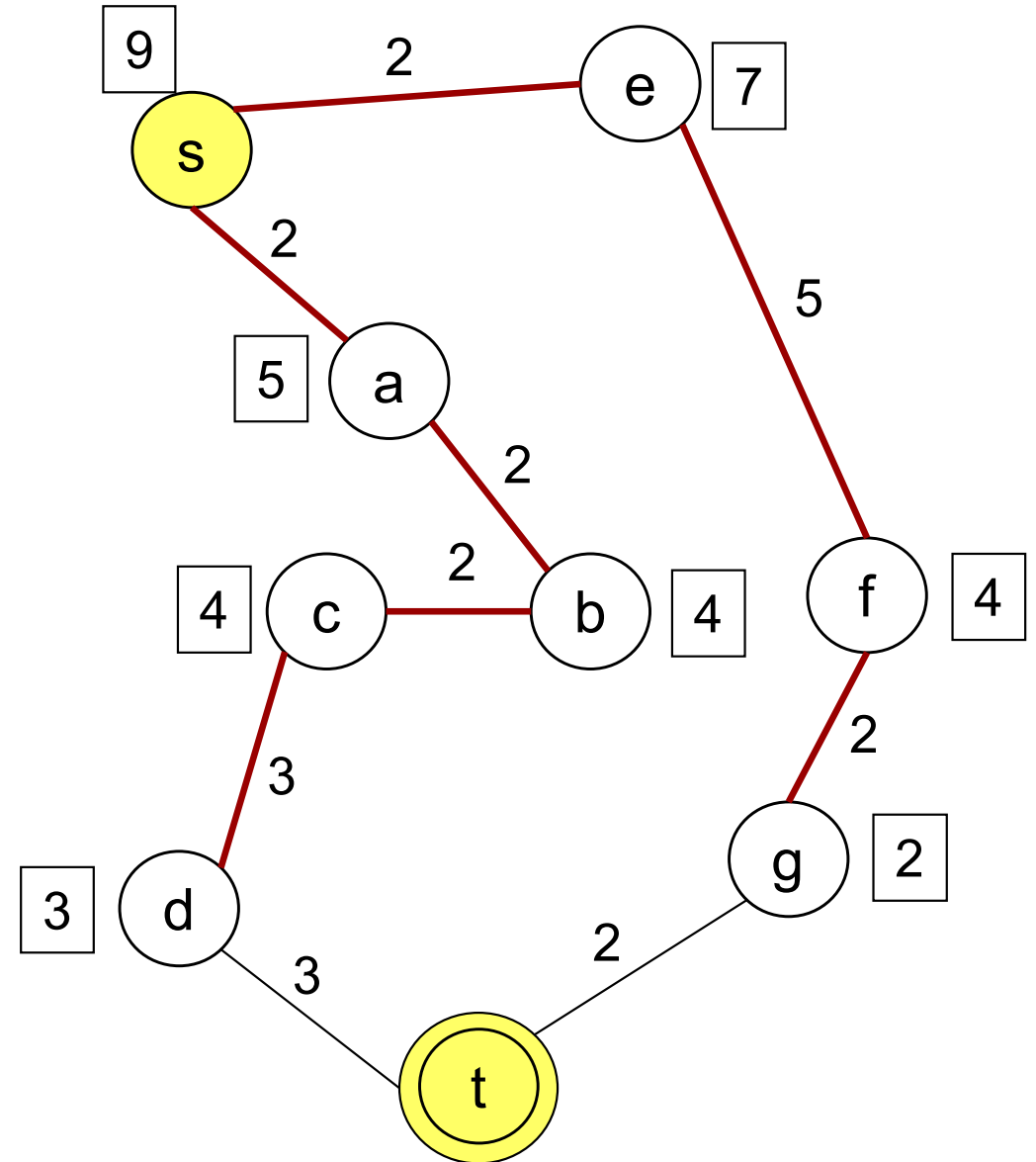
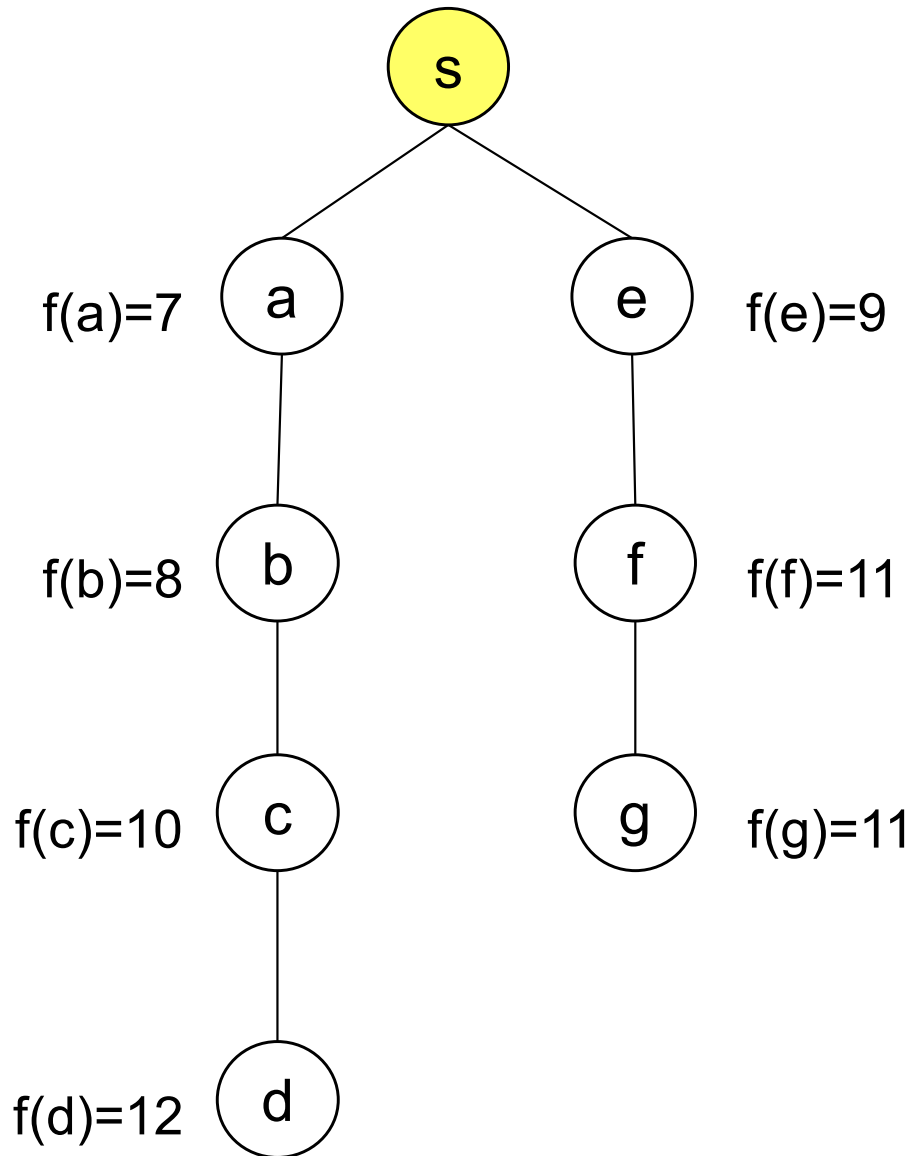
A*



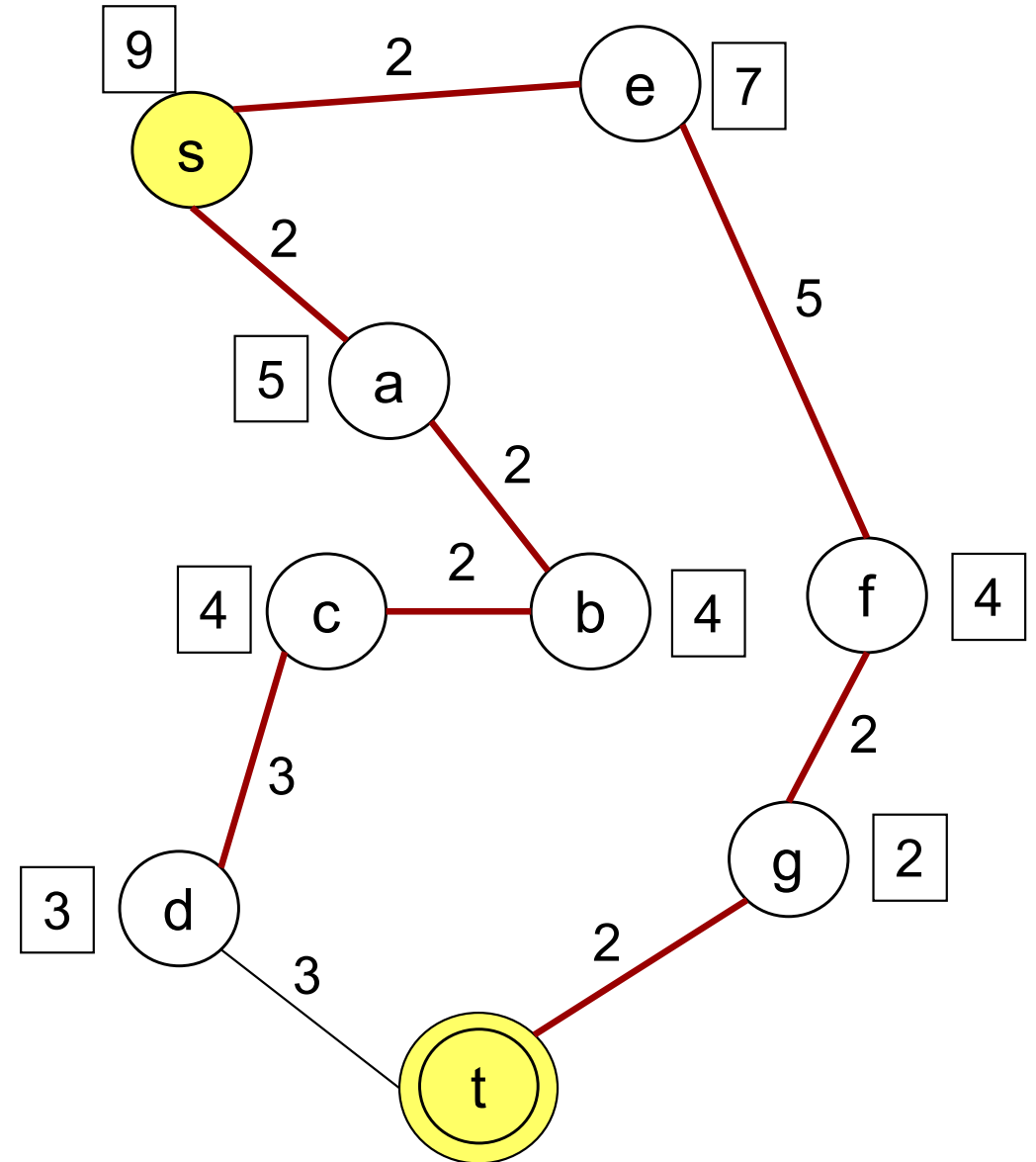
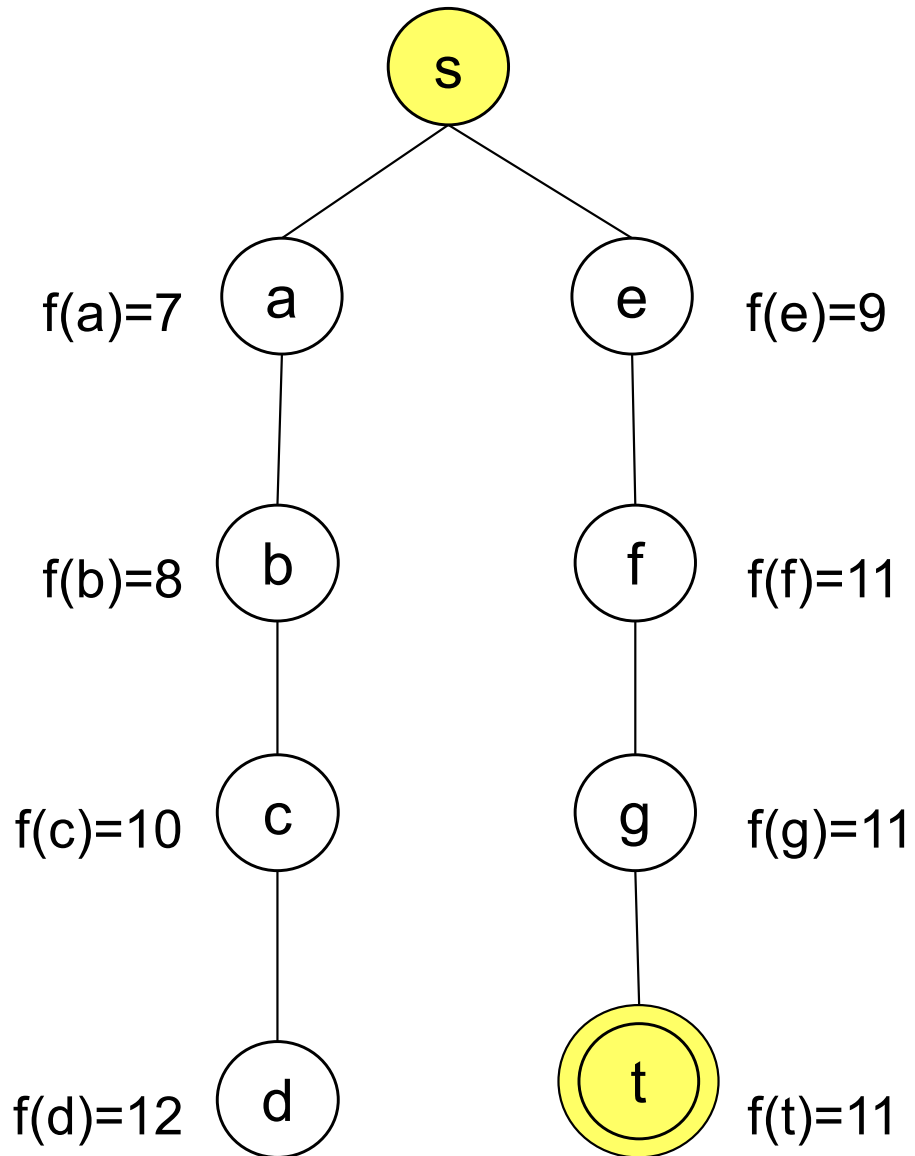
A*



A*

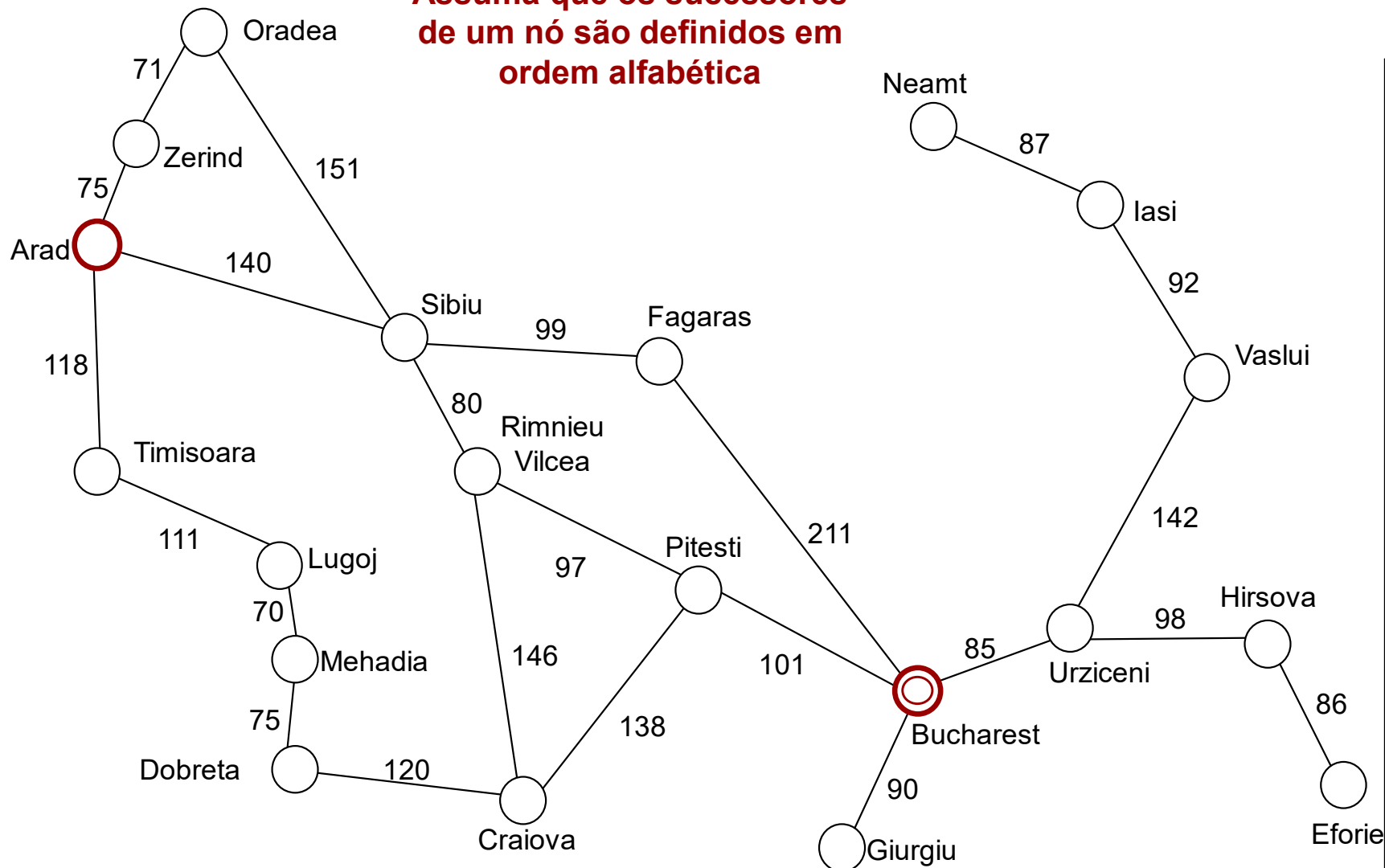


A*



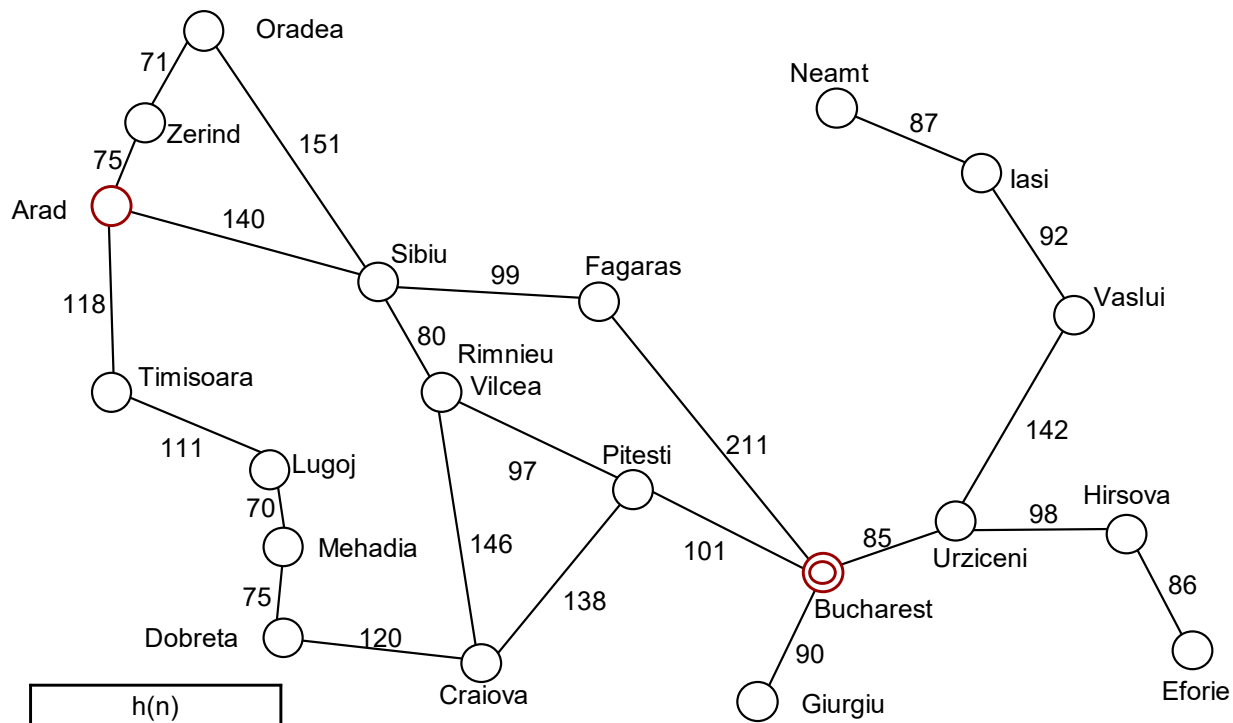
A*: Encontre o caminho de Arad até Bucharest

Assuma que os sucessores de um nó são definidos em ordem alfabética



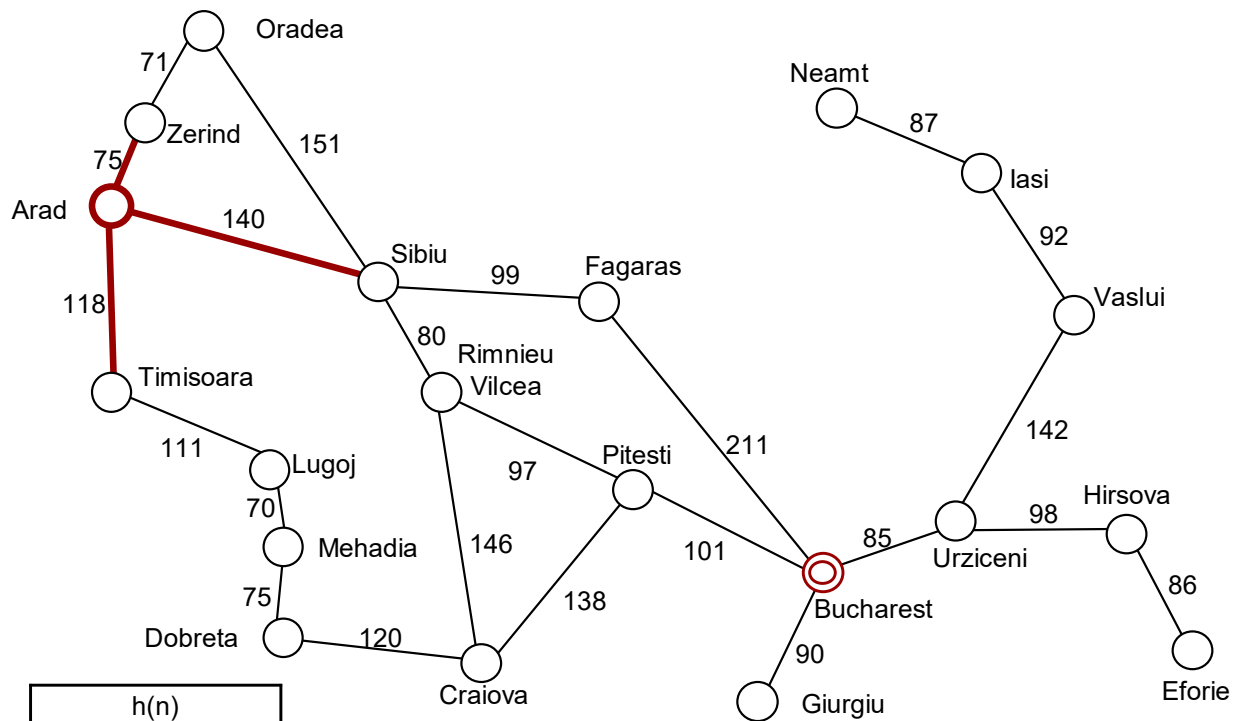
Distância em linha reta até Bucharest

h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnieniu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

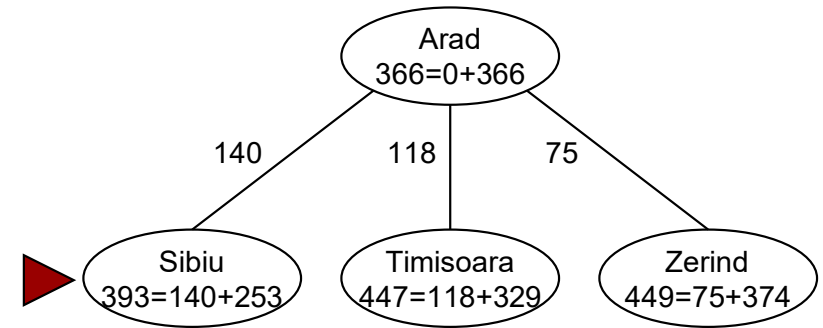


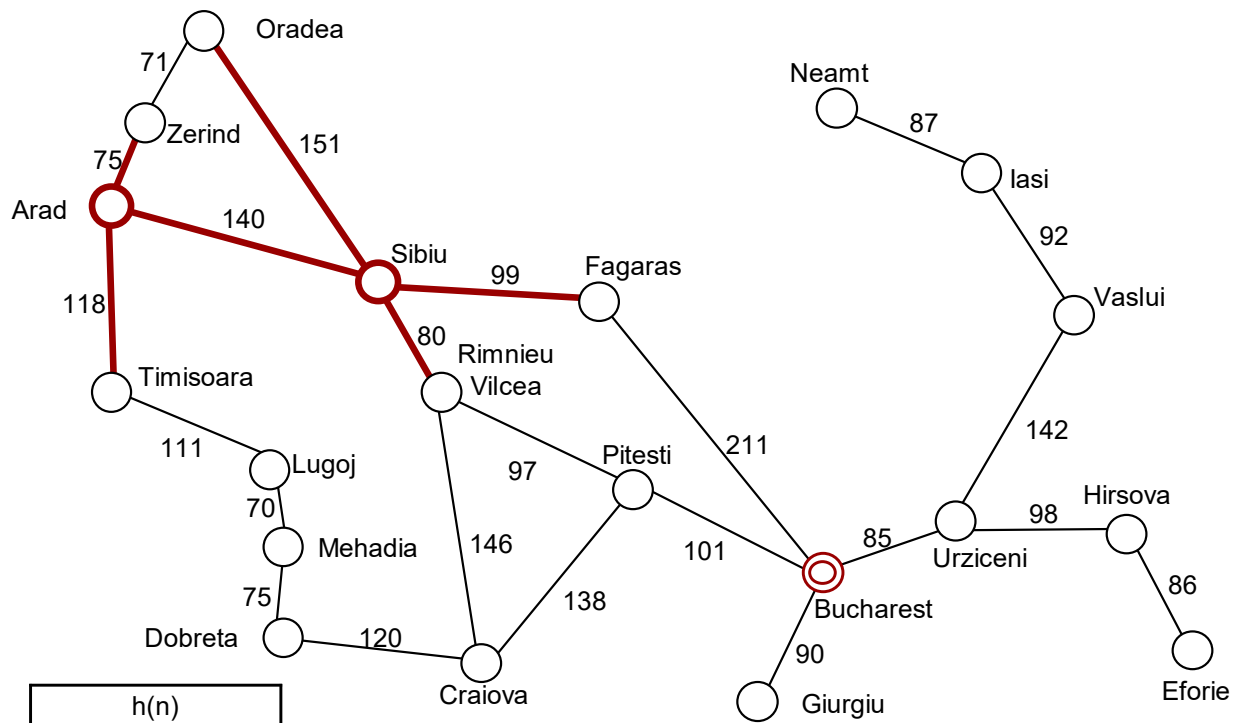
► Arad
366=0+366

h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnieniu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

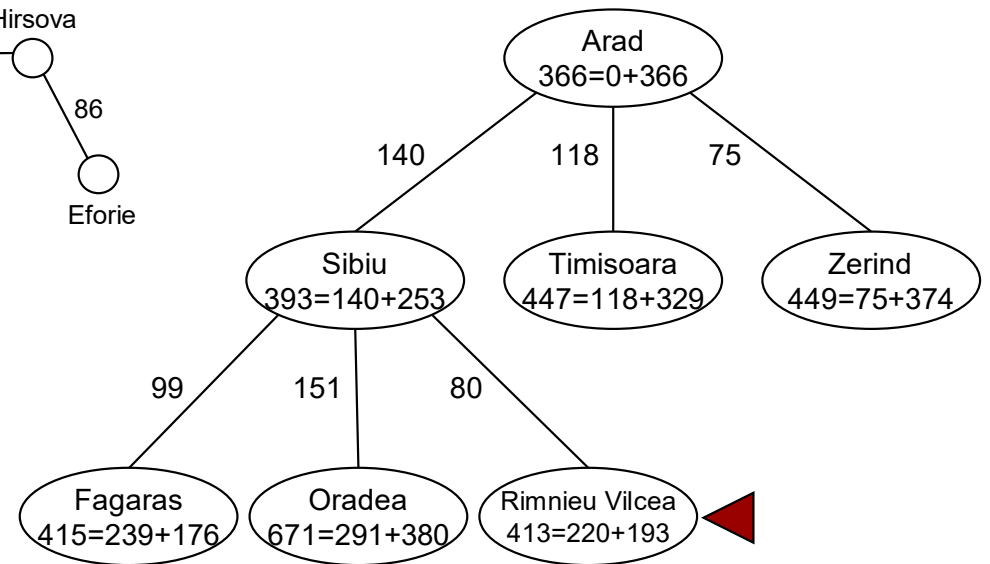


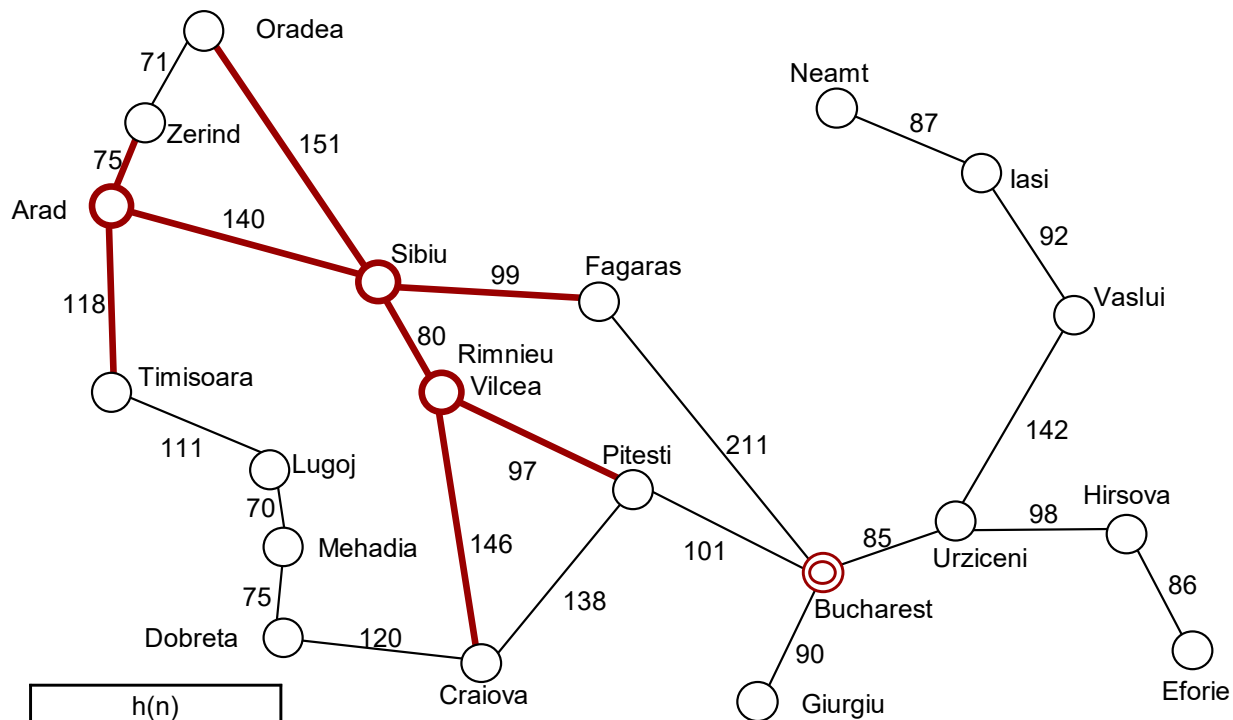
h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnieniu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



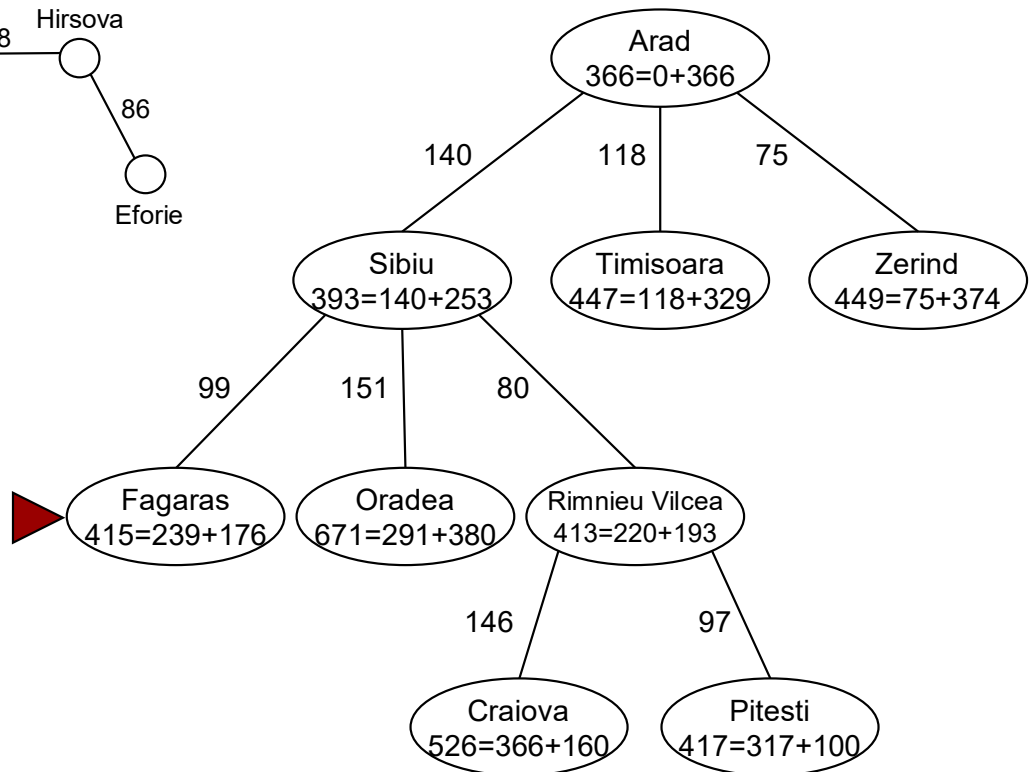


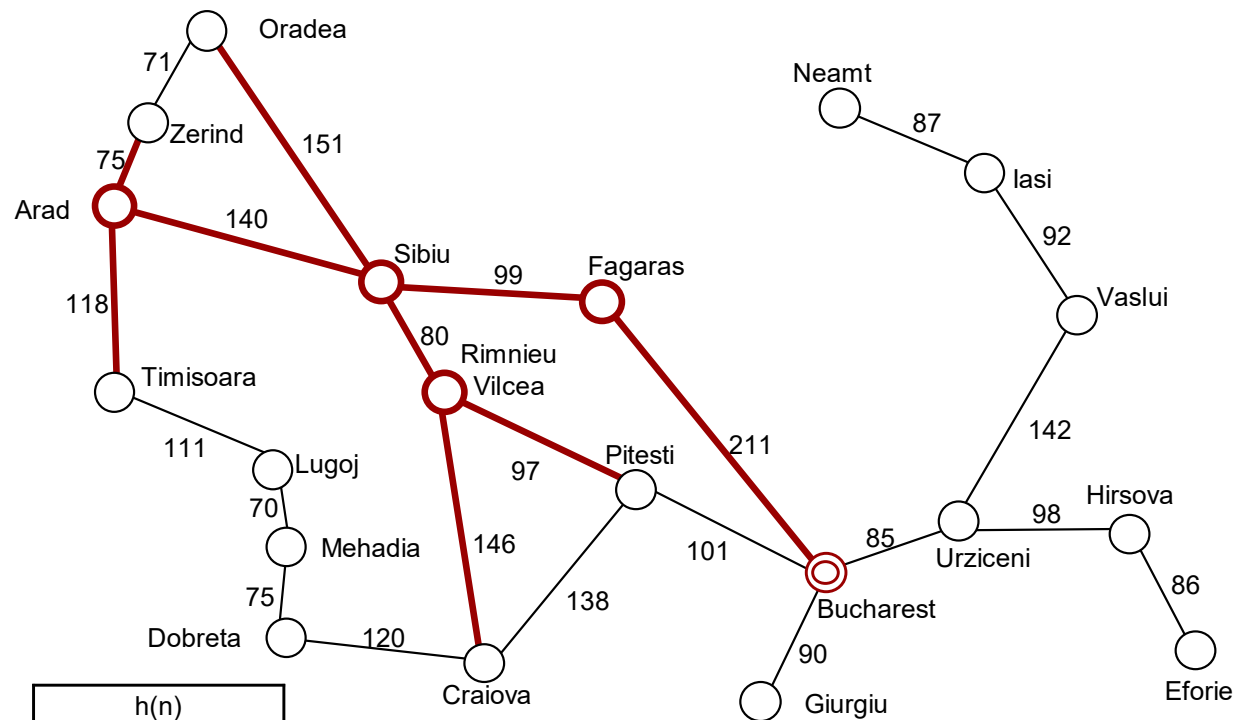
h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnieniu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



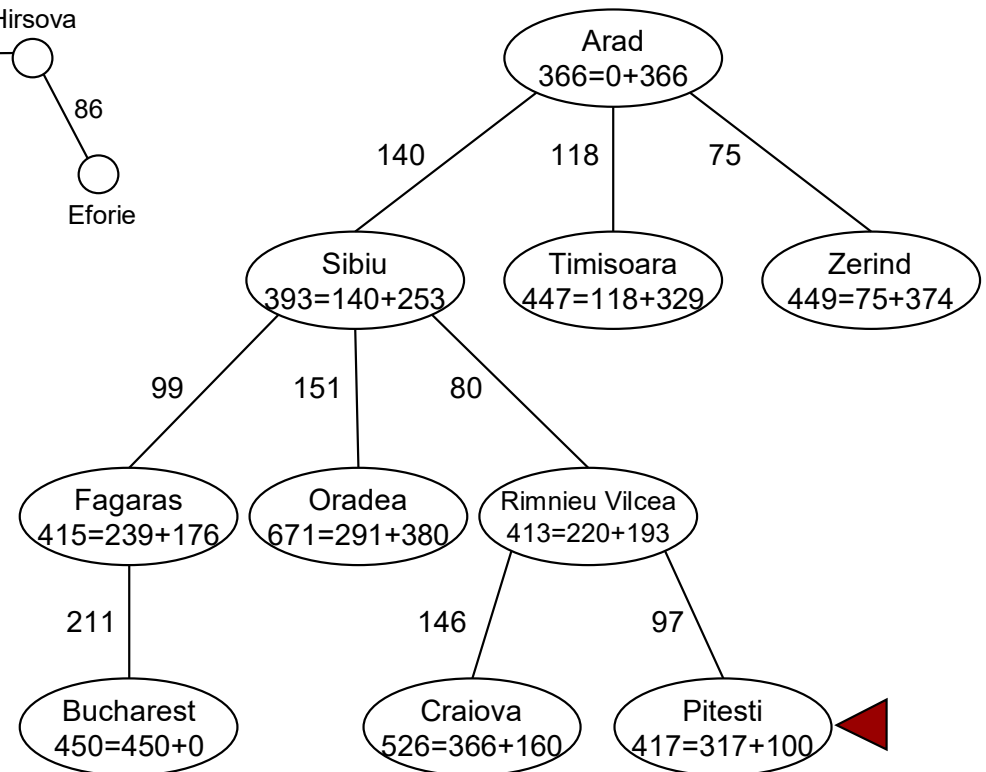


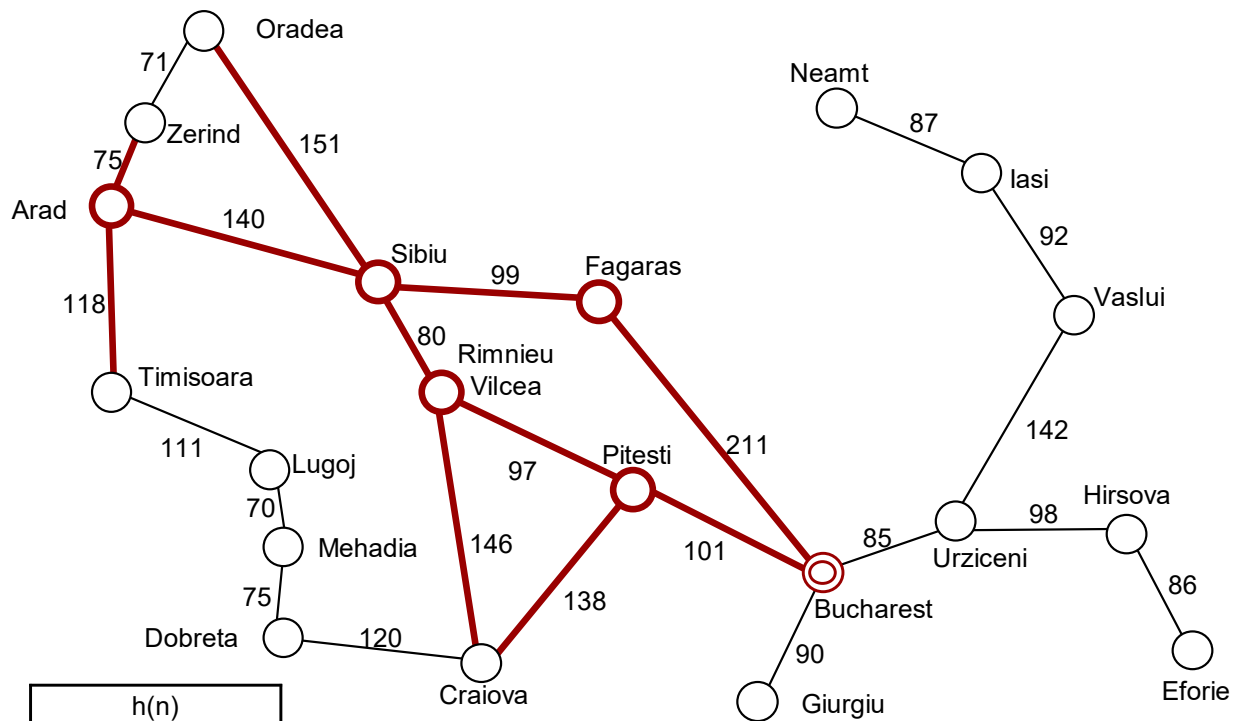
h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnien Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



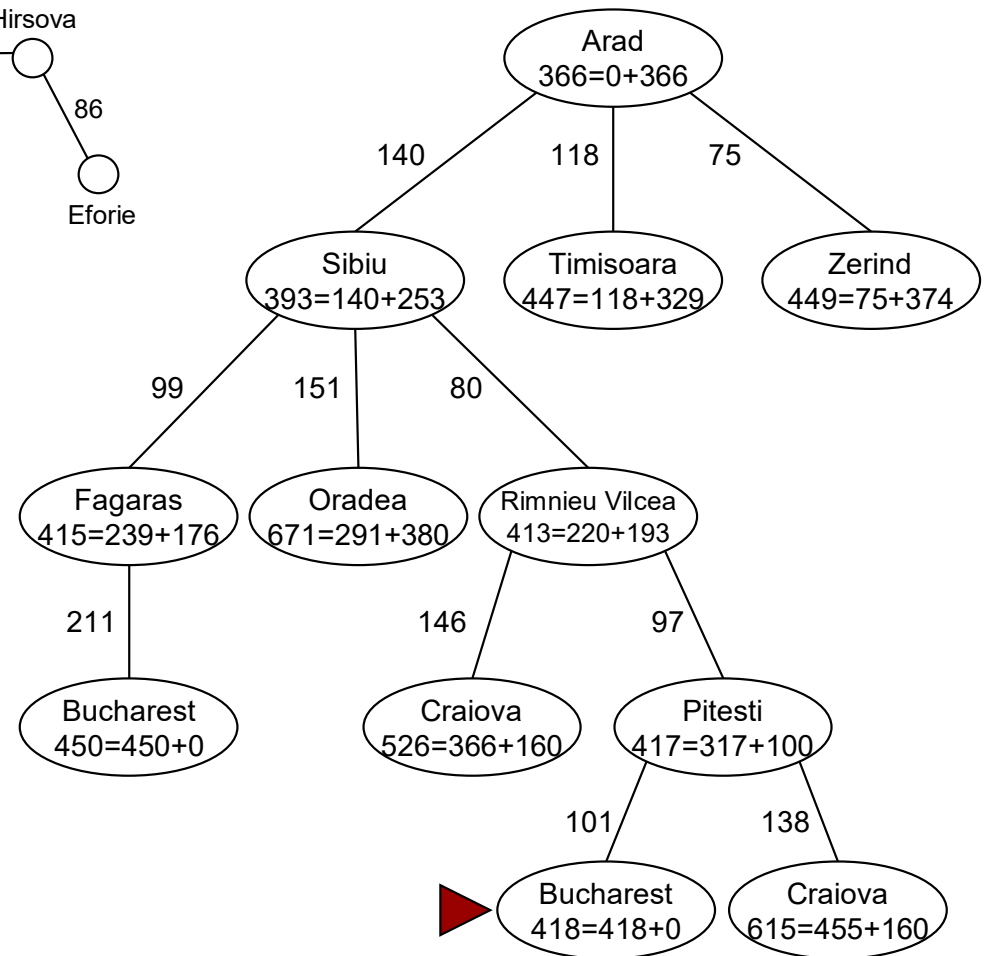


h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnien Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

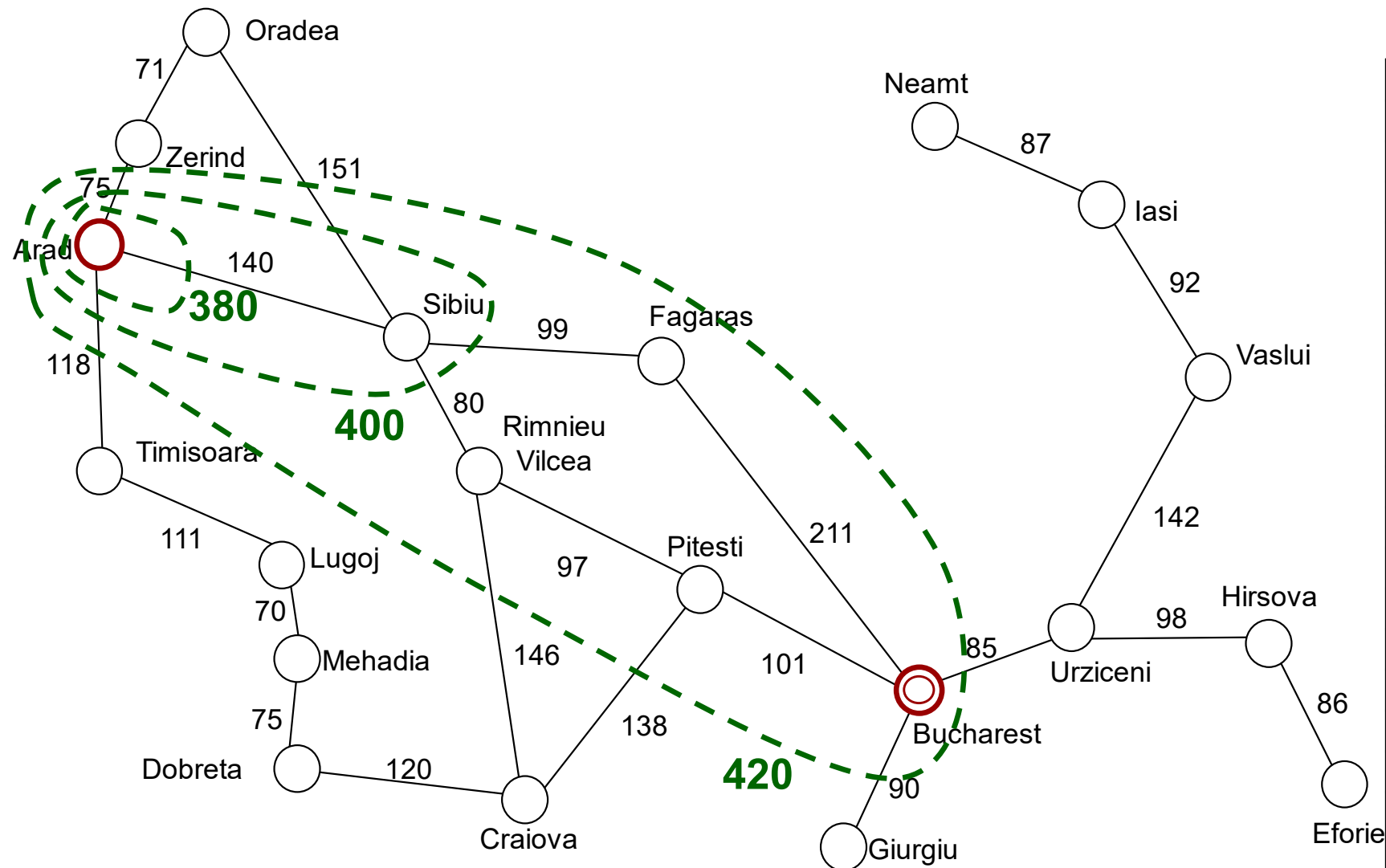




h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnien Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



A*: Contornos em $f=380$, $f=400$ e $f=420$ (nós no interior do contorno têm valores- f menores ou iguais aos do contorno)



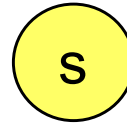
Distância em linha reta até Bucharest

h(n)	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnien Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A*

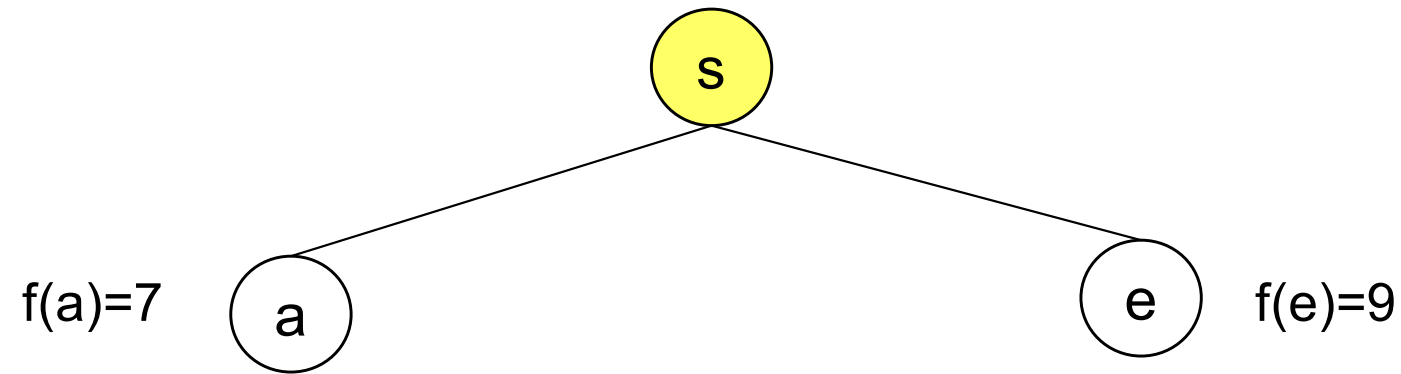
- ❑ A árvore de busca será representada de duas formas:
 - $I(N, F/G)$ representa um único nó folha (*leaf*)
 - ❖ N é um nó do espaço de estados
 - ❖ G é $g(N)$, custo do caminho encontrado desde o nó inicial até N
 - ❖ F é $f(N) = G + h(N)$
 - $t(N, F/G, Subs)$ representa uma árvore com sub-árvores não vazias
 - ❖ N é a raiz da árvore
 - ❖ Subs é uma lista de suas sub-árvores (em ordem crescente de valores-f das sub-árvores)
 - ❖ G é $g(N)$
 - ❖ F é o valor-f atualizado de N, ou seja, o valor-f do sucessor mais promissor de N

A^*



$I(s, 0/0)$

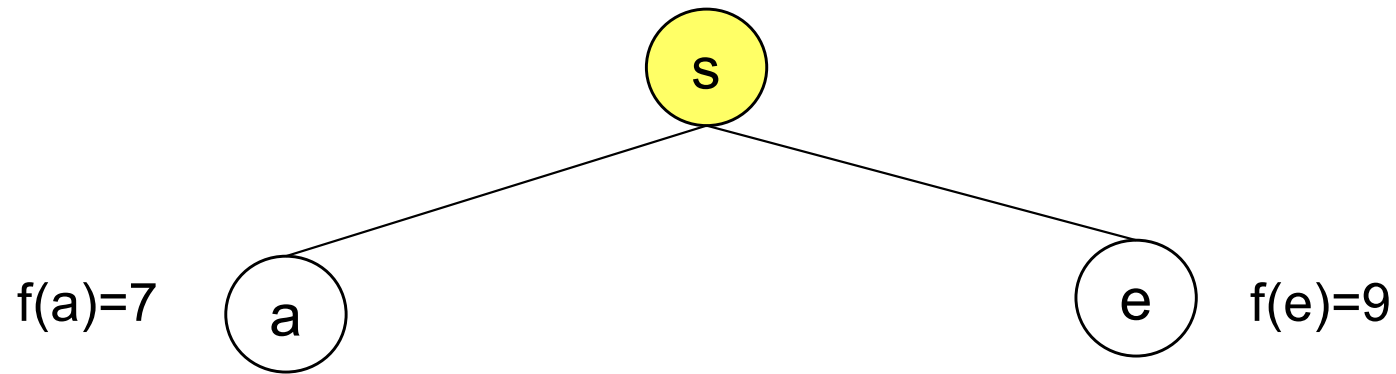
A*



$t(s, 7/0, [l(a, 7/2), l(e, 9/2)])$

O valor-f da raiz **s** é $f(s)=7$ pois é o valor-f do sucessor mais promissor de **s**

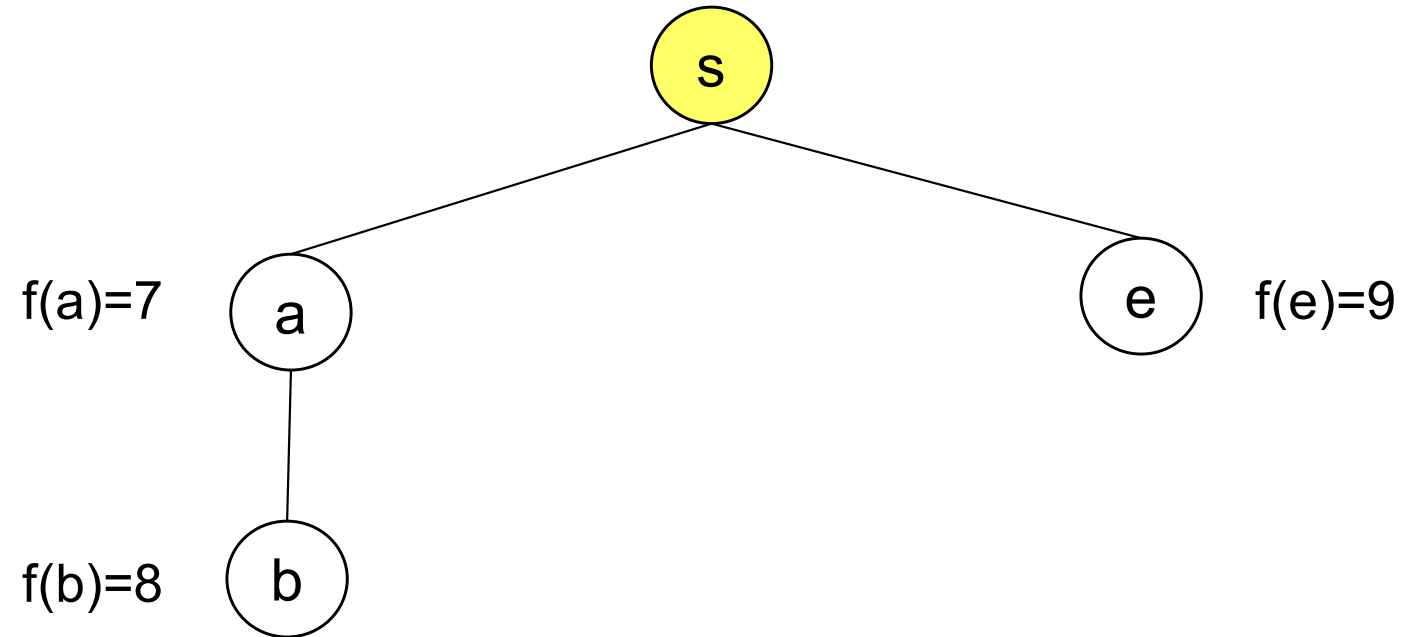
A*



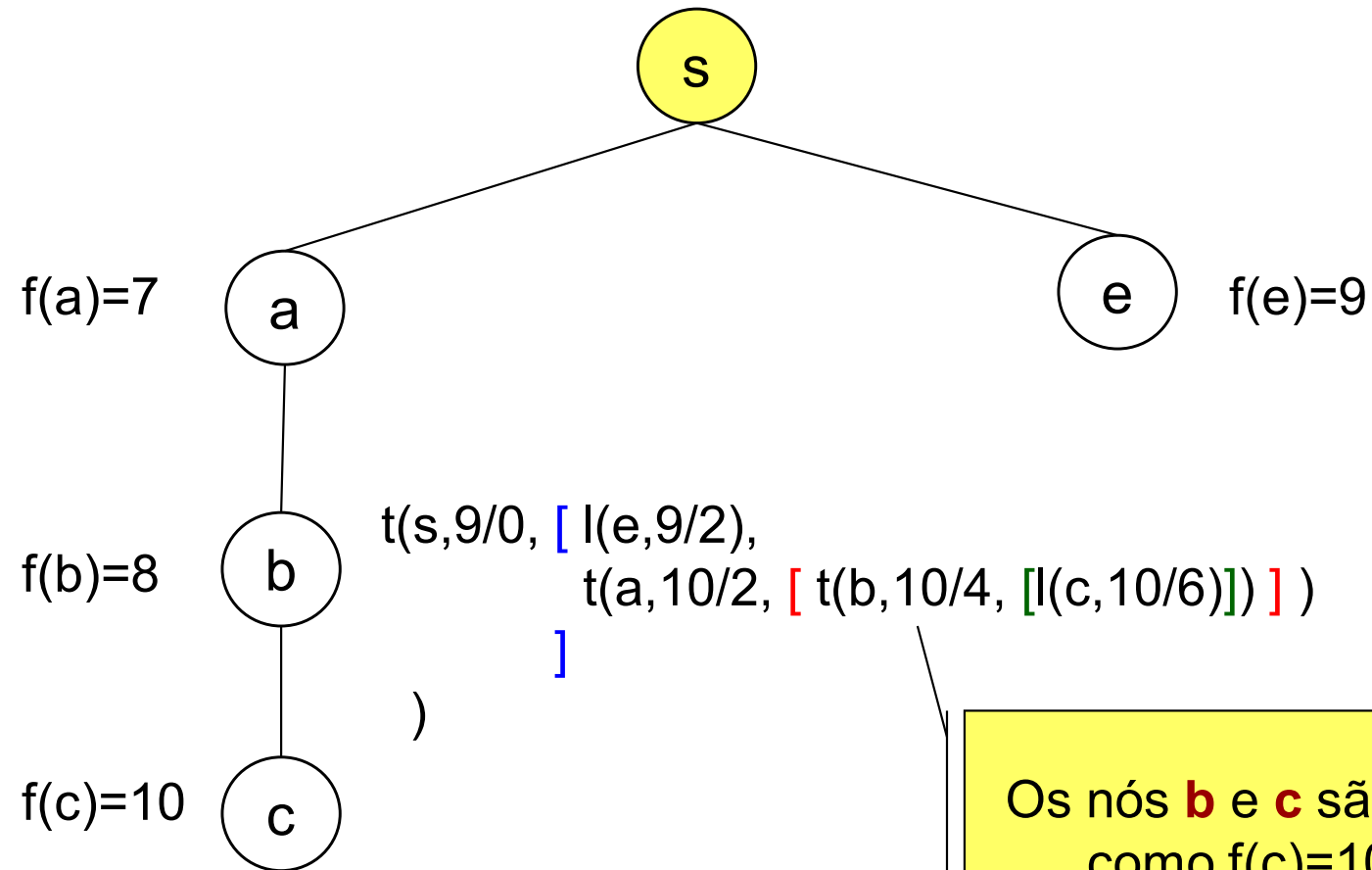
$t(s, 7/0, [l(a, 7/2), l(e, 9/2)])$

O competidor mais próximo de **a** é **e**, com $f(e)=9$
Portanto, **a** é permitido expandir enquanto $f(a)$ não exceder 9

A*

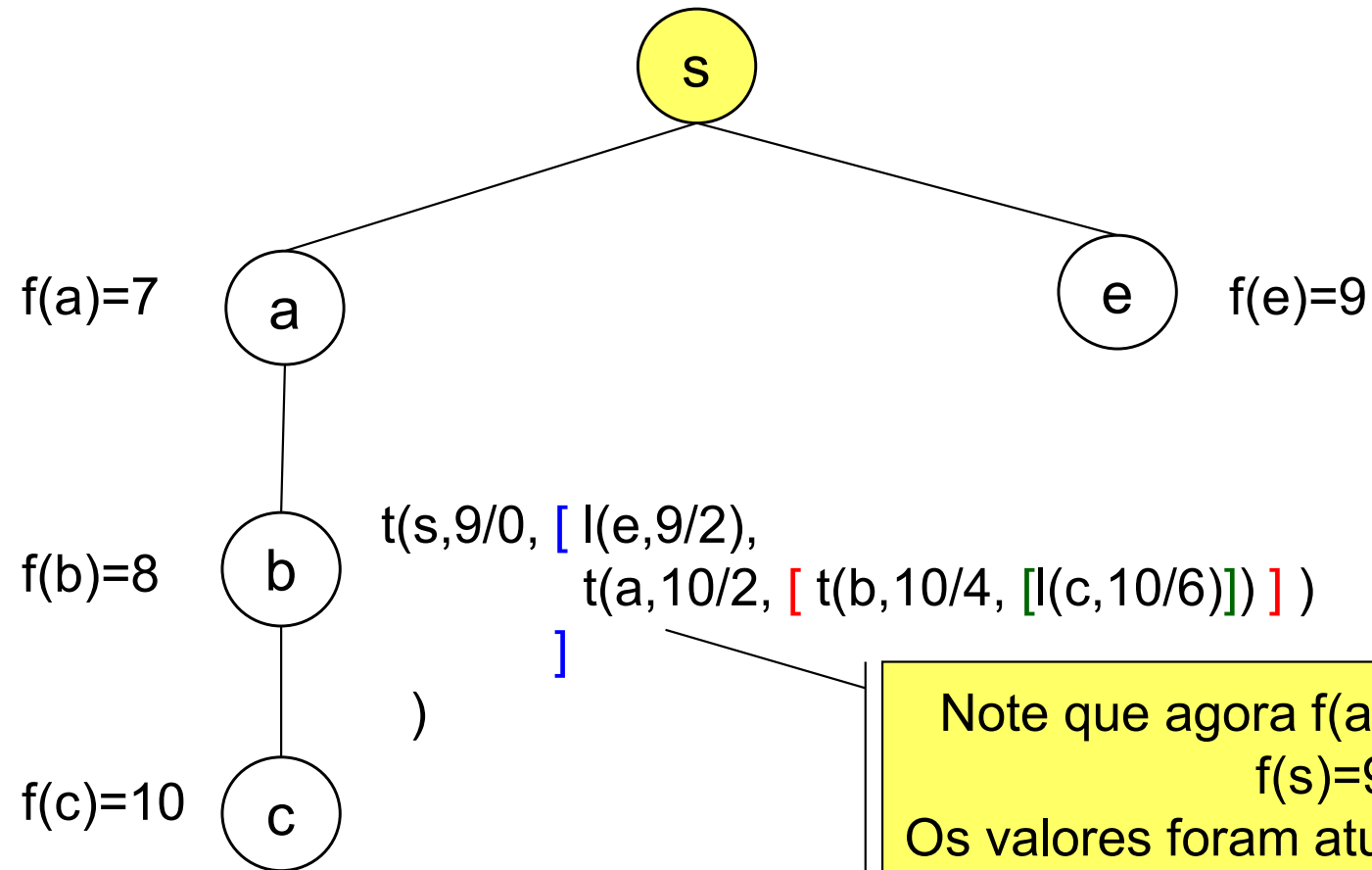


A*



Os nós **b** e **c** são expandidos e como $f(c)=10$ o limite de expansão é atingido e então a sub-árvore via **a** não tem mais permissão para expandir

A*



Note que agora $f(a)=10$ enquanto $f(s)=9$
Os valores foram atualizados devido ao fato que novos nós, **b** e **c**, foram gerados
Agora o sucessor mais promissor de **s** é **e** com $f(e)=9$

A*

- A atualização dos valores- f é necessário para permitir o programa reconhecer a sub-árvore mais promissora em cada nível da árvore de busca (a árvore que contém o nó mais promissor)
- Esta atualização leva a uma generalização da definição da função f de nós para árvores

A*

- Para um único nó (folha) **n**, temos a definição original
 - $f(n) = g(n) + h(n)$
- Para uma árvore T , cuja raiz é **n** e as sub-árvores de **n** são $S_1, S_2, \dots, S_k$
 - $f(T) = \min f(S_i) \quad 1 \leq i \leq k$

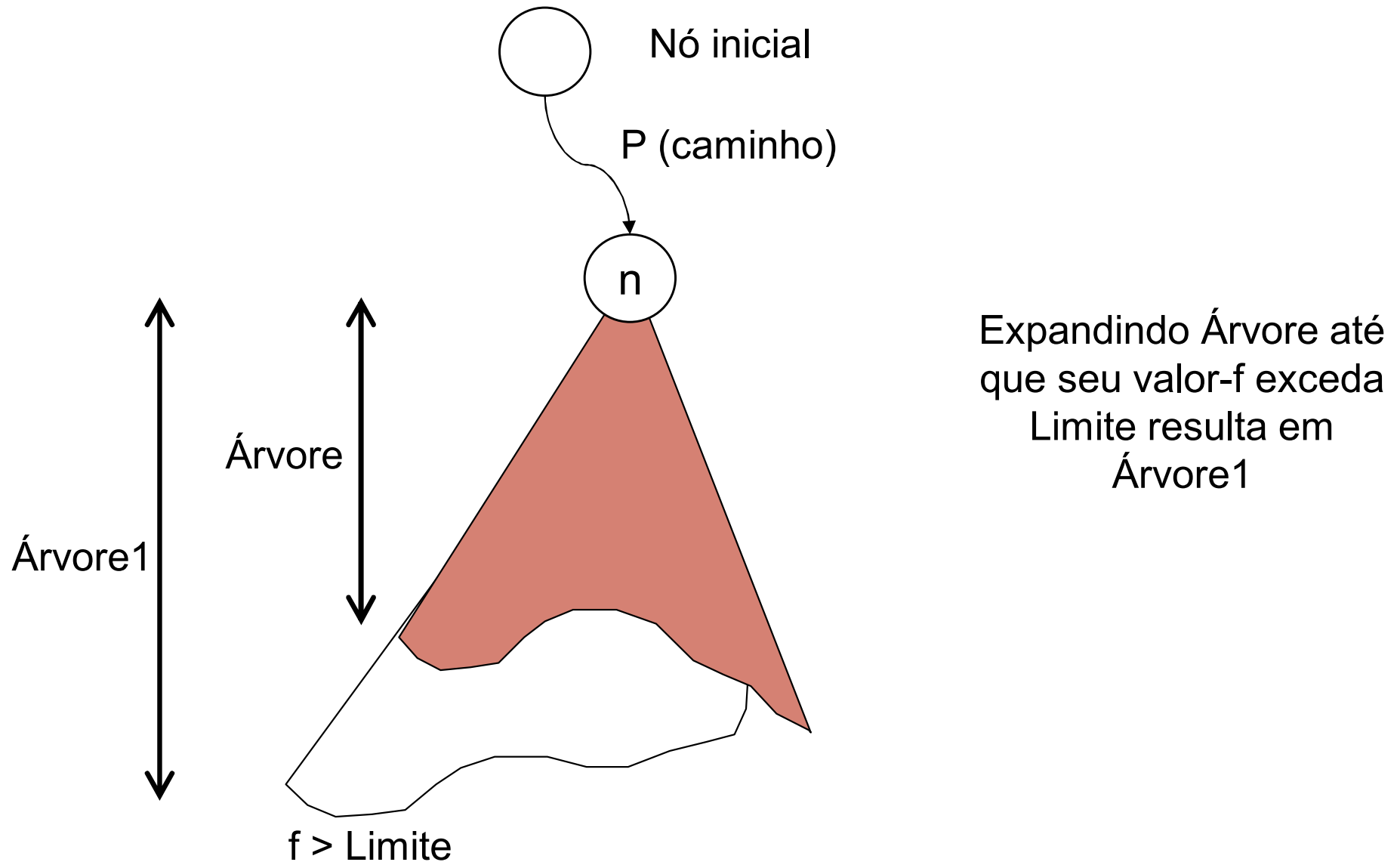
A*

- ❑ O predicado principal é **expandir(P,Árvore,Limite,Árvore1,Resolvido,Solução)**
- ❑ Este predicado expande uma (sub)árvore atual enquanto o valor-f dela permaneça inferior ou igual à Limite
- ❑ Argumentos:
 - **P**: caminho entre o nó inicial e **Árvore**
 - **Árvore**: atual (sub)árvore
 - **Limite**: valor-f limite para expandir **Árvore**
 - **Árvore1**: **Árvore** expandida dentro de **Limite**; assim o valor-f de **Árvore1** é maior que **Limite** (a menos que um nó final tenha sido encontrado durante a expansão)
 - **Resolvido**: Indicador que assume 'sim', 'não' ou 'nunca'
 - **Solução**: Um caminho (solução) do nó inicial através de **Árvore1** até um nó final dentro de **Limite** (se existir tal nó)

A*

- ❑ Os argumentos de entrada são **P**, **Árvore** e **Limite**
- ❑ **expandir/6** produz três tipos de resultados, indicados pelo valor do argumento **Resolvido**
 1. **Resolvido** = sim
Solução = uma solução encontrada expandindo **Árvore** dentro de **Limite**
Árvore1 = não instanciada
 2. **Resolvido** = não
Árvore1 = **Árvore** expandida de forma que seu valor-f exceda **Limite**
(vide slide seguinte)
Solução = não instanciada
 3. **Resolvido** = nunca
Árvore1 e **Solução** = não instanciadas
- ❑ O último caso indica que **Árvore** é uma alternativa inviável e nunca deve ter outra chance de crescer; isto ocorre quando o valor-f de **Árvore** $\leq$ **Limite** mas a árvore não pode crescer porque nenhuma folha dela possui sucessor ou o sucessor existente criaria um ciclo

A Relação expandir/6



Algoritmo A*

```
% Assuma que 9999 é maior que qualquer valor-f
resolvail(No,Solucao) :-
    expandir([],l(No,0/0),9999,_,sim,Solucao).

% expandir(P,Arvore,Limite,Arvore1,Resolvido,Solucao)
% P é um caminho entre nó inicial da busca e subárvore Arvore, Arvore1 é Arvore
% expandida até Limite. Se um nó final é encontrado então Solucao é a solução e
% Resolvido = sim

% Caso 1: nó folha final, construir caminho da solução
expandir(P,l(N,_) ,_,_,sim,[N|P]) :-
    final(N).

% Caso 2: nó folha, valor-f <= Limite. Gerar sucessores e expandir dentro de Limite
expandir(P,l(N,F/G),Limite,Arvore1,Resolvido,Solucao) :-
    F <= Limite,
    (findall(M/Custo, (s(N,M,Custo), \+ pertence(M,P)),Vizinhos),
    Vizinhos \= [],
    !,
    avalie(G,Vizinhos,Ts),
    melhorf(Ts,F1),
    expandir(P,t(N,F1/G,Ts),Limite,Arvore1,Resolvido,Solucao)
    ;
    Resolvido = nunca
    ).
```

Algoritmo A* (cont.)

```
% Caso 3: não-folha, valor-f <= Limite. Expanda a subárvore mais
% promissora; dependendo dos resultados, o predicado continue
% decide como proceder
expandir(P,t(N,F/G,[T|Ts]),Limite,Arvore1,Resolvido,Solucao) :-
    F <= Limite,
    melhorf(Ts,MF),
    min(Limite,MF,Limite1),    % Limite1 = min(Limite,MF)
    expandir([N|P],T,Limite1,T1,Resolvido1,Solucao),
    continue(P,t(N,F/G,[T1|Ts]),Limite,Arvore1,Resolvido1,Resolvido,Solucao).

% Caso 4: não-folha com subárvores vazias
% Beco sem saída que nunca será resolvido
expandir(_,t(_,_,[]),_,_,nunca,_) :- !.

% Caso 5: valor f > Limite, árvore não pode crescer
expandir(_,Arvore,Limite,Arvore,nao,_) :-
    f(Arvore,F),
    F > Limite.
```

Algoritmo A* (cont.)

```
% continue(Caminho,Arvore,Limite,NovaArvore,SubarvoreResolvida,ArvoreResolvida,Solucao)
continue(_,_,_,_,sim,sim,_).    % solução encontrada
% Limite ultrapassado, procurar outra subárvore para expandir
continue(P,t(N,F/G,[T1|Ts]),Limite,Arvore1,nao,Resolvido,Solucao) :-
    inserir(T1,Ts,NTs),
    melhorf(NTs,F1),
    expandir(P,t(N,F1/G,NTs),Limite,Arvore1,Resolvido,Solucao).
% abandonar T1 pois é beco sem saída
continue(P,t(N,F/G,[T1|Ts]),Limite,Arvore1,nunca,Resolvido,Solucao) :-
    melhorf(Ts,F1),
    expandir(P,t(N,F1/G,Ts),Limite,Arvore1,Resolvido,Solucao).

% avalie(G0,[No1/Custo1,...],[l(MelhorNo,MelhorF/G,...)])
% ordena a lista de folhas pelos seus valores-f
avalie(_,[],[]).
avalie(G0,[N/C|NaoAvaliados],Ts) :-
    G is G0 + C,
    h(N,H),
    F is G + H,
    avalie(G0,NaoAvaliados,Avaliados),
    inserir(l(N,F/G),Avaliados,Ts).
```

Algoritmo A* (cont.)

```
% insere T na lista de árvore Ts mantendo a ordem dos valores-f
inserir(T,Ts,[T|Ts]) :-
    f(T,F) ,
    melhorf(Ts,F1) ,
    F <= F1, !.
inserir(T,[T1|Ts],[T1|Ts1]) :-
    inserir(T,Ts,Ts1) .

% Obter o valor f
f(l(_,F/_),F) .           % valor-f de uma folha
f(t(_,F/_,_),F) .         % valor-f de uma árvore

melhorf([T|_],F) :-        % melhor valor-f de uma lista de árvores
    f(T,F) .
melhorf([],9999) .         % Nenhuma árvore: definir valor-f ruim

min(X,Y,X) :-
    X <= Y, !.
min(X,Y,Y) .

pertence(E,[E|_]) .
pertence(E,[_|T]) :-
    pertence(E,T) .
```

Admissibilidade de A^*

- ❑ Um algoritmo de busca é chamado de **admissível** se ele sempre produz uma solução ótima (caminho de custo mínimo), assumindo que uma solução exista
- ❑ A implementação apresentada, que produz todas as soluções por meio de *backtracking* e pode ser considerada admissível se a primeira solução encontrada é ótima
- ❑ Para cada nó **n** no espaço de estados vamos denotar **$h^*(n)$** como sendo o custo de um caminho ótimo de **n** até um nó final
- ❑ Um teorema sobre a admissibilidade de A^* diz que um algoritmo A^* que utiliza uma função heurística **h** tal que para todos os nós no espaço de estados $h(n) \leq h^*(n)$ é admissível
- ❑ Este resultado tem grande valor prático
 - Mesmo que não conheçamos o exato valor de h^* , nós só precisamos achar um **limite inferior** para h^* e utilizá-la como h em A^*
 - Isto é suficiente para garantir que A^* irá encontrar uma solução ótima

Admissibilidade de A^*

- Há um limite inferior trivial
 - $h(n) = 0$ para todo n no espaço de estados
- Embora este limite trivial garanta admissibilidade sua desvantagem é que não há nenhuma heurística e assim não há como fornecer nenhum auxílio para a busca, resultando em alta complexidade
 - A^* usando $h(n)=0$ comporta-se de forma similar à busca em largura e igual à busca de custo uniforme
 - De fato, A^* se comporta exatamente igual à busca em largura se todos os arcos entre nós têm custo unitário, ou seja, $s(X,Y,1)$

Admissibilidade de A^*

- ❑ Portanto é interessante utilizar $h > 0$ para garantir admissibilidade e h o mais próximo possível de h^* ($h \leq h^*$) para garantir eficiência
- ❑ Se múltiplas heurísticas estão disponíveis:
 - $h(n) = \text{máximo} \{h_1(n), h_2(n), \dots, h_m(n)\}$
- ❑ De maneira ideal, se h^* é conhecida, podemos utilizar h^* diretamente
 - A^* utilizando h^* encontra uma solução ótima diretamente, sem nunca precisar realizar *backtracking*

A*: Função de Avaliação

- Uma heurística h é monotônica/consistente se, para cada nó n e seu sucessor n' , ela obedece à regra:

$$h(n) \leq c(n, n') + h(n')$$

- Onde $c(n, n')$ é o custo da aresta entre n e n'

- Exemplo

- Imagine que o seu GPS diz que faltam 10 km até o destino ($h(n) = 10$)
- Você dá um passo que custa 1 km ($c = 1$)
- Se o GPS agora disser que faltam 2 km pro destino ($h(n) = 2$), ele quebrou a consistência, pois $10 \leq 1 + 2$ é falso
- A estimativa tem que ser realista

A*: Função de Avaliação

- ❑ Sabemos que o custo acumulado de n' é o custo de n mais o custo da aresta entre eles
 - $g(n') = g(n) + c(n, n')$
- ❑ Pela regra da consistência, temos
 - $h(n) \leq c(n, n') + h(n')$
- ❑ Se somarmos $g(n)$ dos dois lados da desigualdade, temos
 - $g(n) + h(n) \leq g(n) + c(n, n') + h(n')$
 - O lado esquerdo é a definição de $f(n)$
 - O lado direito é a definição de $f(n')$
- ❑ Logo, $f(n) \leq f(n')$
- ❑ Em outras palavras, o custo de cada nó gerado nunca diminui

Lidando com Heurísticas Inconsistentes (A Equação Pathmax)

- ❑ Se a heurística h é não-monotônica (inconsistente), o valor da função de avaliação f pode "cair" de um nó n para o seu sucessor n'
- ❑ Pathmax: Para forçar a monotonicidade de f ao longo de um caminho, propagamos o maior valor de f encontrado até o momento

$$f(n') = \max(f(n), g(n') + h(n'))$$

- ❑ Isso garante que o custo total estimado nunca diminua ao longo de uma mesma rota ($f(n') \geq f(n)$), recuperando a expansão em anéis concêntricos (contornos) do algoritmo A^*

Complexidade de A*

- ❑ A utilização de heurística para guiar o algoritmo A* reduz a busca a apenas uma região do espaço do problema
- ❑ Apesar da redução no esforço da busca, a ordem de complexidade é ainda exponencial na profundidade de busca $O(b^{\lceil C^*/\epsilon \rceil}) \approx O(b^d)$
 - Isso é válido para tempo e memória uma vez que o algoritmo mantém todos os nós gerados
- ❑ Em situações práticas o espaço de memória é mais crítico e A* pode utilizar toda a memória disponível em questão de minutos

Complexidade de A^*

- ❑ **A^* é o algoritmo de busca mais rápido**, ou seja, para uma dada heurística, nenhum outro algoritmo pode expandir menos nós que A^*
- ❑ A velocidade de A^* depende da qualidade da heurística
 - Se a heurística é desprezível (por exemplo, $h(n)=0$ para todo n) o algoritmo degenera para a busca de custo uniforme
 - Se a heurística é perfeita ($h=h^*$) não há busca de fato; o algoritmo marcha diretamente até o objetivo
- ❑ Geralmente, os problemas reais se encontram entre as duas situações acima e, portanto, o tempo de execução de A^* vai depender da qualidade da heurística

Complexidade de A^*

- ❑ Algumas variações de A^* foram desenvolvidas para utilizar menos memória, penalizando o tempo
 - A idéia básica é similar à busca em profundidade iterativa
 - O espaço necessário reduz de exponencial para linear na profundidade de busca
 - O preço é a re-expansão de nós já expandidos no espaço de busca
- ❑ Veremos duas dessas técnicas:
 - IDA* (Iterative Deepening A^*)
 - RBFS (Recursive Best-First Search)

IDA*

- IDA* é similar à busca em profundidade iterativa
 - Na busca em profundidade iterativa as buscas em profundidade são realizadas em limites crescentes de profundidade; em cada iteração a busca em profundidade é limitada pelo limite de profundidade atual
 - Em IDA* as buscas em profundidade são limitadas pelo limite atual representando valores-f dos nós

IDA*

procedure idastar(Inicio, Solucao)

Limite $\leftarrow$ f(Inicio)

repeat

Iniciando no nó Início, realize busca em profundidade sujeita à condição que um nó N é expandido apenas se $f(N) \leq$ Limite

if busca em profundidade encontrou nó final **then**

indique 'Solução encontrada'

else

Calcule NovoLimite como o mínimo valor-f dos nós alcançados ao ultrapassar Limite, ou seja,

$\text{NovoLimite} \leftarrow \min\{f(N) : N \text{ gerado pela busca e } f(N) > \text{Limite}\}$

endif

Limite $\leftarrow$ NovoLimite

until Solução encontrada

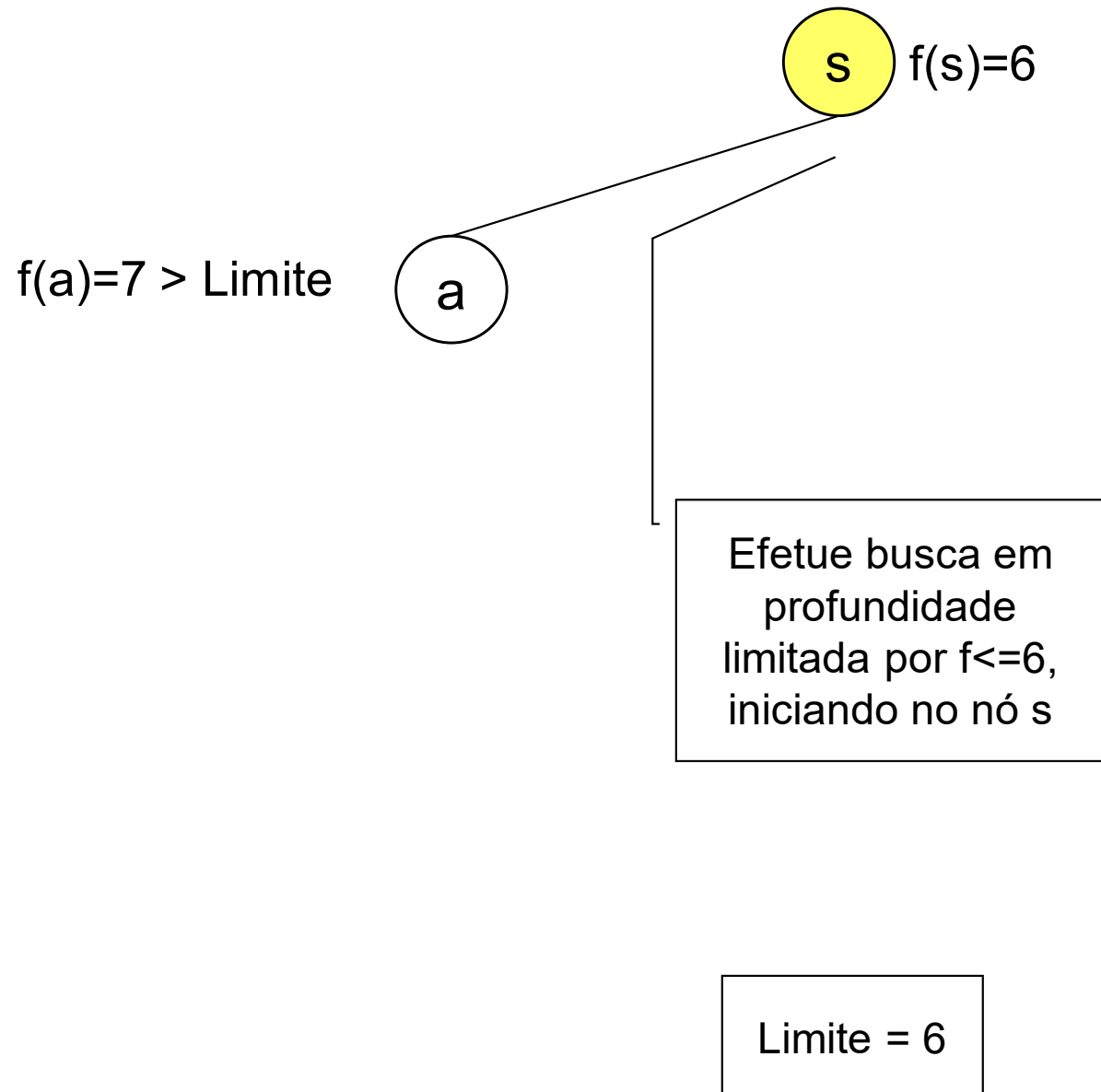
IDA*

s $f(s)=6$

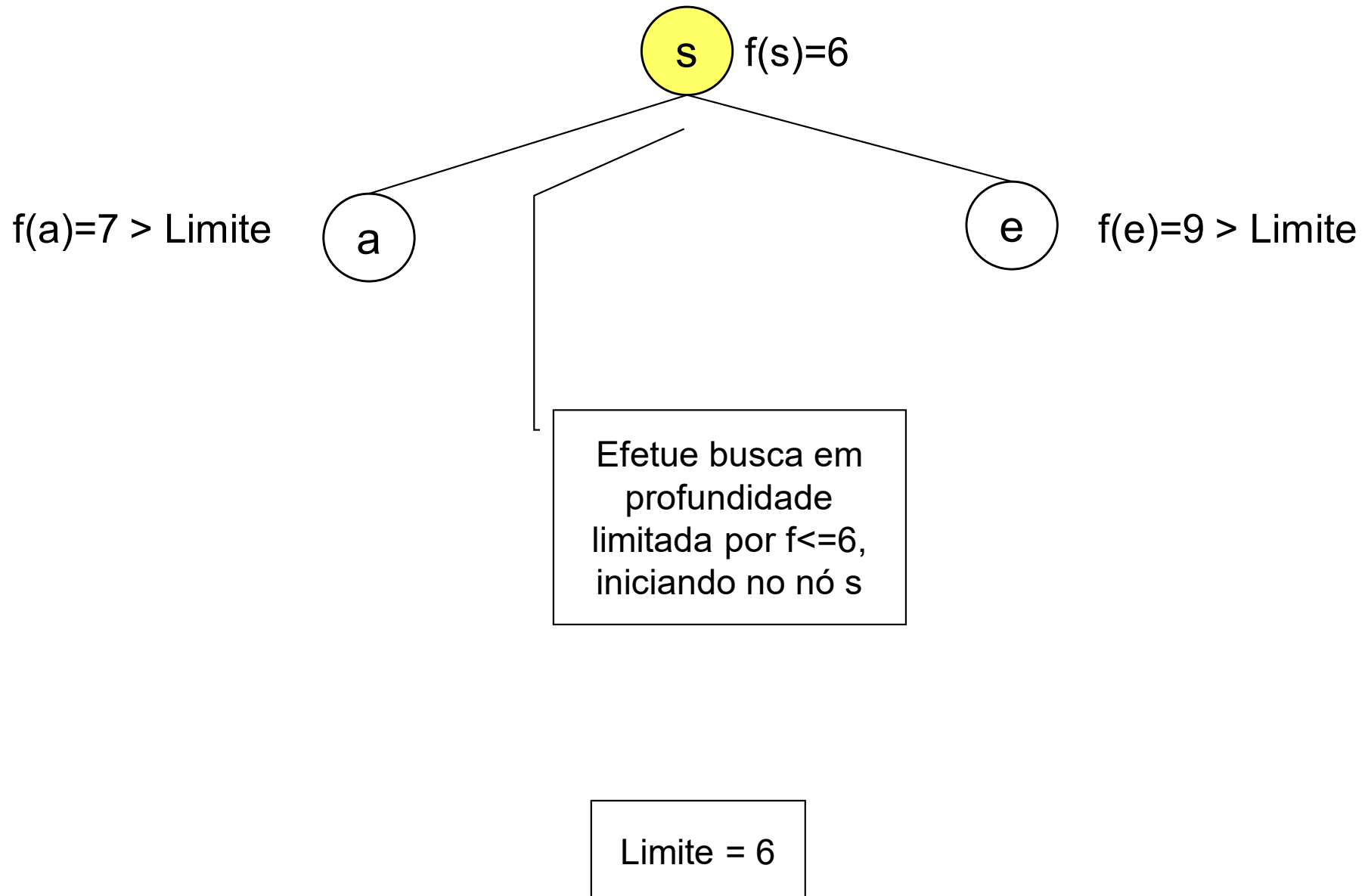
Efetue busca em
profundidade
limitada por $f \leq 6$,
iniciando no nó s

Limite = 6

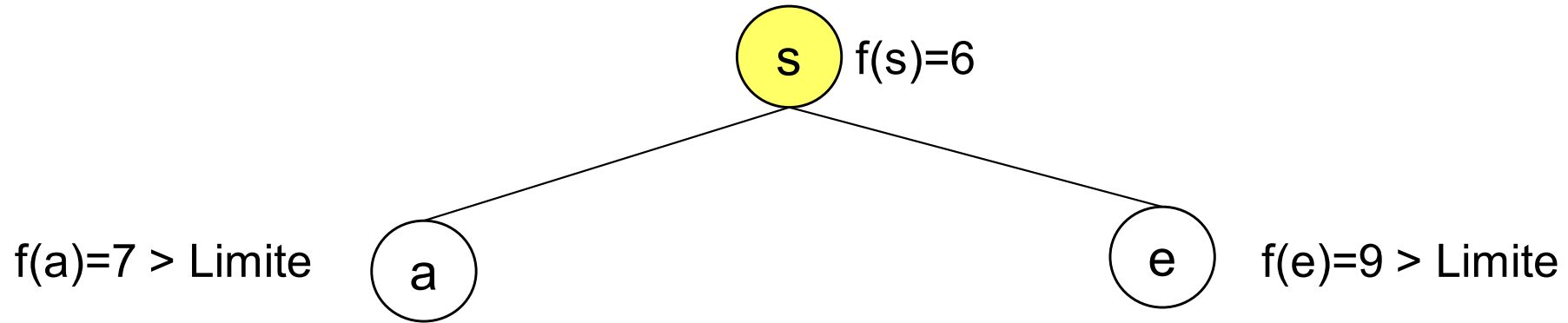
IDA*



IDA*



IDA*



NovoLimite = $\text{mín}\{7,9\} = 7$
Efetue busca em
profundidade limitada por
 $f \leq 7$, iniciando pelo nó s

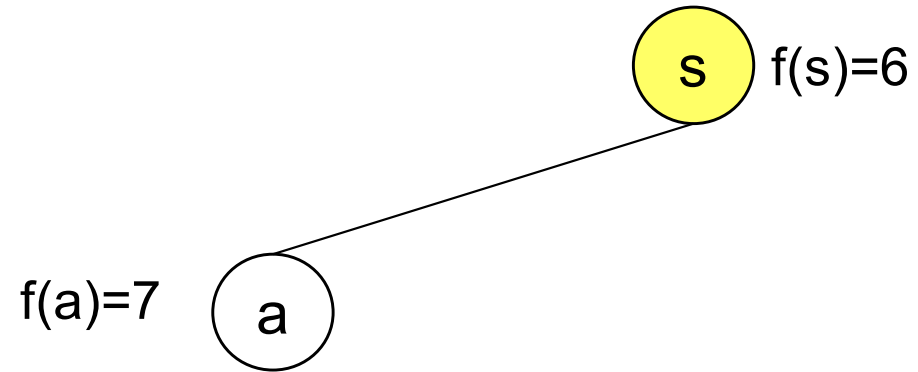
Limite = 7

IDA*

s $f(s)=6$

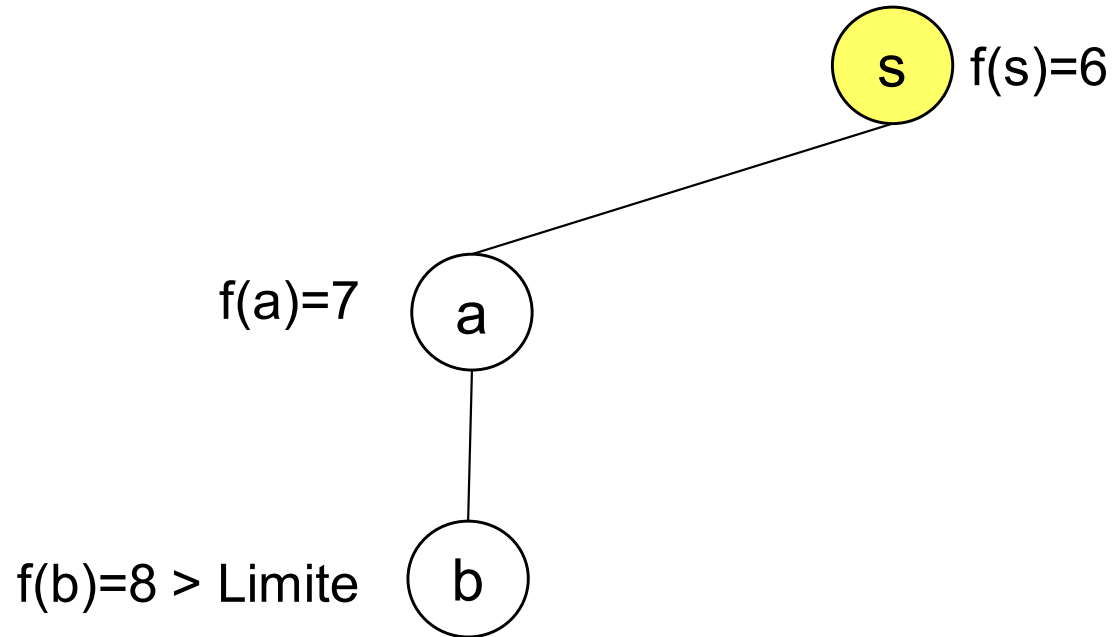
Limite = 7

IDA*



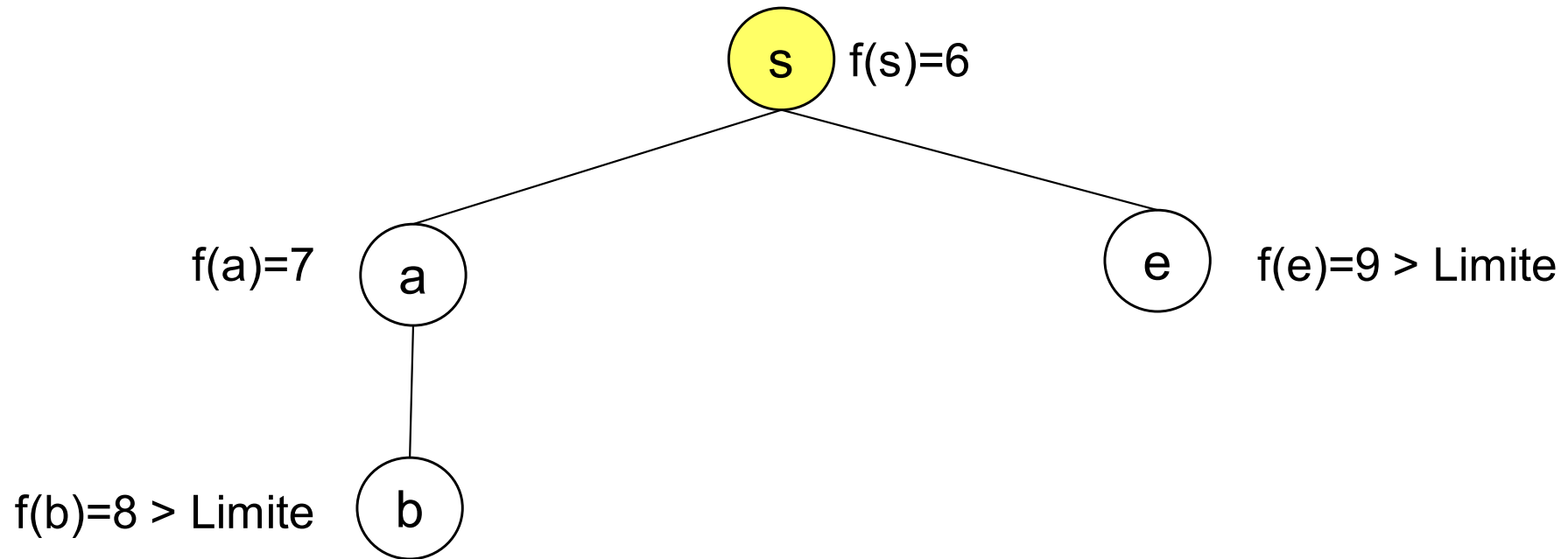
Limite = 7

IDA*



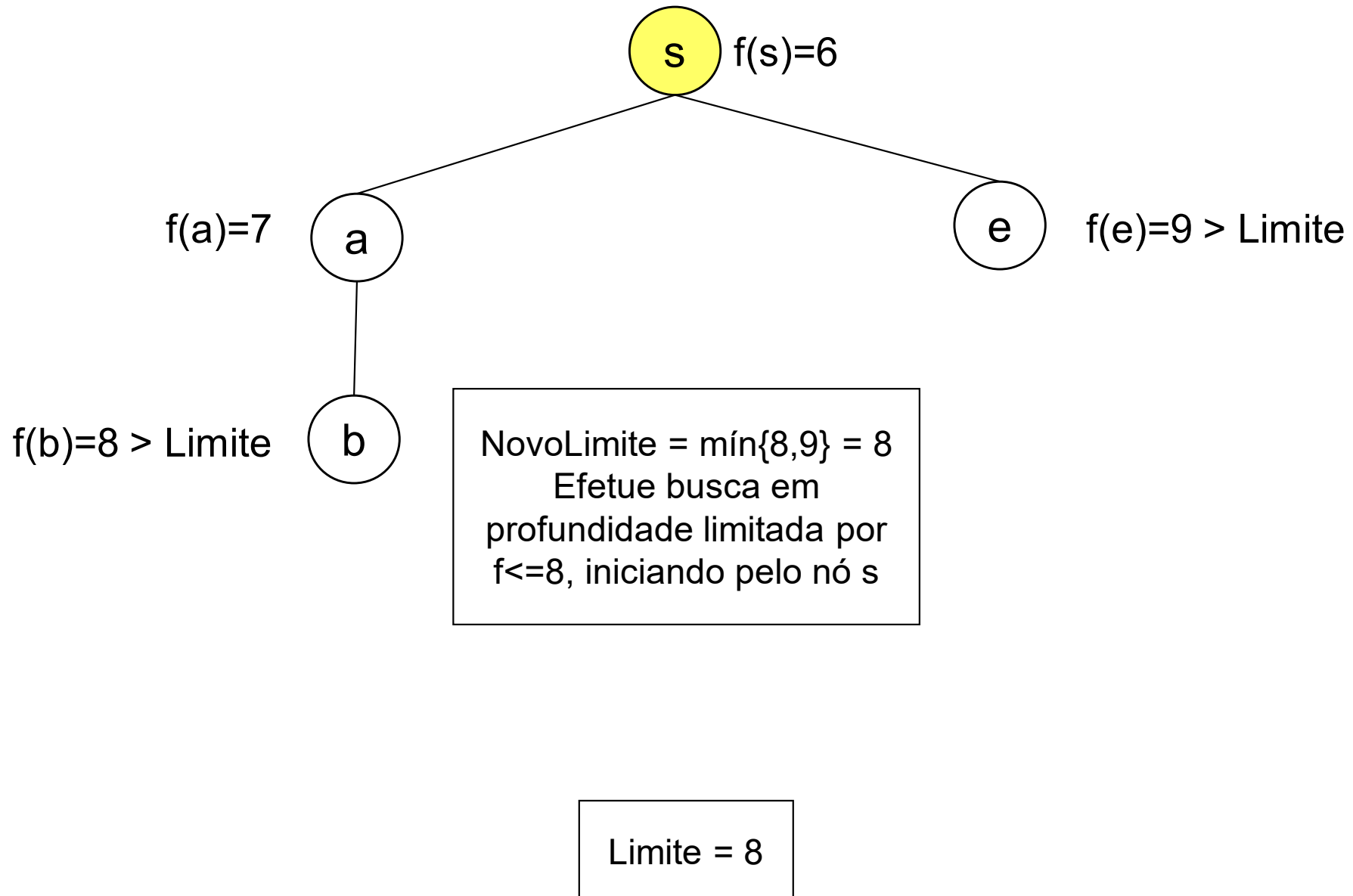
Limite = 7

IDA*



Limite = 7

IDA*

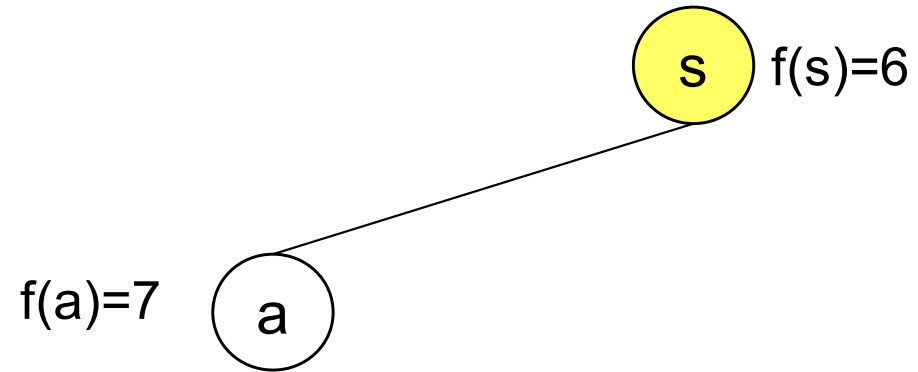


IDA*

s $f(s)=6$

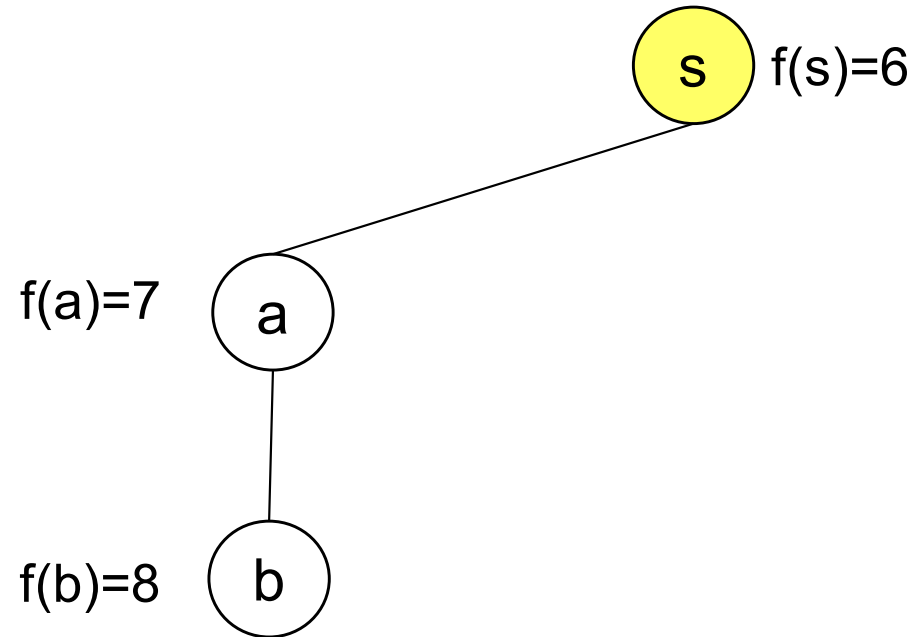
Limite = 8

IDA*



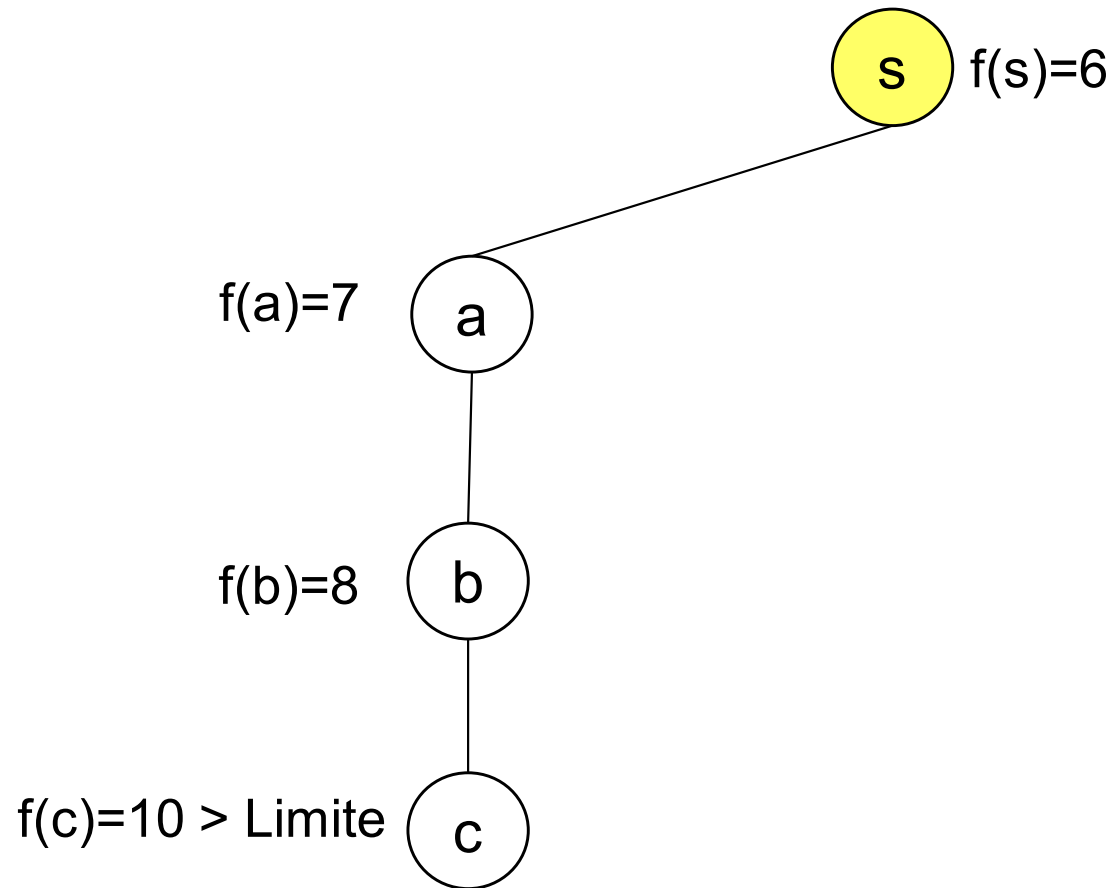
Limite = 8

IDA*



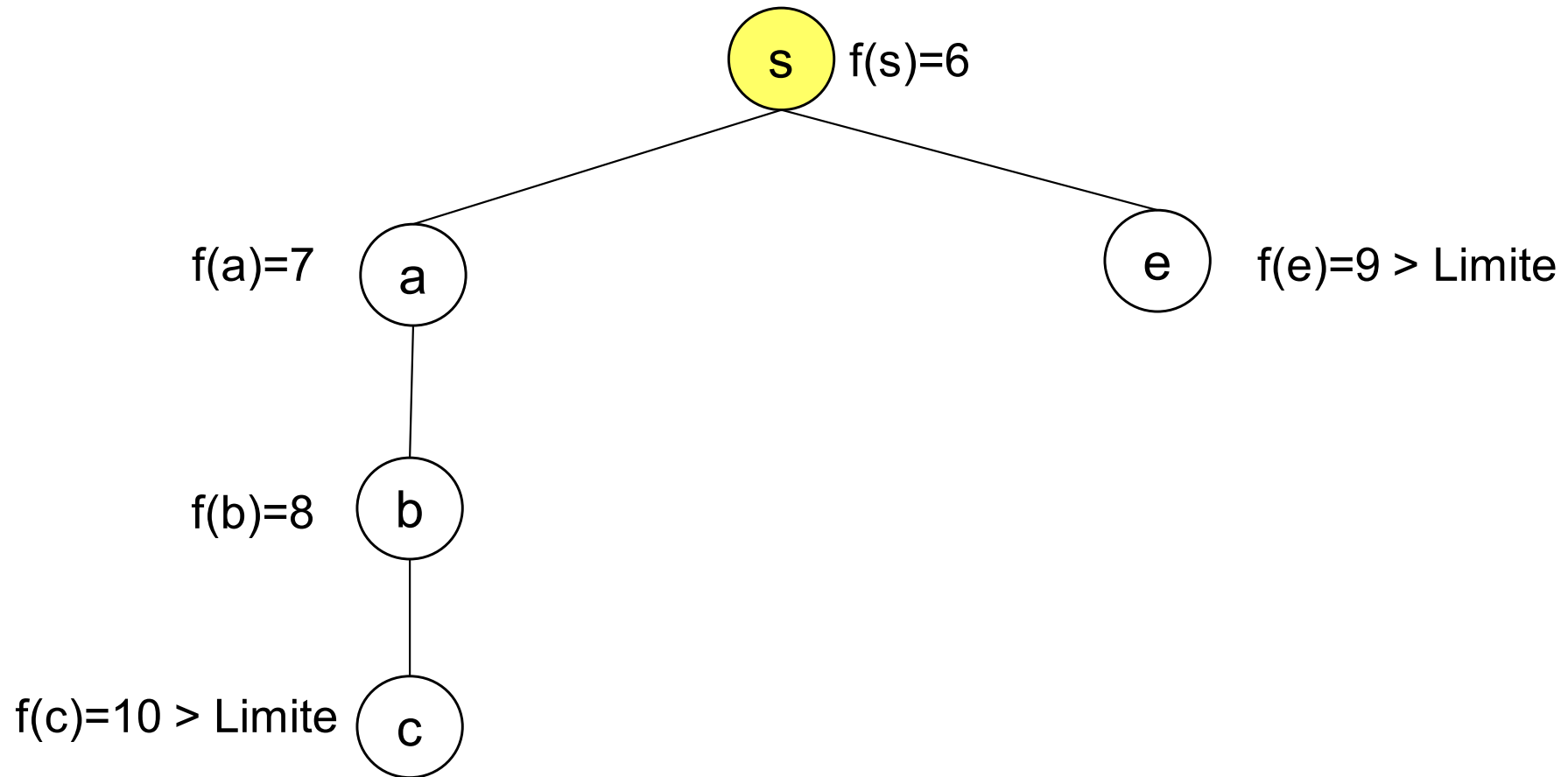
Limite = 8

IDA*



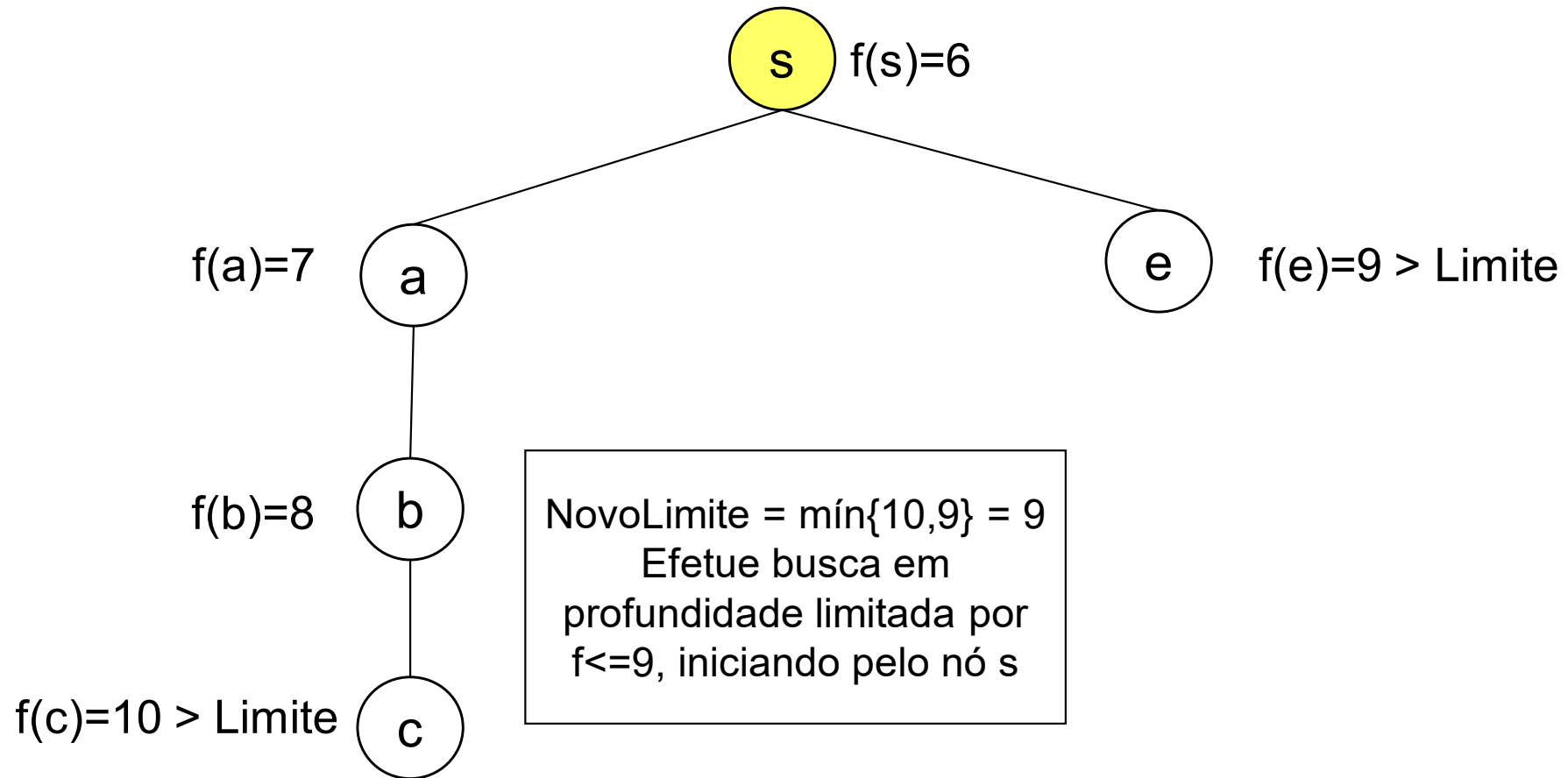
Limite = 8

IDA*



Limite = 8

IDA*



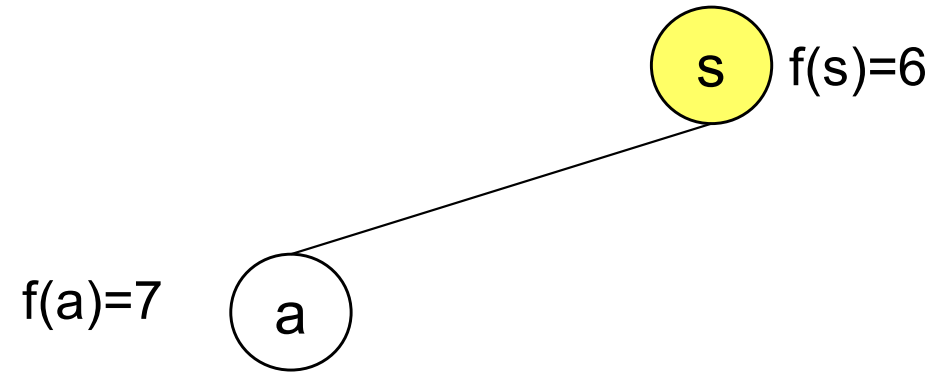
Limite = 9

IDA*

s $f(s)=6$

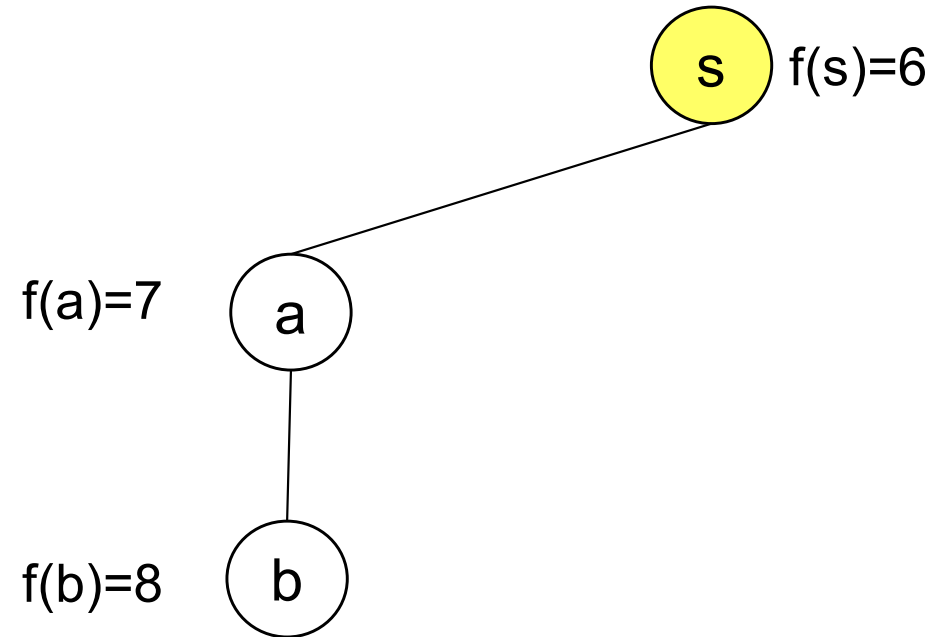
Limite = 9

IDA*



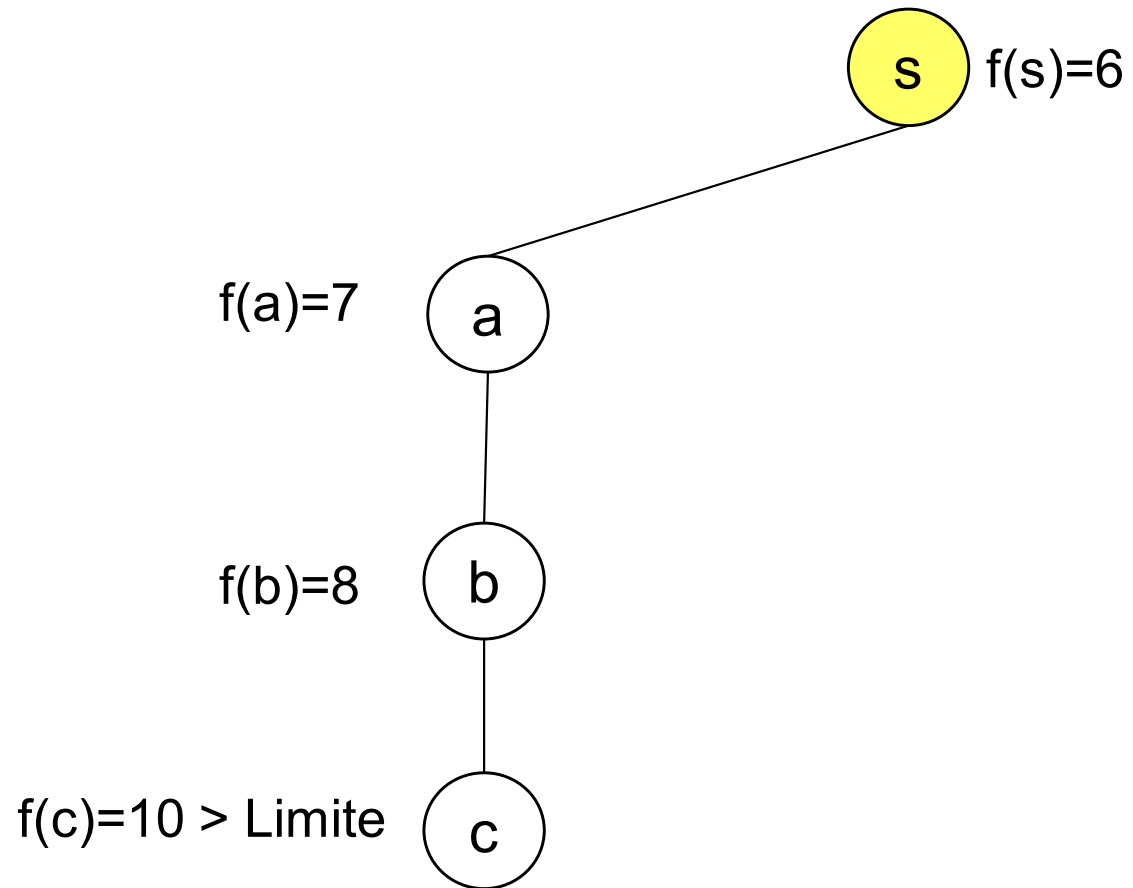
Limite = 9

IDA*



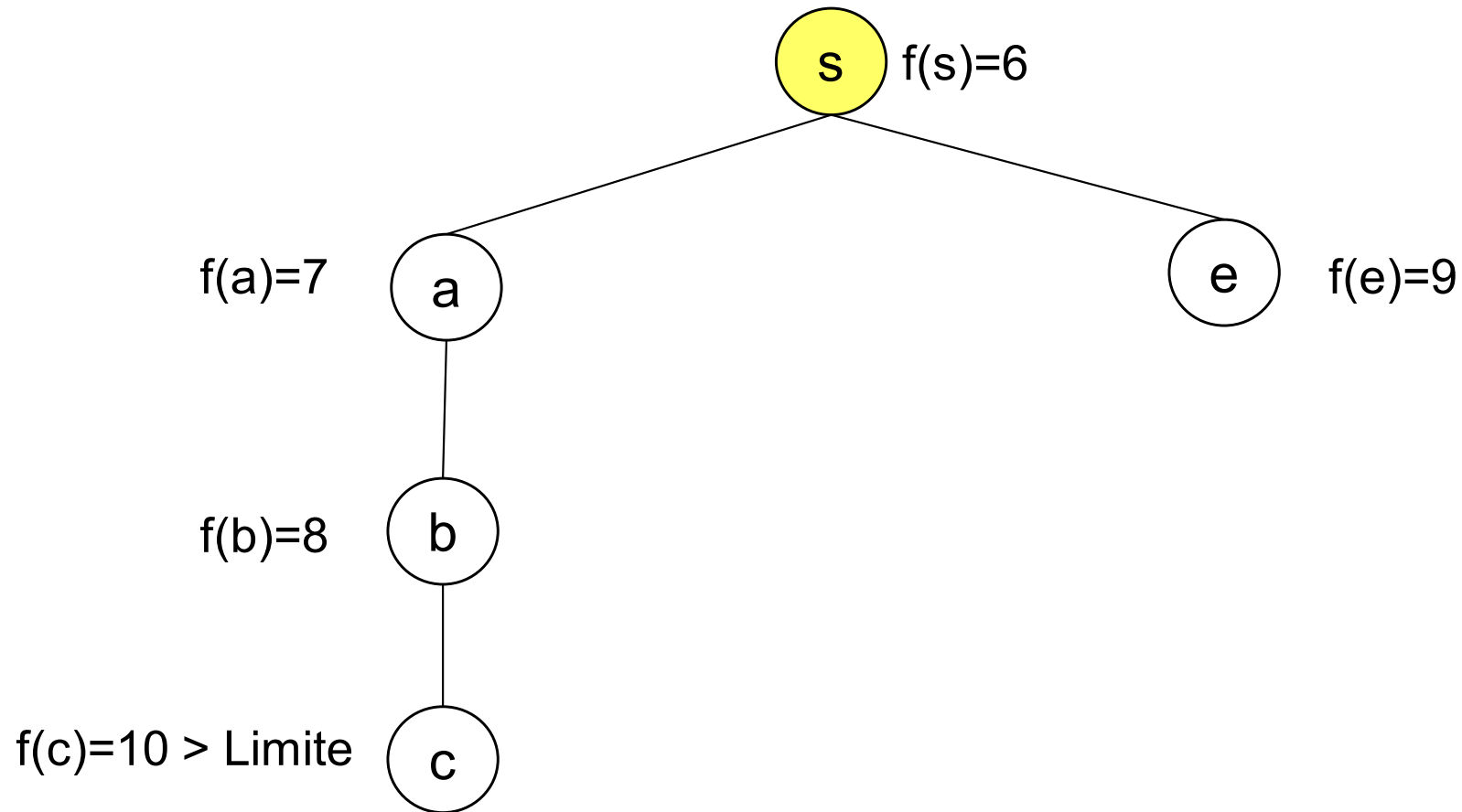
Limite = 9

IDA*



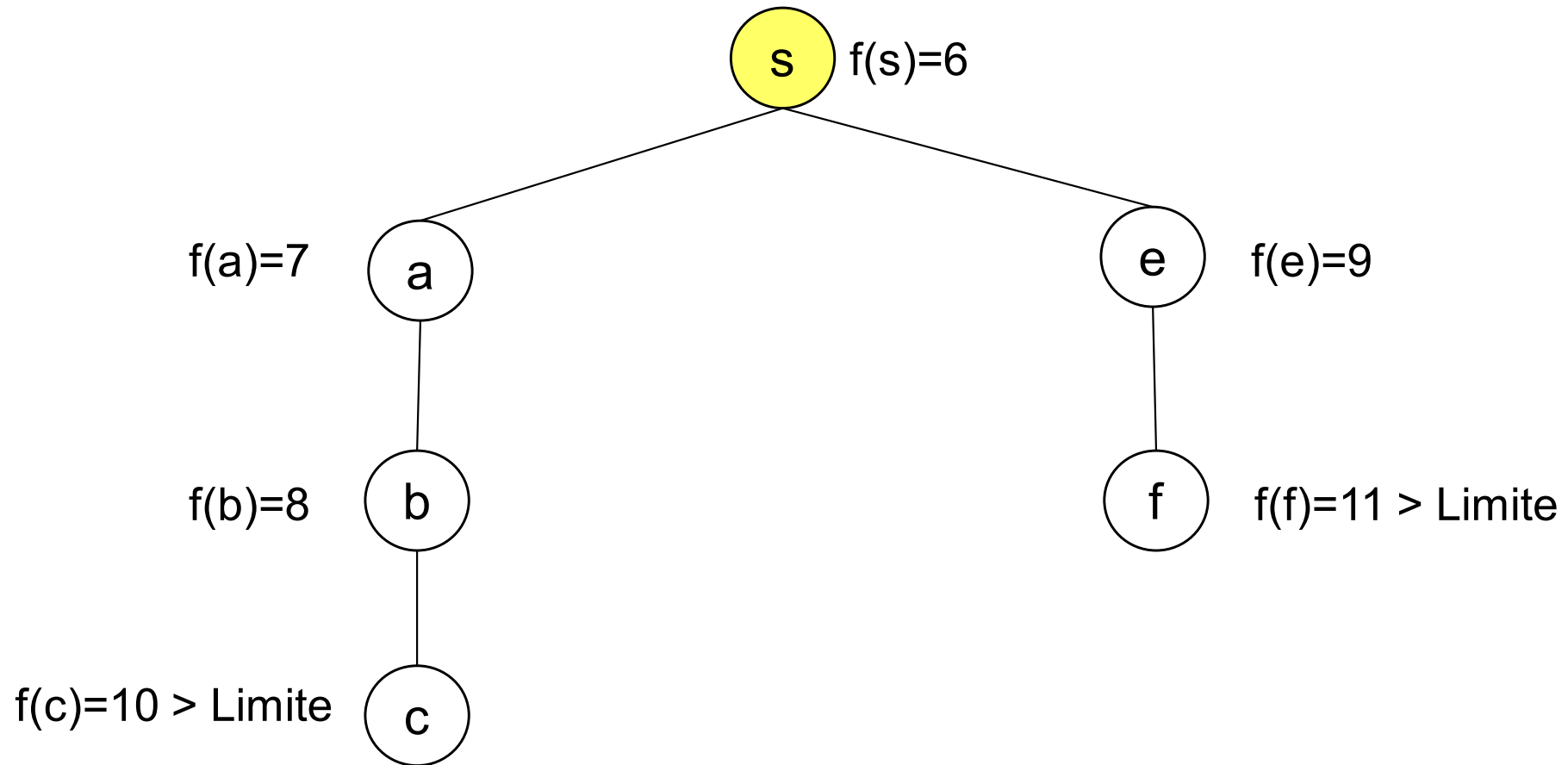
Limite = 9

IDA*



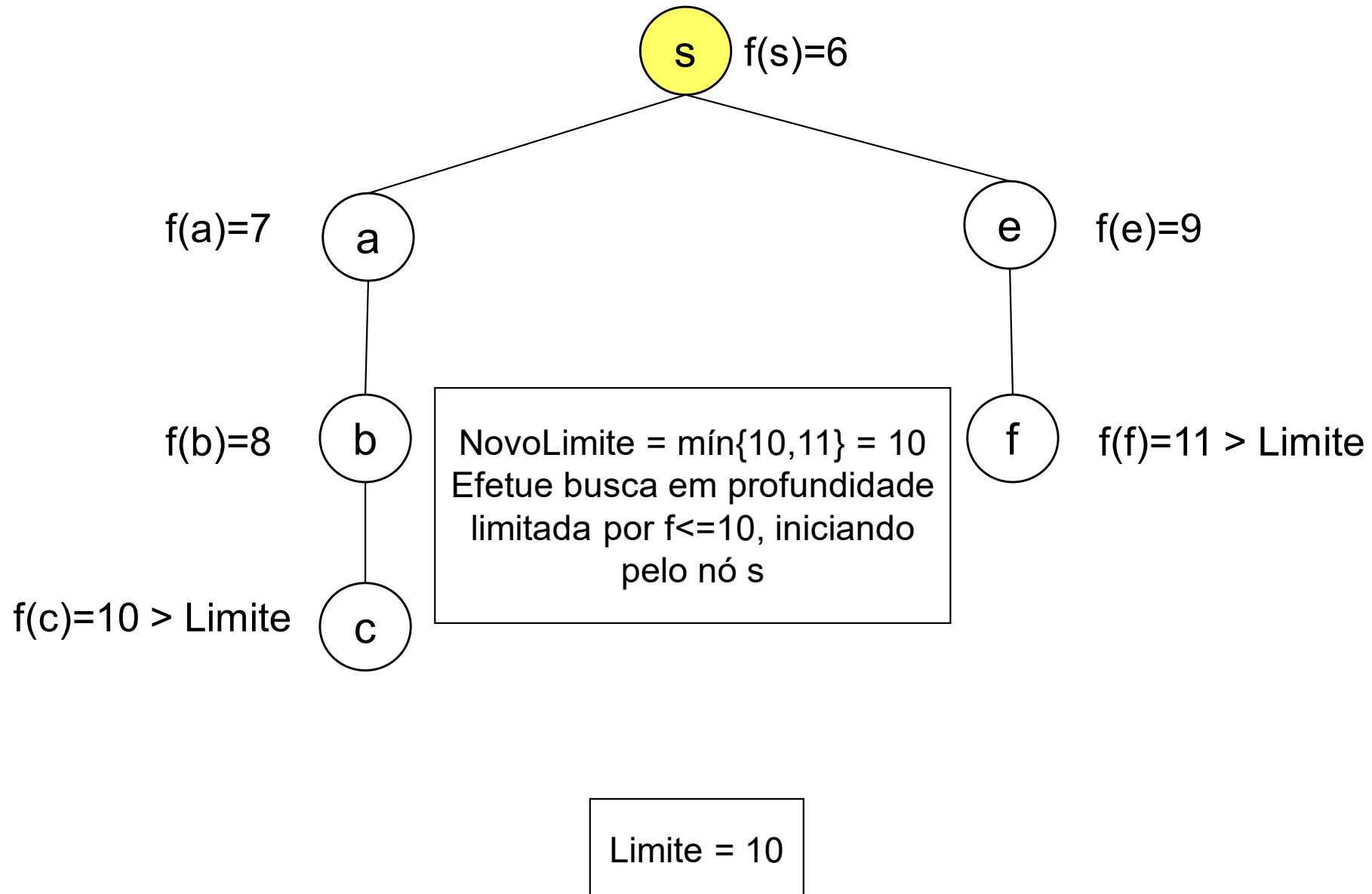
Limite = 9

IDA*



Limite = 9

IDA*

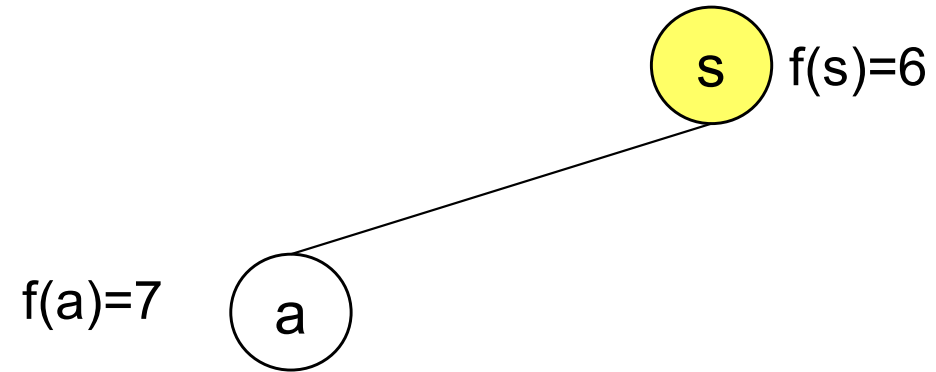


IDA*

s $f(s)=6$

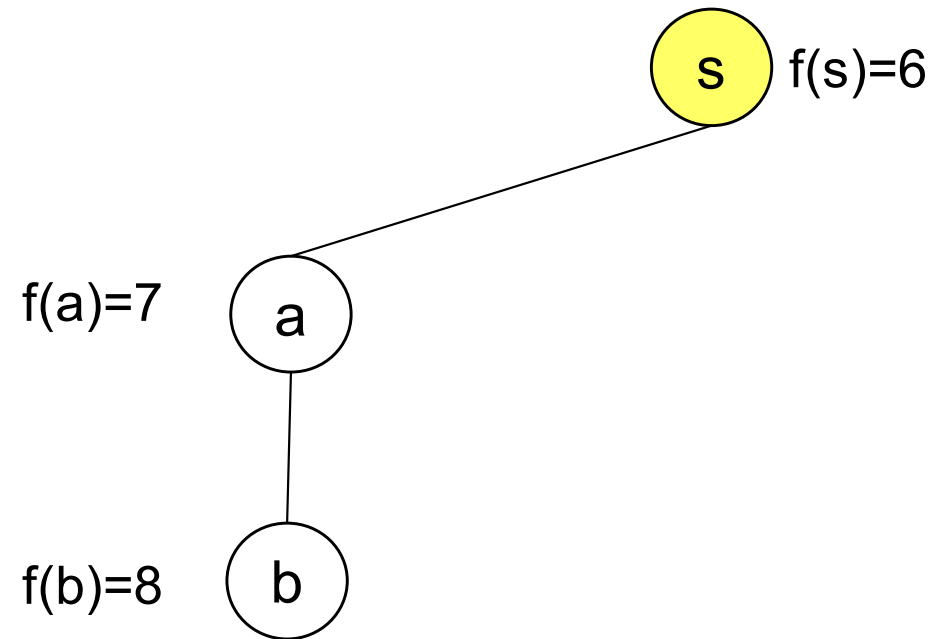
Limite = 10

IDA*



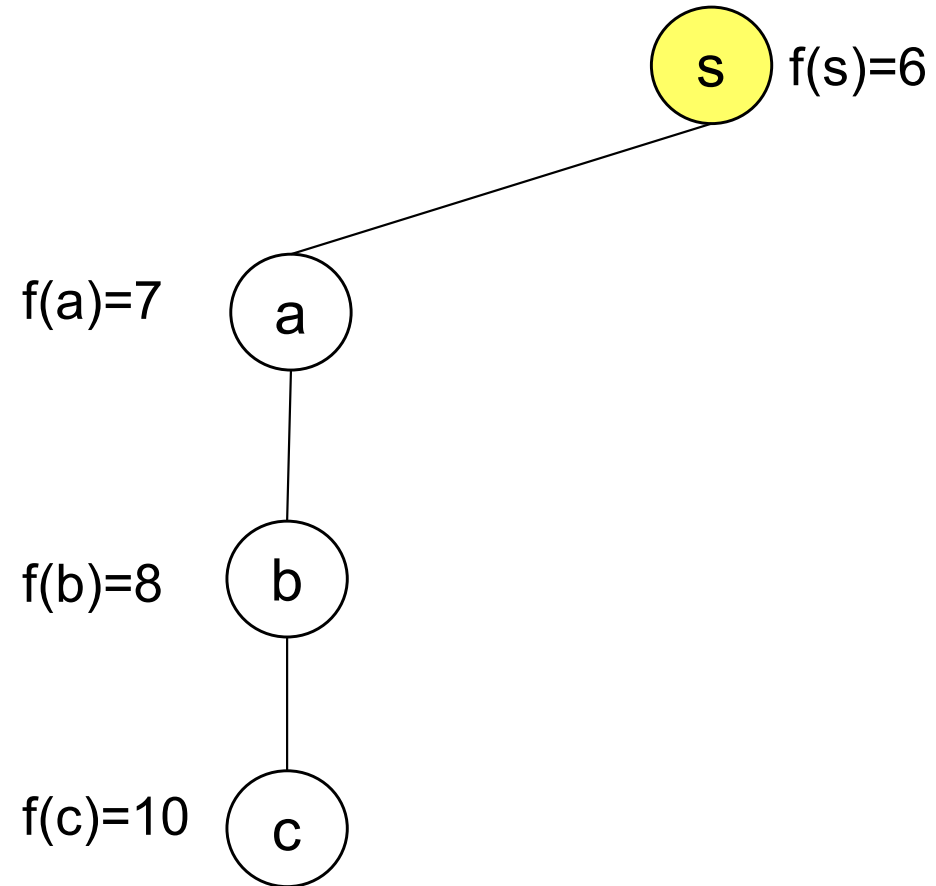
Limite = 10

IDA*



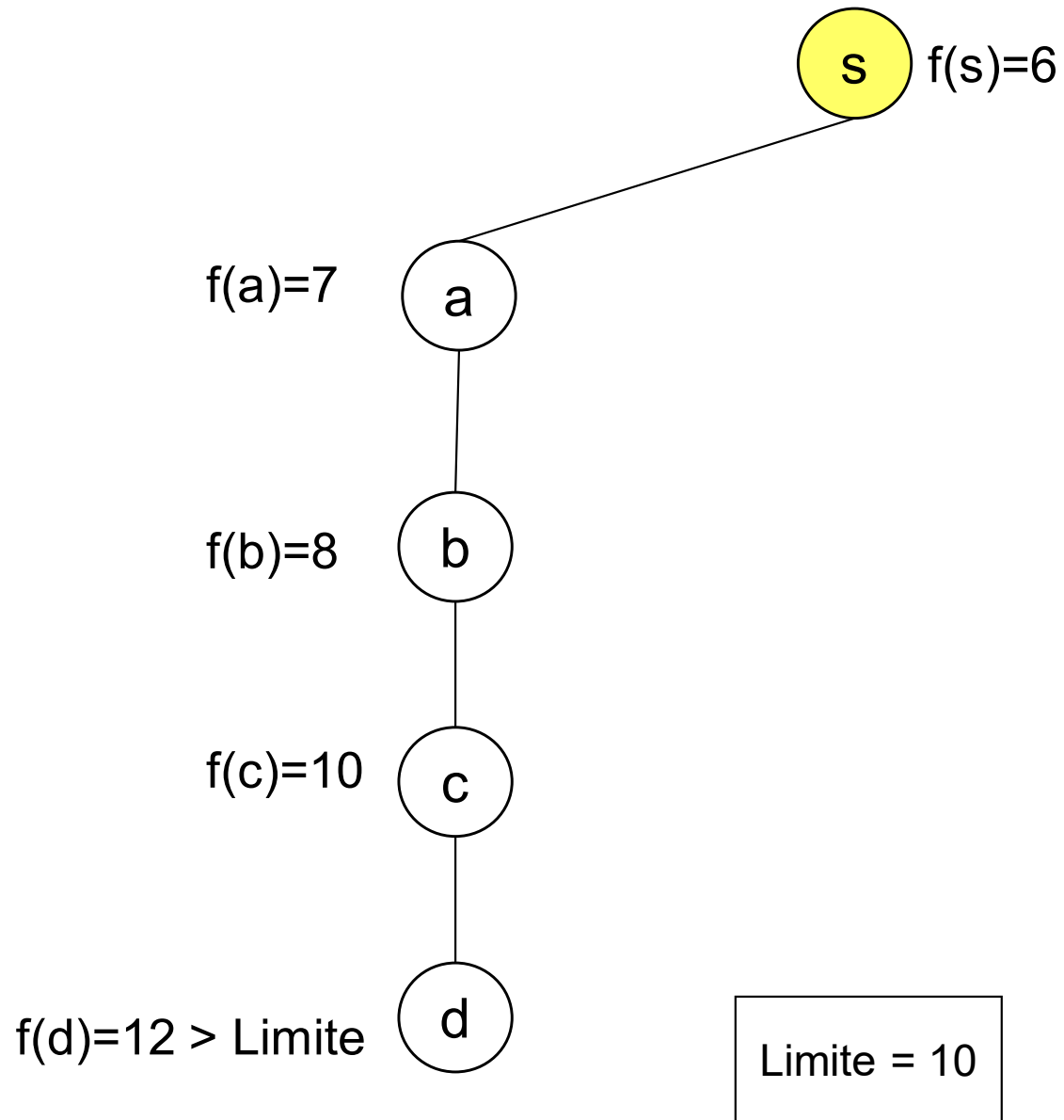
Limite = 10

IDA*

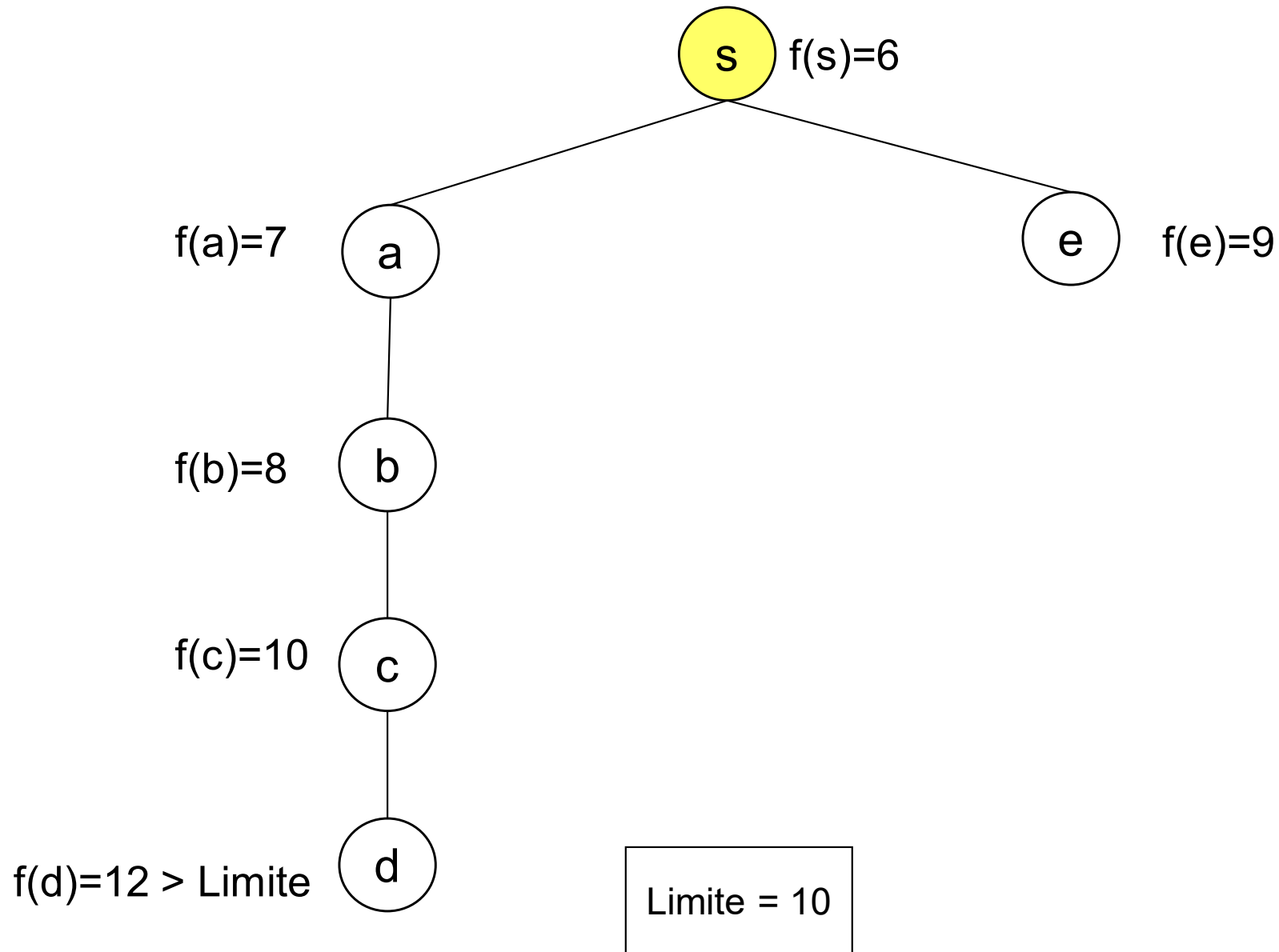


Limite = 10

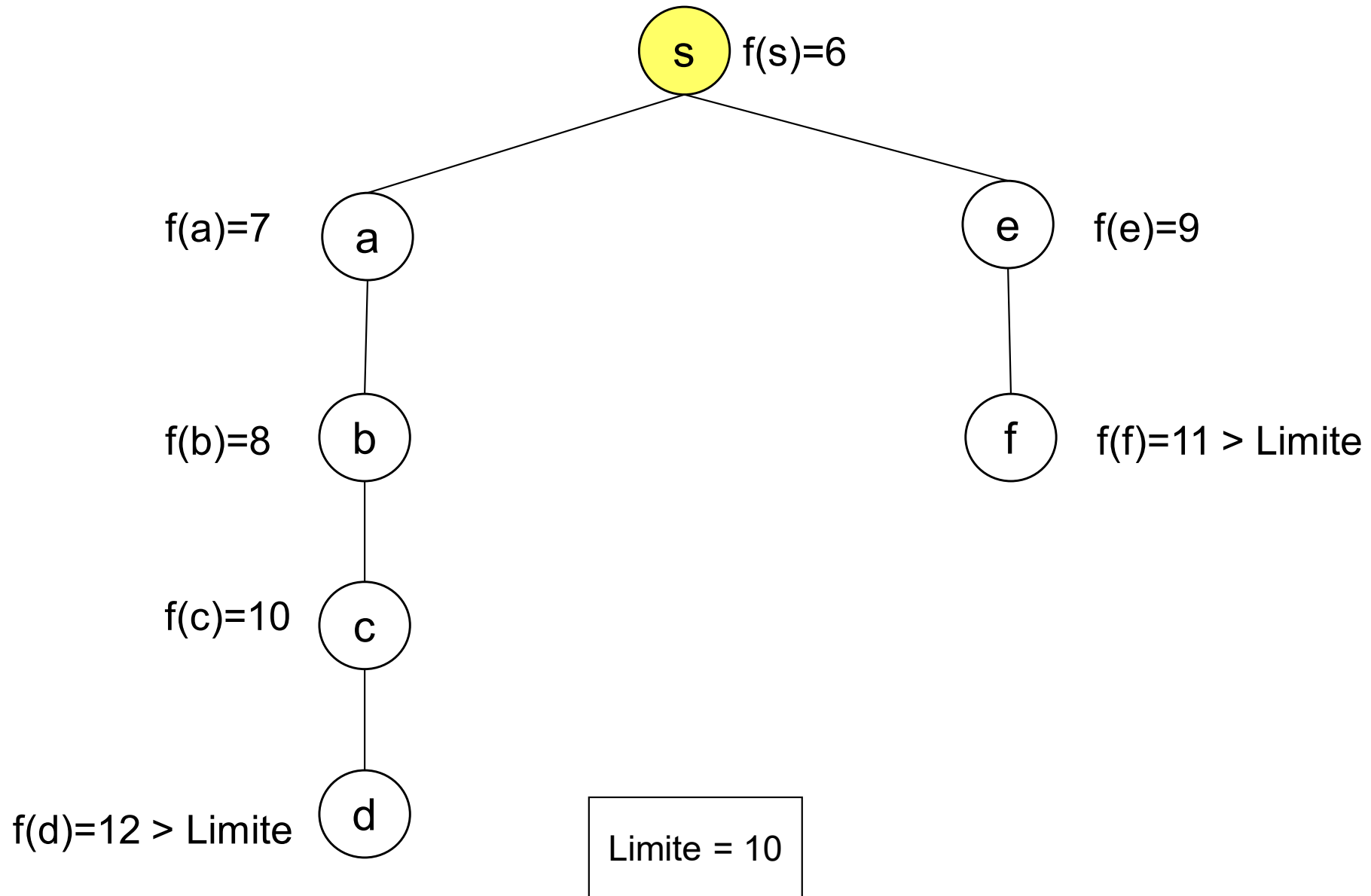
IDA*



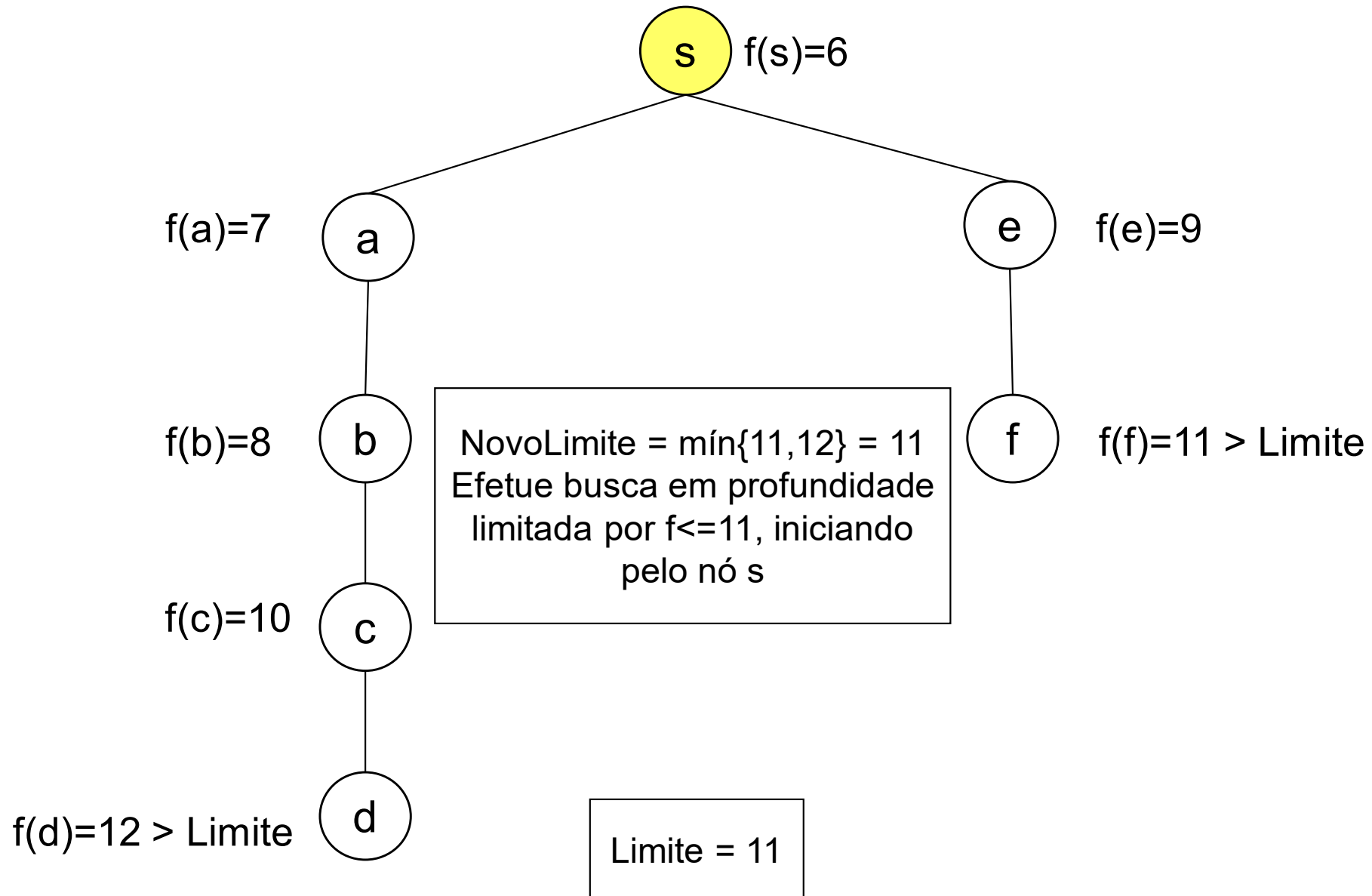
IDA*



IDA*



IDA*

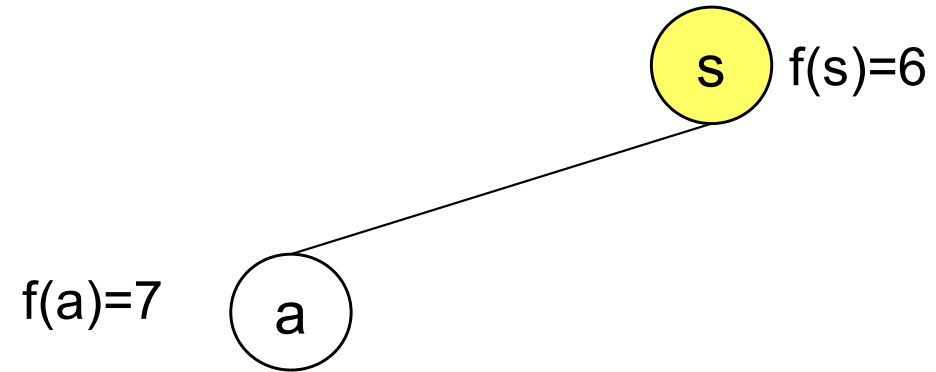


IDA*

s $f(s)=6$

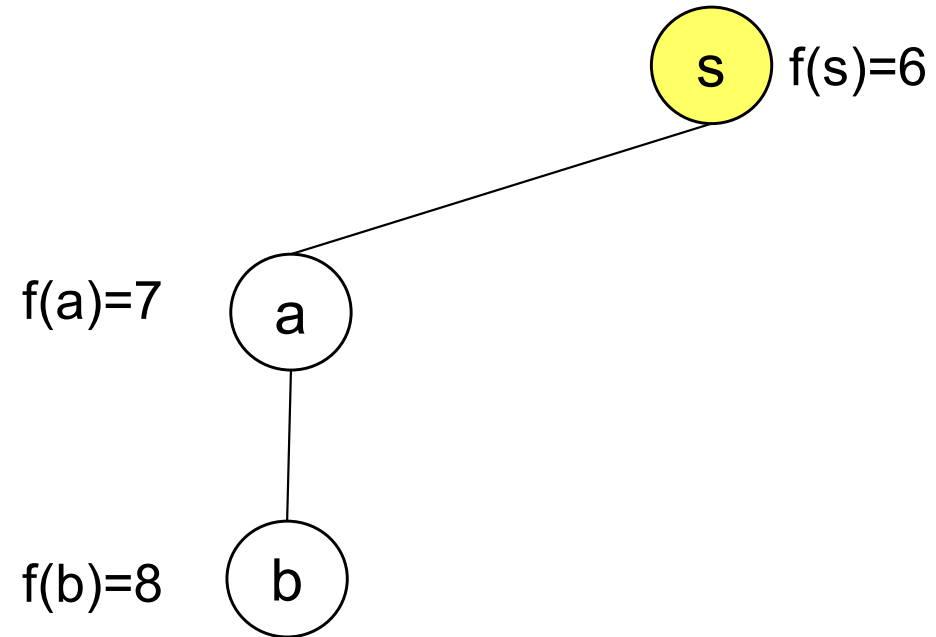
Limite = 11

IDA*



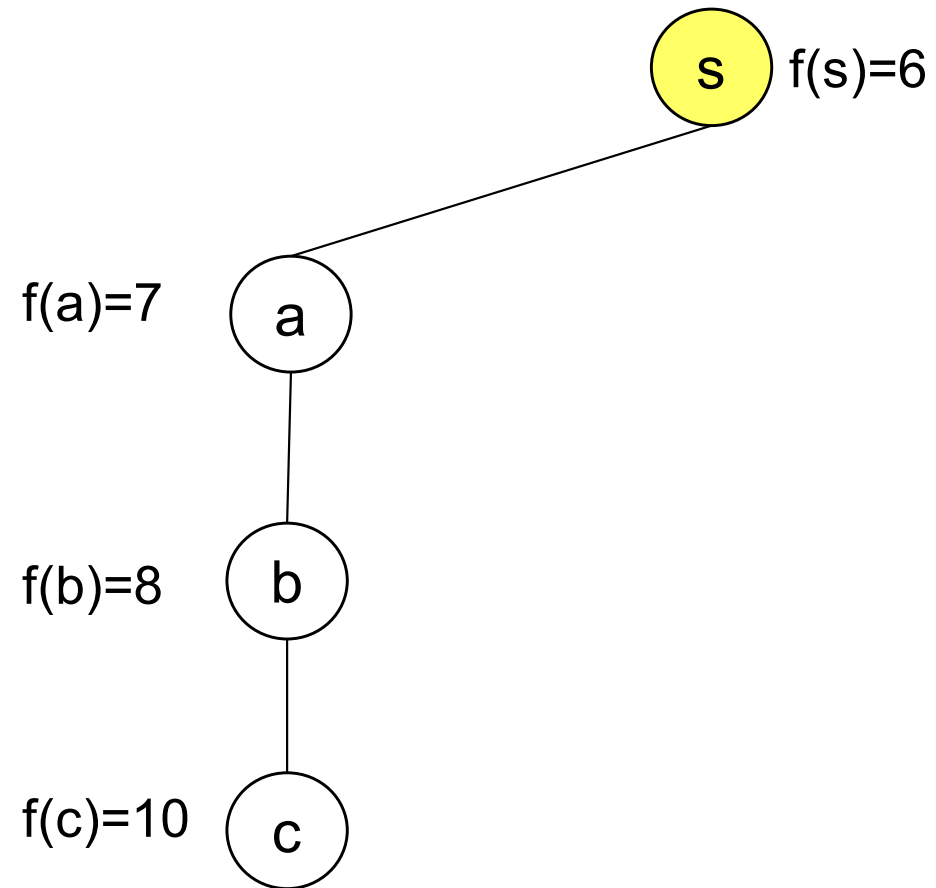
Limite = 11

IDA*



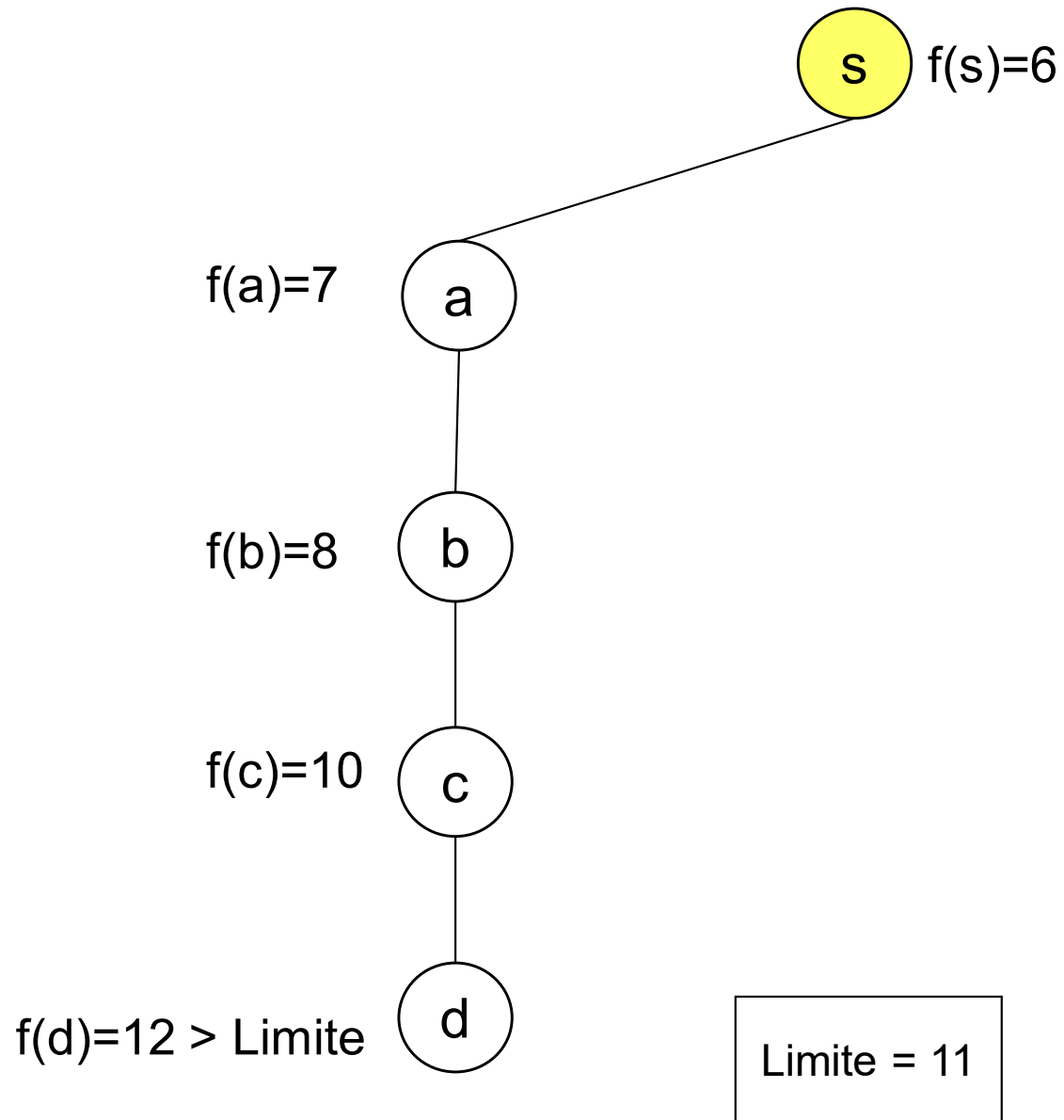
Limite = 11

IDA*

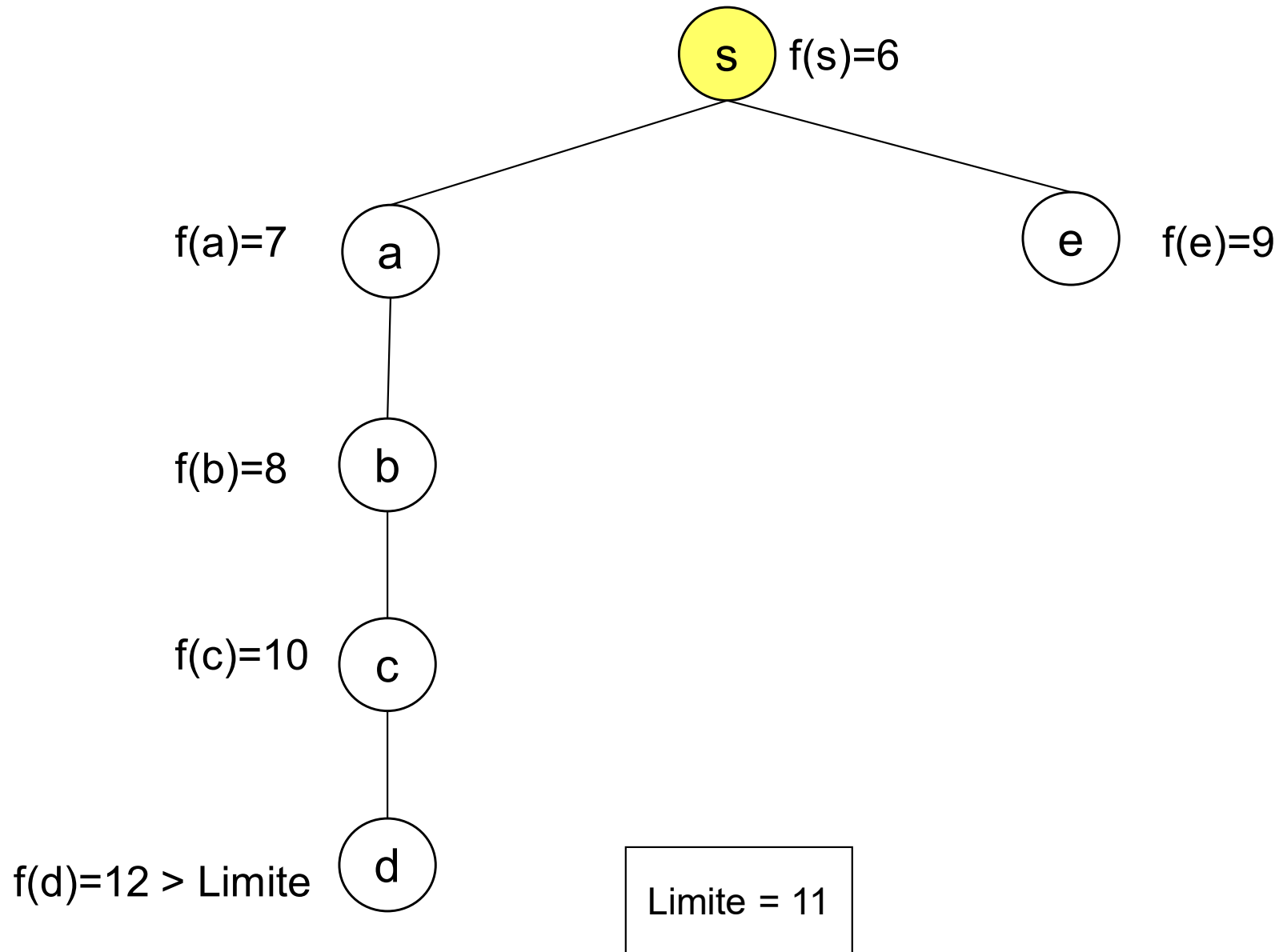


Limite = 11

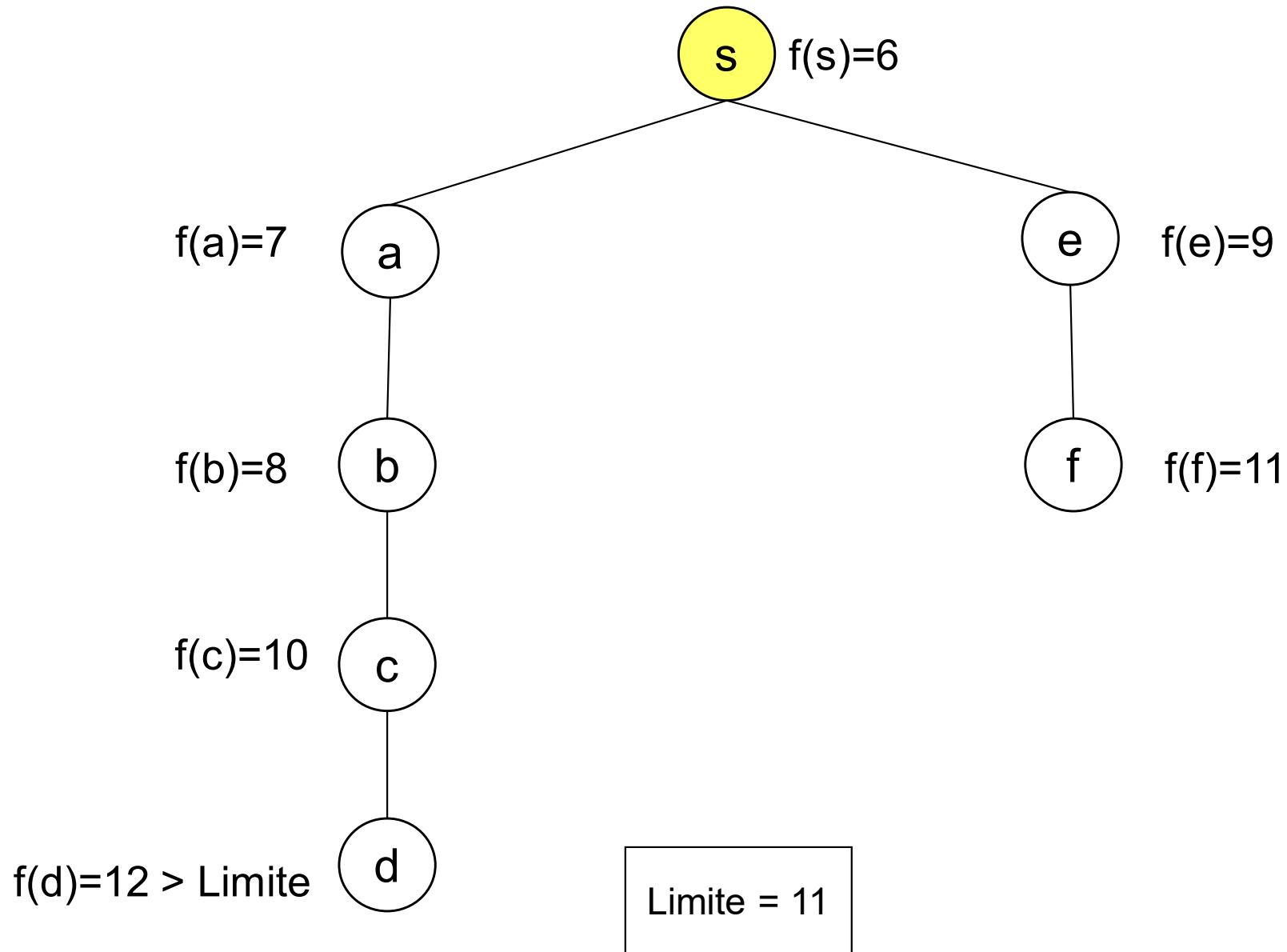
IDA*



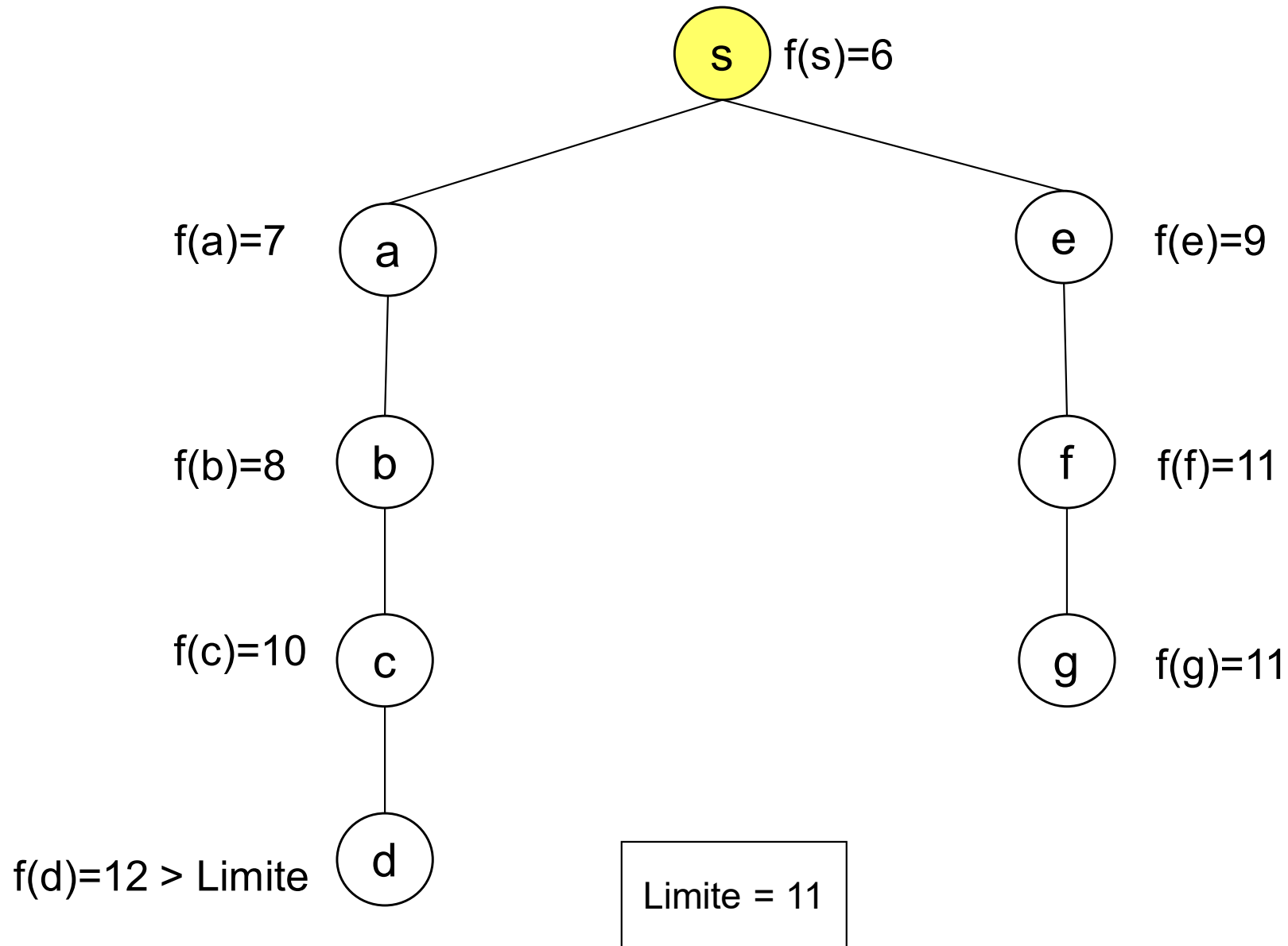
IDA*



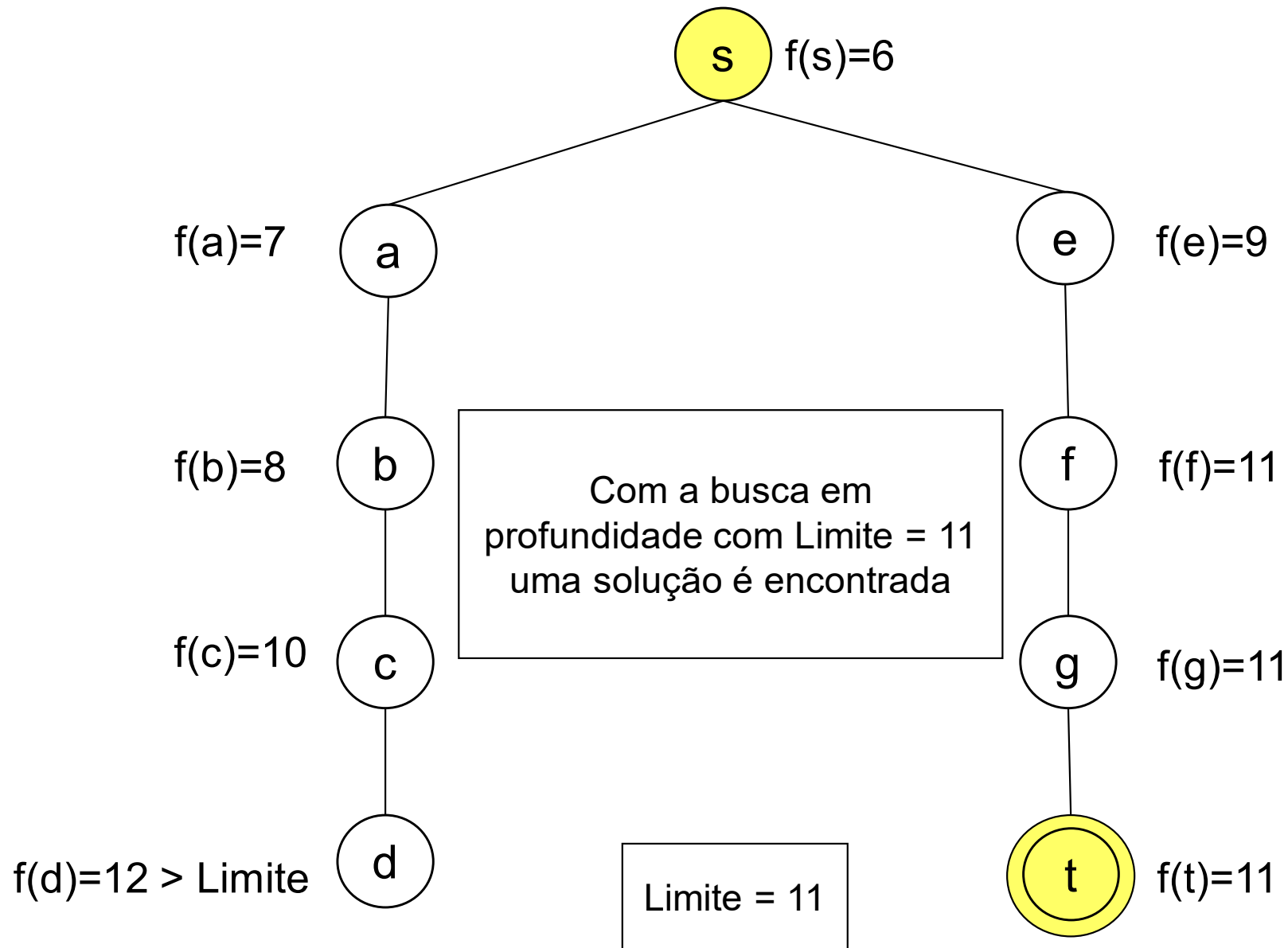
IDA*



IDA*



IDA*



IDA*

```
% Assuma que 9999 é maior que qualquer valor-f  
:- dynamic proximo_limite/1, solucao/1.
```

```
resolvai2(No, Solucao) :-  
    retract(proximo_limite(_)), % limpa proximo limite  
    fail  
    ;  
    assert(proximo_limite(0)), % inicializa proximo limite  
    idastar(1(No, 0/0), Solucao).
```

```
idastar(1(N, F/G), Solucao) :-  
    retract(proximo_limite(Limite)),  
    assert(proximo_limite(9999)),  
    df(1(N, F/G), [N], Limite, Solucao).  
idastar(No, Solucao) :-  
    proximo_limite(Limite),  
    Limite < 9999,  
    idastar(No, Solucao).
```

IDA*

```
% df(No,Caminho,Limite,Solucao)
% Realiza busca em profundidade dentro de Limite
% Caminho é um caminho entre nó inicial ate o No atual
% F e' o valor-f do no atual que se encontra no inicio do Caminho

% Caso 1: nó N final dentro de Limite, construir caminho da solucao
df(l(N,F/G),[N|P],Limite,[N|P]) :-
    F =< Limite,
    final(N).

% Caso 2: nó N com valor-f <= Limite
% Gerar sucessor de N e expandir dentro de Limite
df(l(N,F/G),[N|P],Limite,Solucao) :-
    F =< Limite,
    s(N,M,Custo),
    \+ pertence(M,P),
    Gm is G + Custo,      % avaliar no' M
    h(M,Hm),
    Fm is Gm + Hm,
    df(l(M,Fm/Gm),[M,N|P],Limite,Solucao).
```

IDA*

```
% Caso 3: valor f > Limite, atualizar proximo limite
% e falhar
df(1(N,F/G),_,Limite,_) :-
    F > Limite,
    atualize_proximo_limite(F),
    fail.

atualize_proximo_limite(F) :-
    proximo_limite(Limite),
    Limite =< F, !                               % nao altere proximo limite
;
    retract(proximo_limite(Limite)),!,         % diminua proximo limite
    assert(proximo_limite(F)).

pertence(E, [E|_]).
pertence(E, [_|T]) :-
    pertence(E,T).
```

IDA*

- Uma propriedade interessante de IDA* refere-se à sua admissibilidade
 - Assumindo $f(n) = g(n) + h(n)$, se h é admissível ($h(n) \leq h^*(n)$) então é garantido que IDA* encontre uma solução ótima
- Entretanto, não é garantido que IDA* explore os nós mesma ordem que A* (ou seja, na ordem de valores- f crescentes)
 - Quando f não é da forma $f = g + h$ e f é não monotônica

RBFS

- ❑ Vimos que IDA* possui uma implementação simples
- ❑ Entretanto, no pior caso, quando os valores-f não são compartilhados entre vários nós então muitos limites sucessivos de valores-f são necessários e a nova busca em profundidade expandirá apenas um novo nó enquanto todos os demais são apenas re-expansões de nós expandidos e esquecidos
- ❑ Nessa situação existe uma técnica para economizar espaço denominada RBFS (*recursive best-first search*)

RBFS

- ❑ RBFS é similar a A^* , mas enquanto A^* mantém em memória todos os nós expandidos, RBFS apenas mantém o caminho atual assim como seus irmãos
- ❑ Quando RBFS suspende temporariamente um subprocesso de busca (porque ele deixou de ser o melhor), ela ‘esquece’ a subárvore de busca para economizar espaço
- ❑ Assim como IDA^* , RBFS é apenas linear na profundidade do espaço de estados em quantidade de memória necessária

RBFS

- ❑ O único fato que RBFS armazena sobre a subárvore de busca abandonada é o valor-f atualizado da raiz da subárvore
- ❑ Os valores-f são atualizados copiando-se os valores-f de forma similar ao algoritmo A^*
- ❑ Para distinguir entre os valores-f estáticos e aqueles copiados, usaremos:
 - $f(\mathbf{n})$ = valor-f do nó $\mathbf{n}$ utilizando a função de avaliação (sempre o mesmo valor durante a busca)
 - $F(\mathbf{n})$ = valor-f copiado (é alterado durante a busca uma vez que depende dos nós descendentes de $\mathbf{n}$)
- ❑ $F(\mathbf{n})$ é definida como:
 - $F(\mathbf{n}) = f(\mathbf{n})$ se $\mathbf{n}$ nunca foi expandido durante a busca
 - $F(\mathbf{n}) = \min\{F(\mathbf{n}_i) : \mathbf{n}_i \text{ é um sucessor de } \mathbf{n}\}$

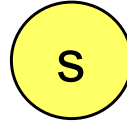
RBFS

- ❑ Assim como A^* , RBFS explora subárvores dentro de um limite de valor-f
- ❑ O limite é determinado pelos valores-F dos filhos ao longo do caminho atual (o melhor valor-F dos filhos, ou seja, o valor-F do competidor mais promissor do nó atual)
- ❑ Seja **n** o melhor nó (aquele com menor valor-F)
 - Então **n** é expandido e seus filhos são explorados até algum limite de valor-f
 - Quando o limite é excedido ($F(\mathbf{n}) > \text{Limite}$) então todos os nós expandidos a partir de **n** são 'esquecidos'
 - Entretanto, o valor $F(\mathbf{n})$ atualizado é retido e utilizado na decisão em como a busca deve continuar

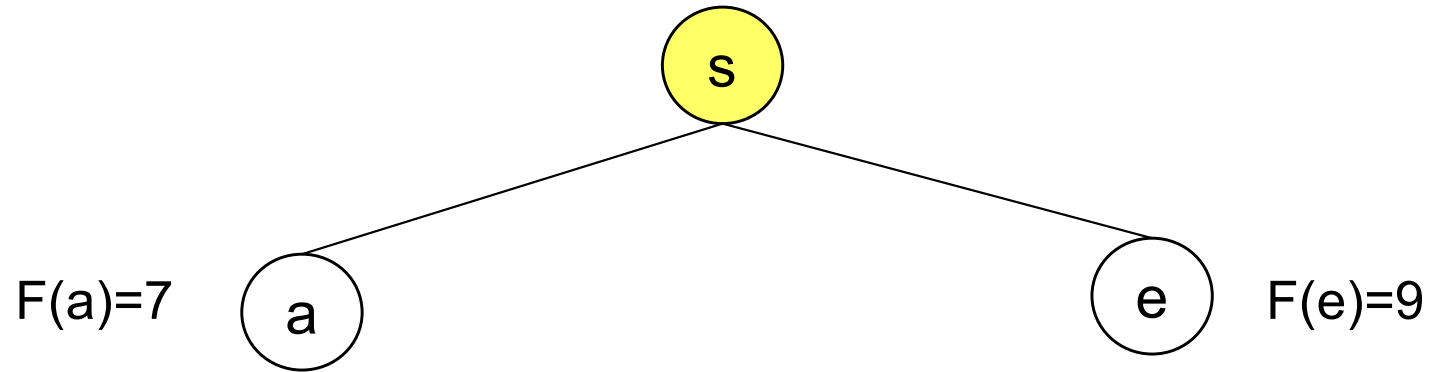
RBFS

- ❑ Os valores-F são determinados não apenas copiando os valores obtidos a partir de um dos filhos mas também são herdados dos nós pais da seguinte forma
- ❑ Seja n um nó que deve ser expandido pela busca
- ❑ Se $F(n) > f(n)$ então sabemos que n deve ter sido expandido anteriormente e que $F(n)$ foi determinado a partir dos filhos de n , mas os filhos foram removidos da memória
- ❑ Suponha que um filho n_i de n seja novamente expandido e o valor estático $f(n_i)$ seja calculado novamente
- ❑ Então $F(n_i)$ é determinado como sendo
 - if $f(n_i) < F(n)$ then $F(n_i) \leftarrow F(n)$ else $F(n_i) \leftarrow f(n_i)$
- ❑ que pode ser escrito como:
 - $F(n_i) \leftarrow \max\{F(n), f(n_i)\}$

RBFS



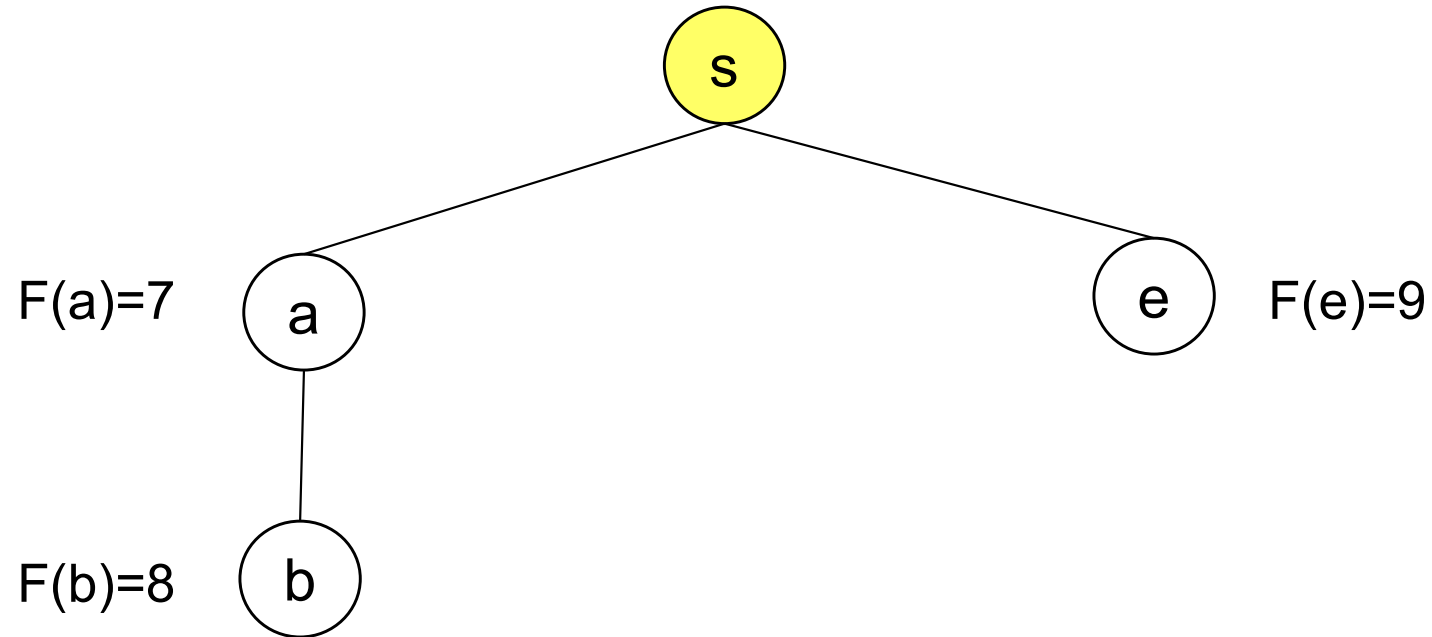
RBFS



O melhor candidato é o nó **a**,
pois $F(a) < F(e)$. A busca
prossegue via **a**

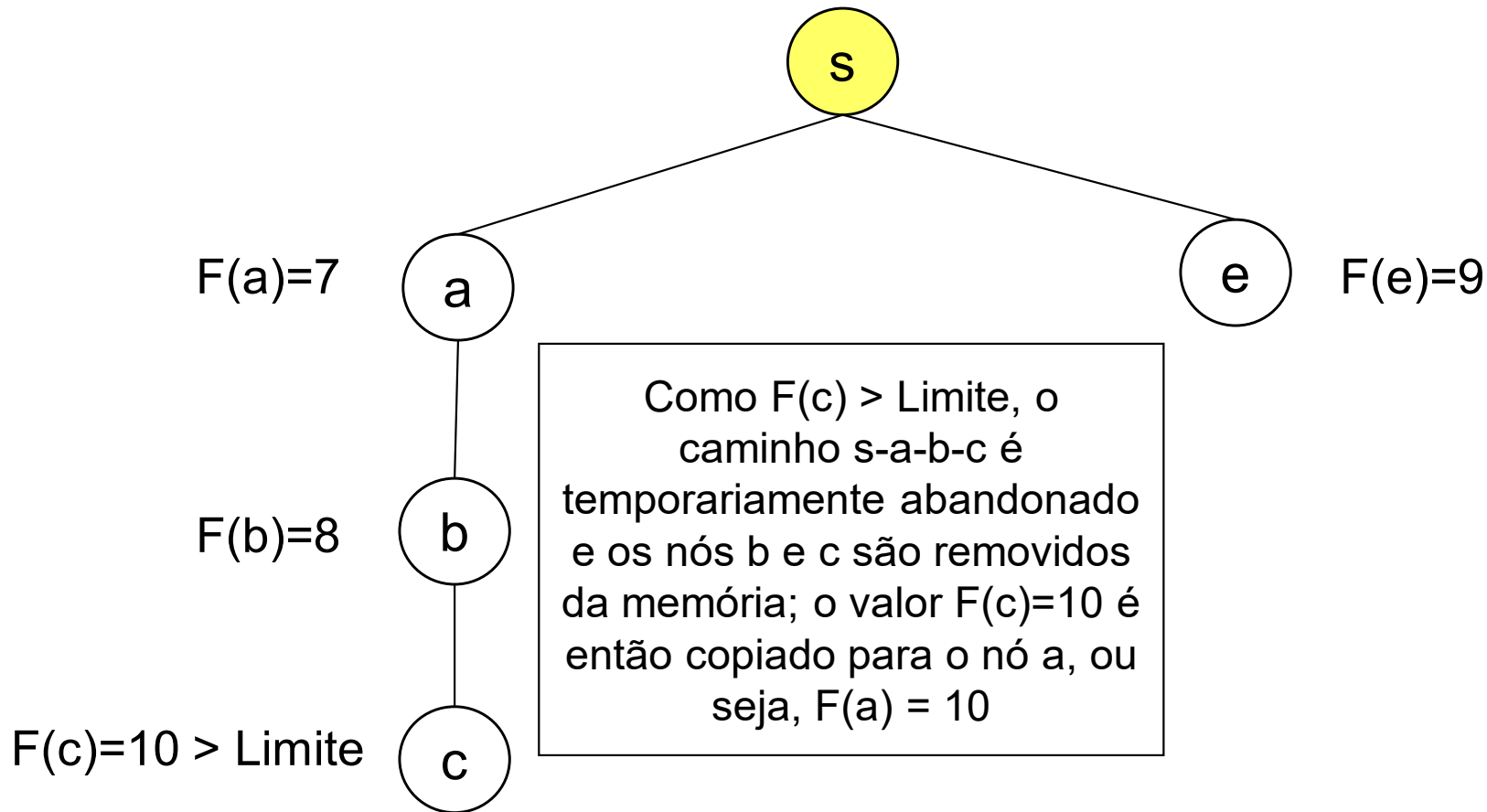
Limite = 9

RBFS



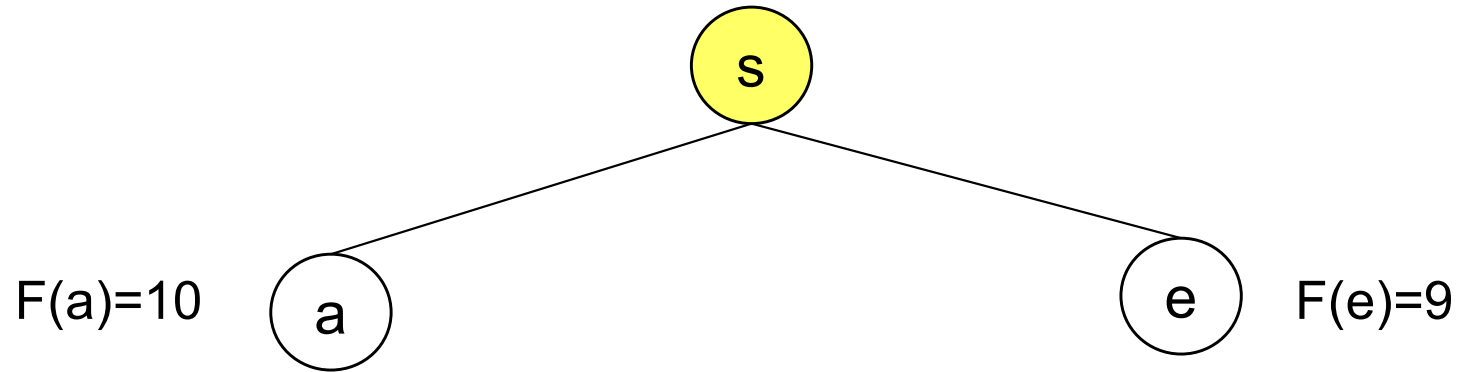
Limite = 9

RBFS



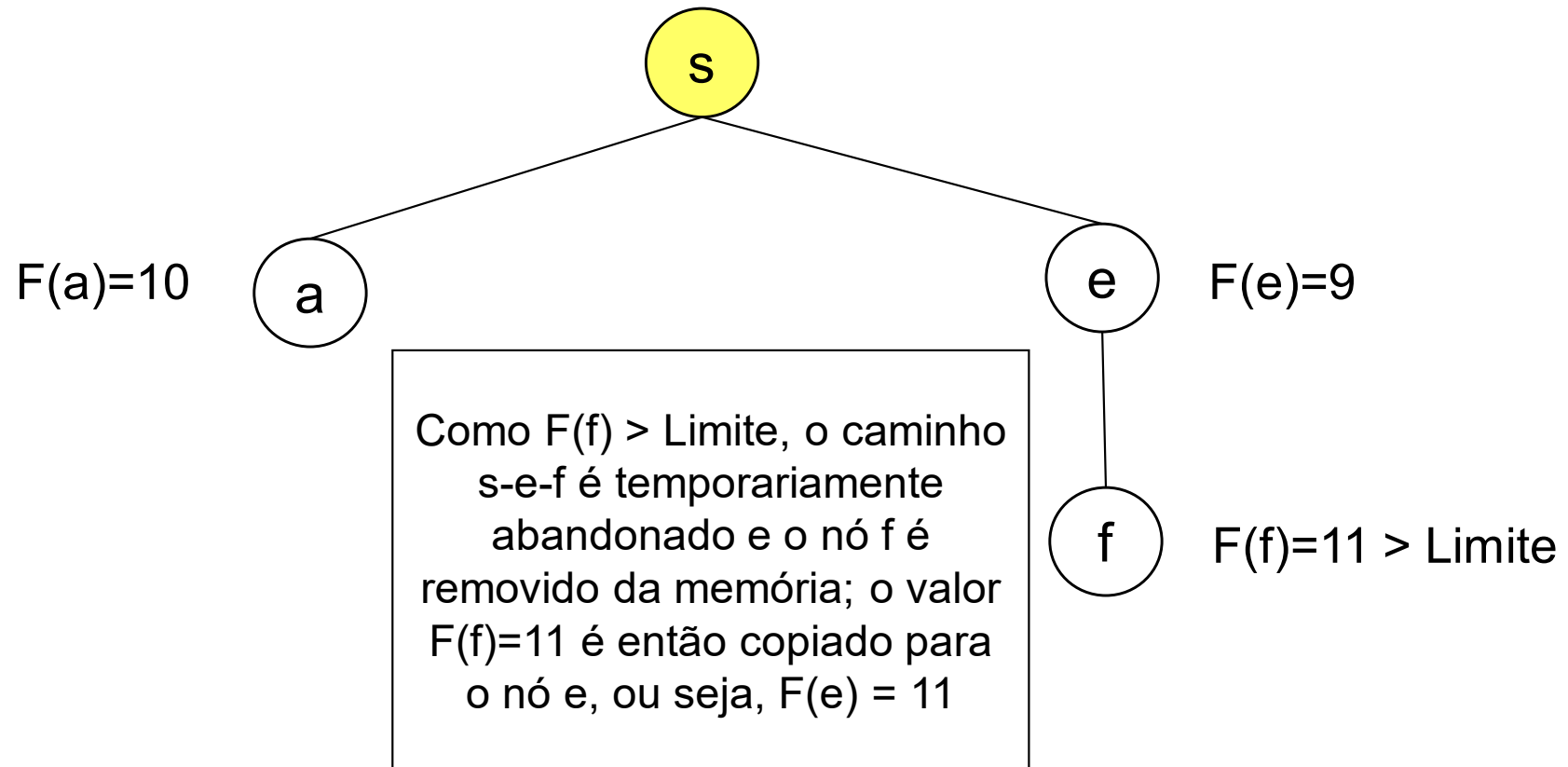
Limite = 9

RBFS

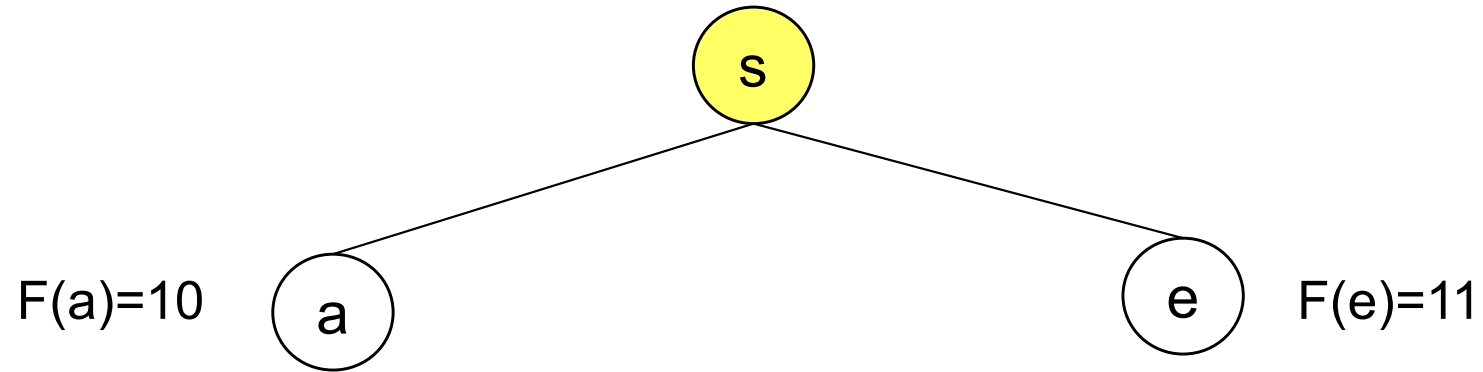


Limite = 10

RBFS

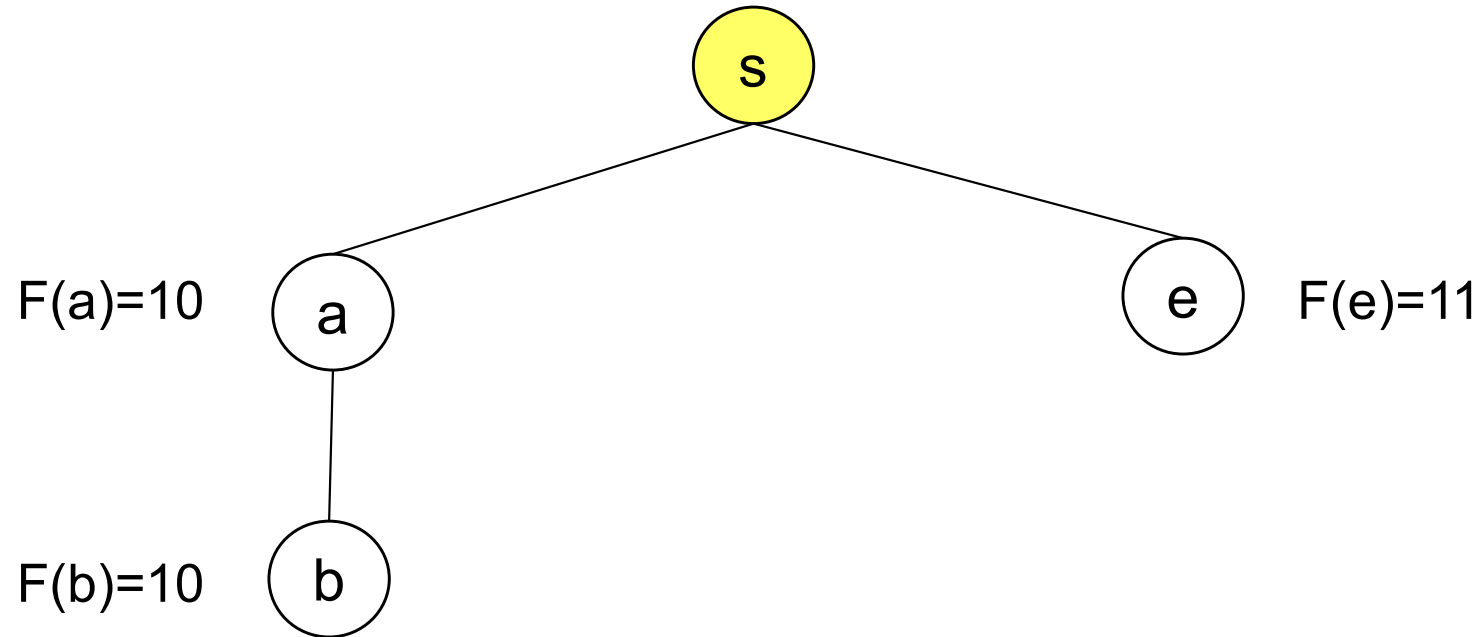


RBFS



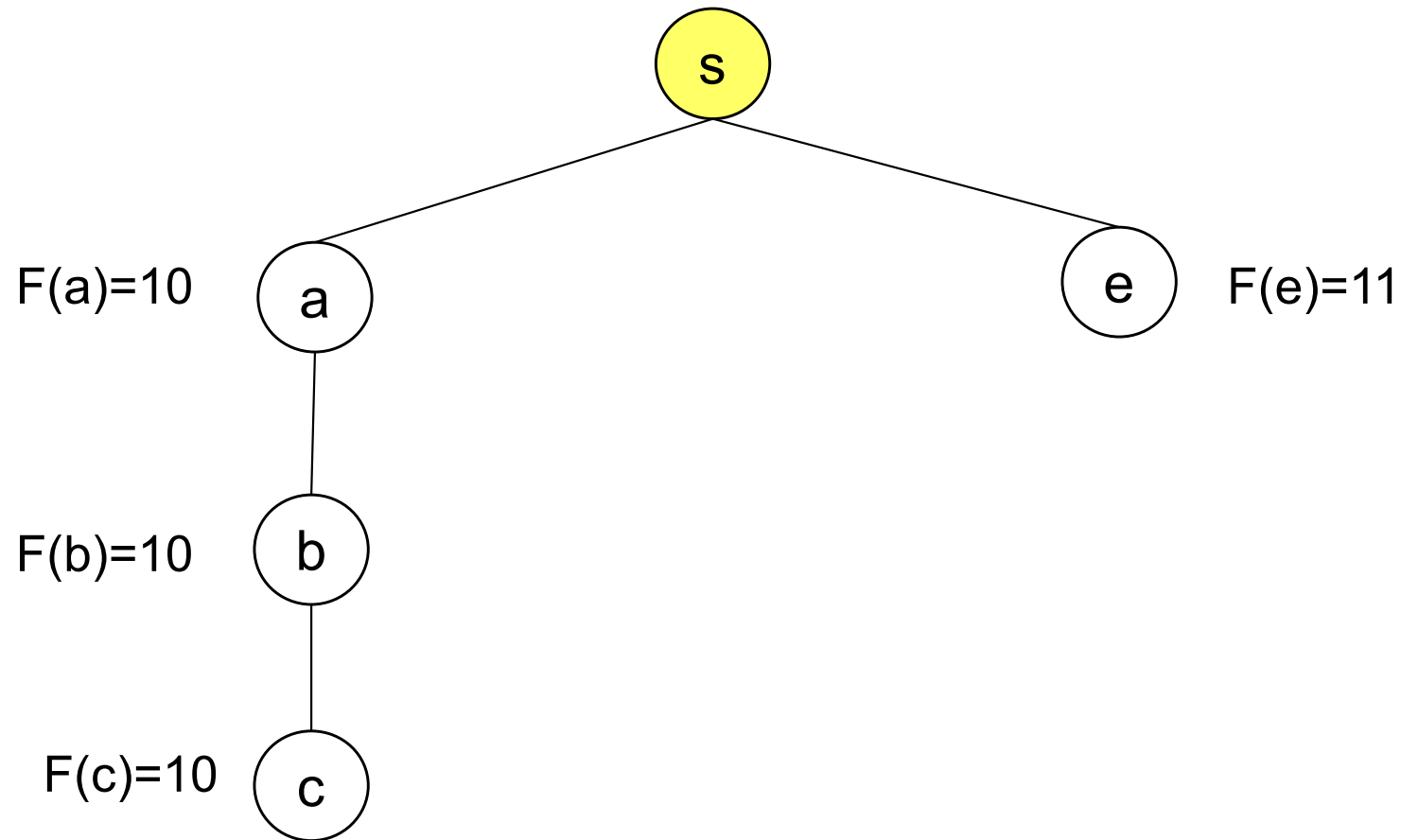
Limite = 11

RBFS



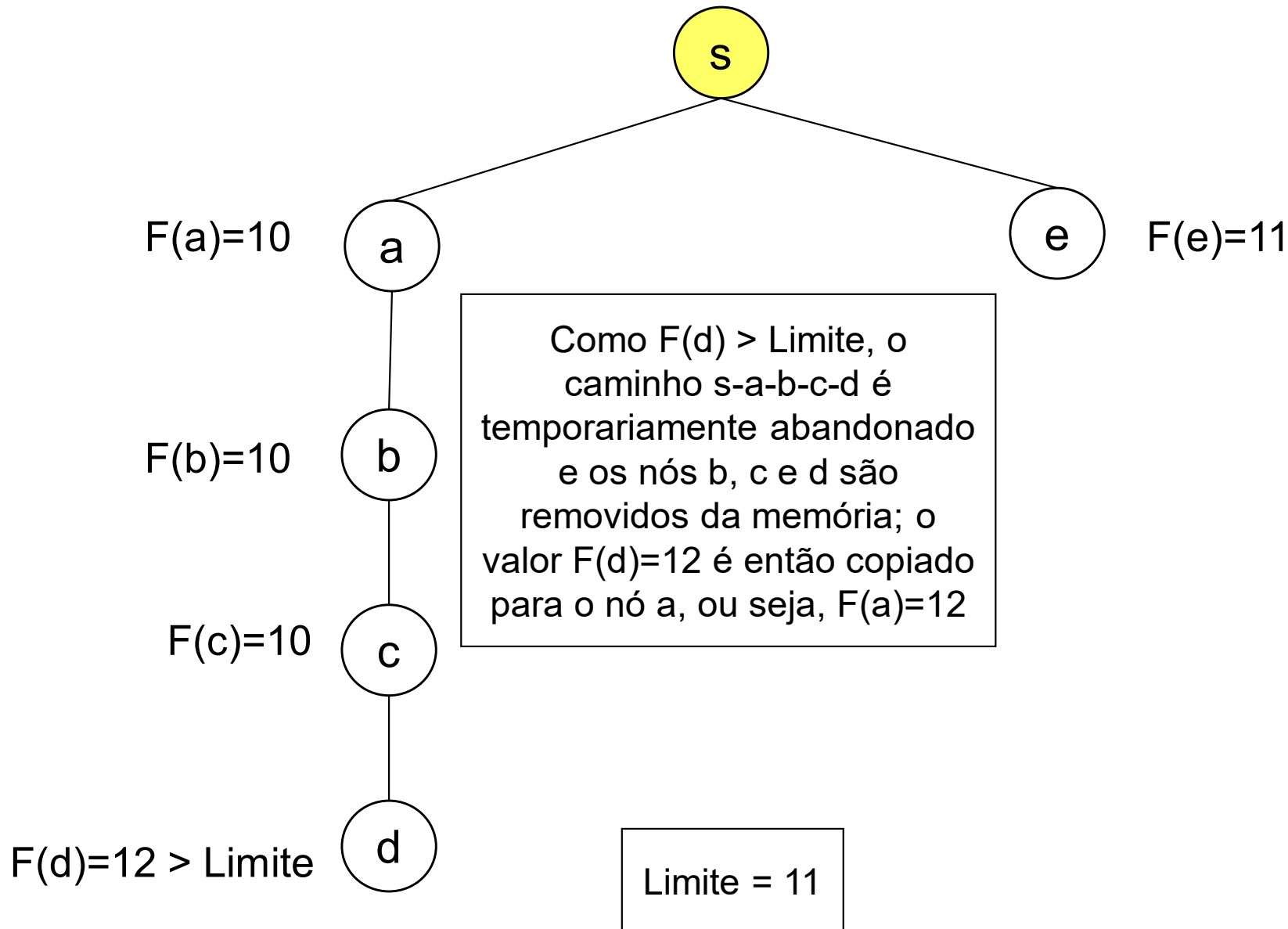
Limite = 11

RBFS

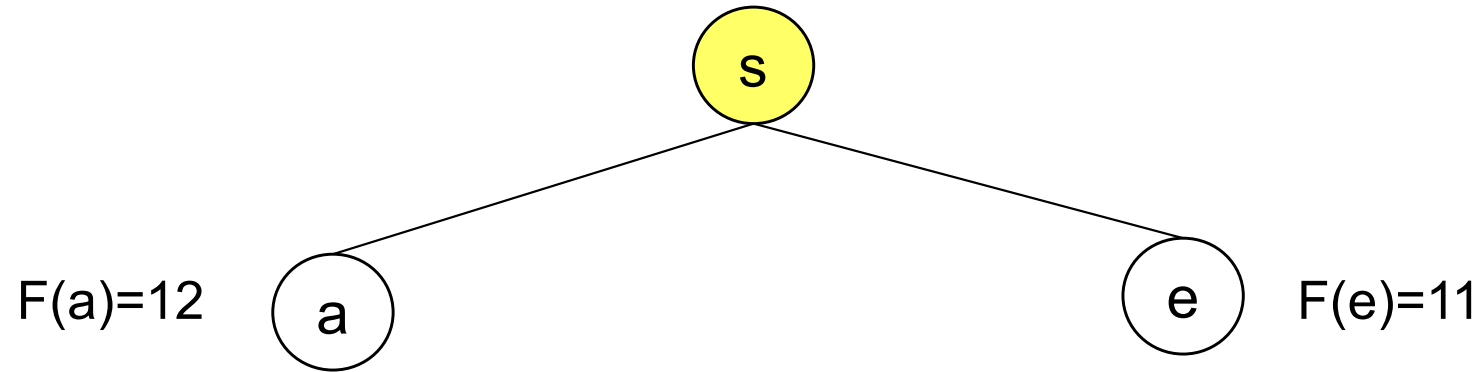


Limite = 11

RBFS

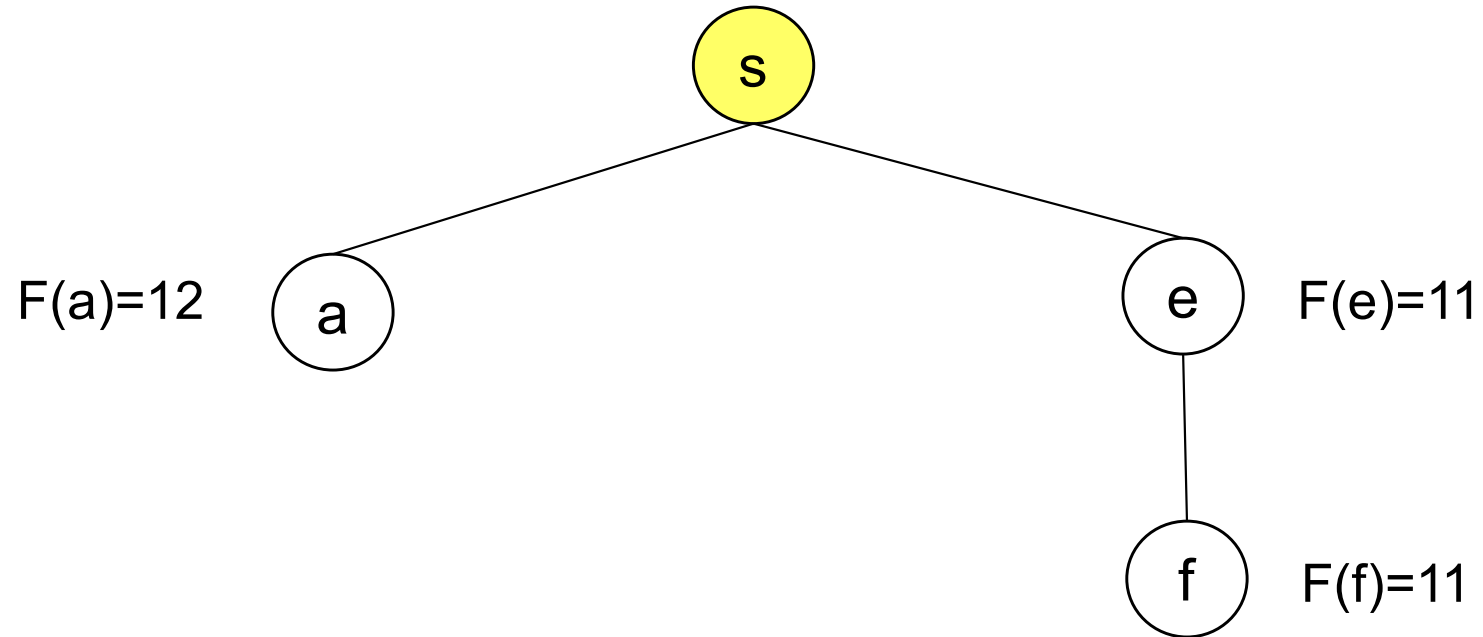


RBFS



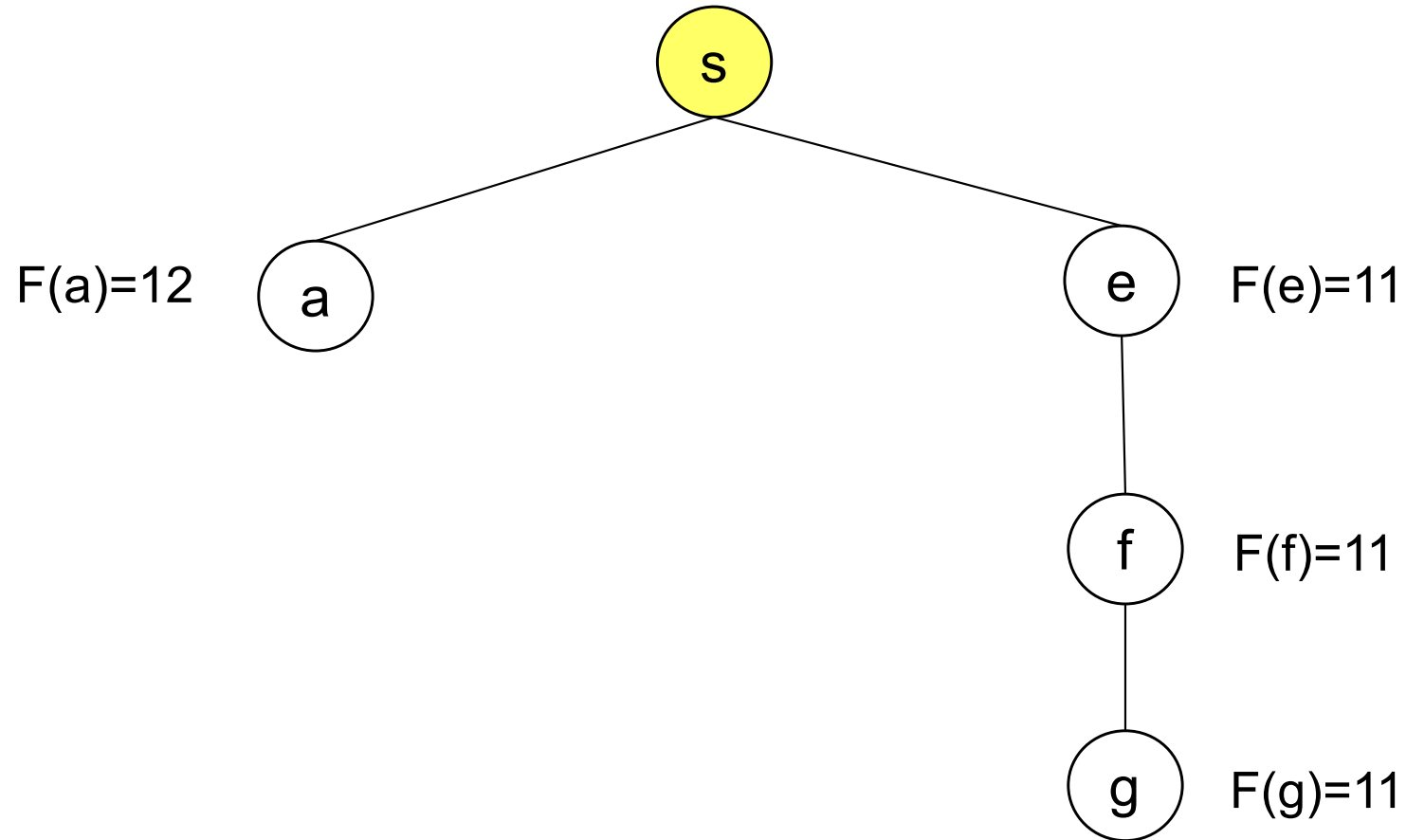
Limite = 12

RBFS



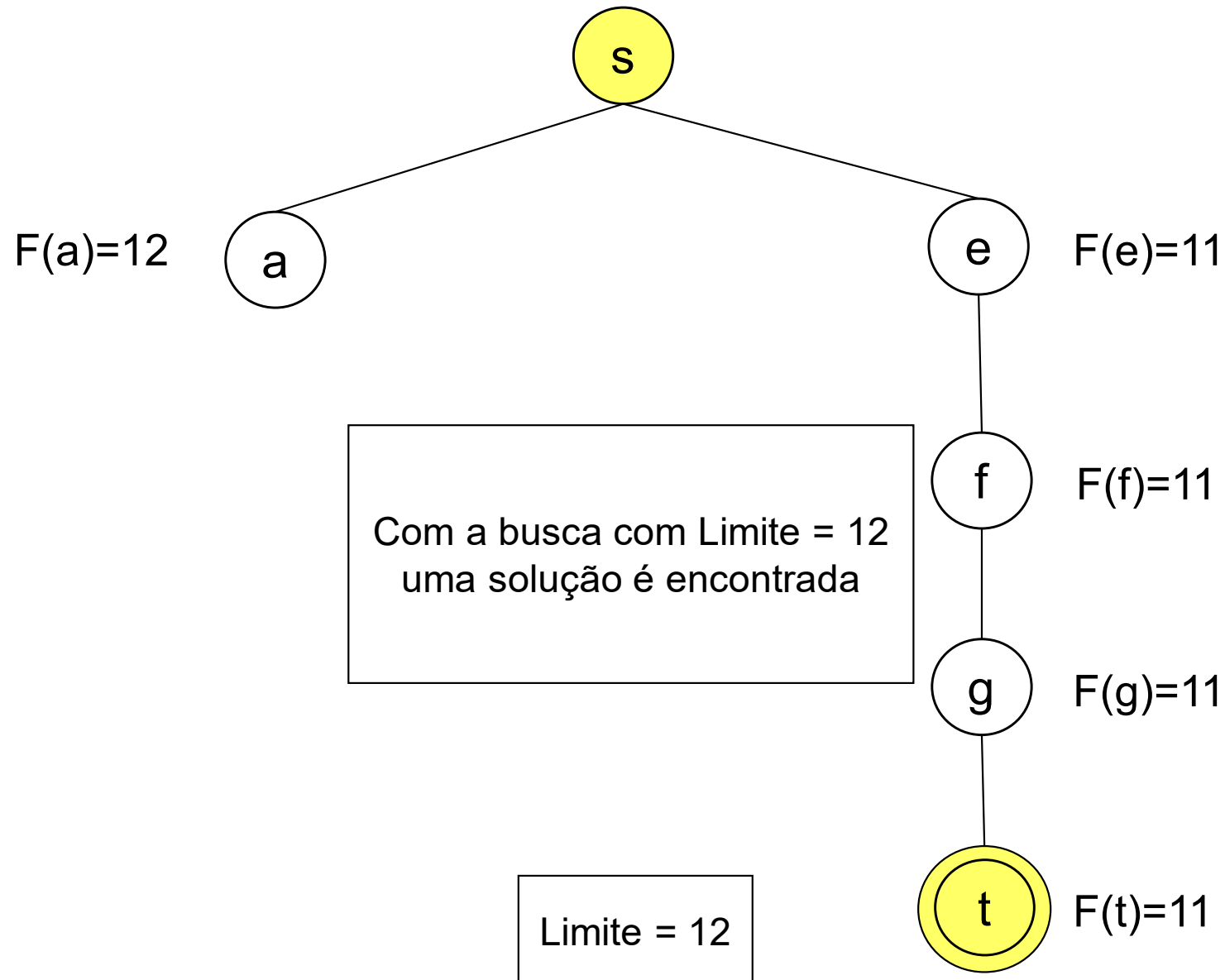
Limite = 12

RBFS



Limite = 12

RBFS



RBFS

- ❑ `rbfs(Caminho, Filhos, Limite, NovoMelhorFF, Resolvido, Solucao):`
 - Caminho = caminho até então na ordem reversa
 - Filhos = filhos da cabeça do Caminho
 - Limite = limite superior no valor-F da busca para os Filhos
 - NovoMelhorFF = melhor valor-f justamente quando busca ultrapassa Limite
 - Resolvido = sim, não, nunca
 - Solucao = caminho da solução, se Resolvido = sim
- ❑ Representacao dos nos: $No = I(Estado, G/F/FF)$
 - G é o custo até Estado
 - F é o valor-f estático de Estado
 - FF é o valor-f de Estado copiado

RBFS

```
% Assuma que 9999 é maior que qualquer valor-f
resolvai3(No,Solucao) :-
    rbfs([], [l(No,0/0/0)], 9999,_,sim,Solucao).

% rbfs(Caminho,Filhos,Limite,NovoMelhorFF,Resolvido,Solucao)
rbfs(Caminho,[l(No,G/F/FF)|Nos],Limite,FF,nao,_) :-
    FF > Limite, !.
rbfs(Caminho,[l(No,G/F/FF)|_],_,_,sim,[No|Caminho]) :-
    F = FF, % Mostrar solucao apenas uma vez,quando F=FF
    final(No).
rbfs(_,[],_,_,nunca,_) :- !. % Sem candidatos, beco sem saida
rbfs(Caminho,[l(No,G/F/FF)|Ns],Limite,NovoFF,Resolvido,Sol) :-
    FF =< Limite, % Dentro de Limite: gerar filhos
    findall(Filho/Custo,
        (s(No,Filho,Custo), \+ pertence(Filho,Caminho)),
        Filhos),
    herdar(F,FF,FFherdado), % Filhos podem herdar FF
    avalie(G,FFherdado,Filhos,Sucessores), % Ordenar filhos
    melhorff(Ns,ProximoMelhorFF), % FF do competidor mais promissor dos filhos
    min(Limite,ProximoMelhorFF,Limite2), !,
    rbfs([No|Caminho],Sucessores,Limite2,NovoFF2,Resolvido2,Sol),
    continue(Caminho,[l(No,G/F/NovoFF2)|Ns],Limite,
        NovoFF,Resolvido2,Resolvido,Sol).
```

RBFS

```
% continue(Caminho,Nos,Limite,NovoFF,FilhoResolvido,Resolvido,Solucao)
continue(Caminho,[N|Ns],Limite,NovoFF,nunca,Resolvido,Sol) :-
    !,
    rbfs(Caminho,Ns,Limite,NovoFF,Resolvido,Sol). % N é um beco sem saida
continue(_,_,_,_,sim,sim,Sol).
continue(Caminho,[N|Ns],Limite,NovoFF,nao,Resolvido,Sol) :-
    inserir(N,Ns,NovoNs), !,% Assegurar que filhos sao ordenados pelos valores
    rbfs(Caminho,NovoNs,Limite,NovoFF,Resolvido,Sol).

avalie(_,_,[],[]).
avalie(G0,FFherdado,[No/C|NCs],Nos) :-
    G is G0 + C,
    h(No,H),
    F is G + H,
    max(F,FFherdado,FF),
    avalie(G0,FFherdado,NCs,Nos2),
    inserir(l(No,G/F/FF),Nos2,Nos).

herdar(F,FF,FF) :- % Filho herda FF do pai se
    FF > F, !.      % FF do pai e' maior que F do pai
herdar(F,FF,0).
```

RBFS

```
insrir(l(N,G/F/FF),Nos,[l(N,G/F/FF)|Nos]) :-  
    melhorff(Nos,FF2),  
    FF =< FF2, !.  
insrir(N,[N1|Ns],[N1|Ns1]) :-  
    insrir(N,Ns,Ns1).  
  
melhorff([l(N,F/G/FF)|Ns],FF). % Primeiro no' = melhor FF  
melhorff([],9999). % Sem nos FF = "infinito"  
  
pertence(E,[E|_]).  
pertence(E,[_|T]) :-  
    pertence(E,T).  
  
min(X,Y,X) :-  
    X =< Y, !.  
min(X,Y,Y).  
  
max(X,Y,X) :-  
    X >= Y, !.  
max(X,Y,Y).
```

Algoritmos de Busca Local

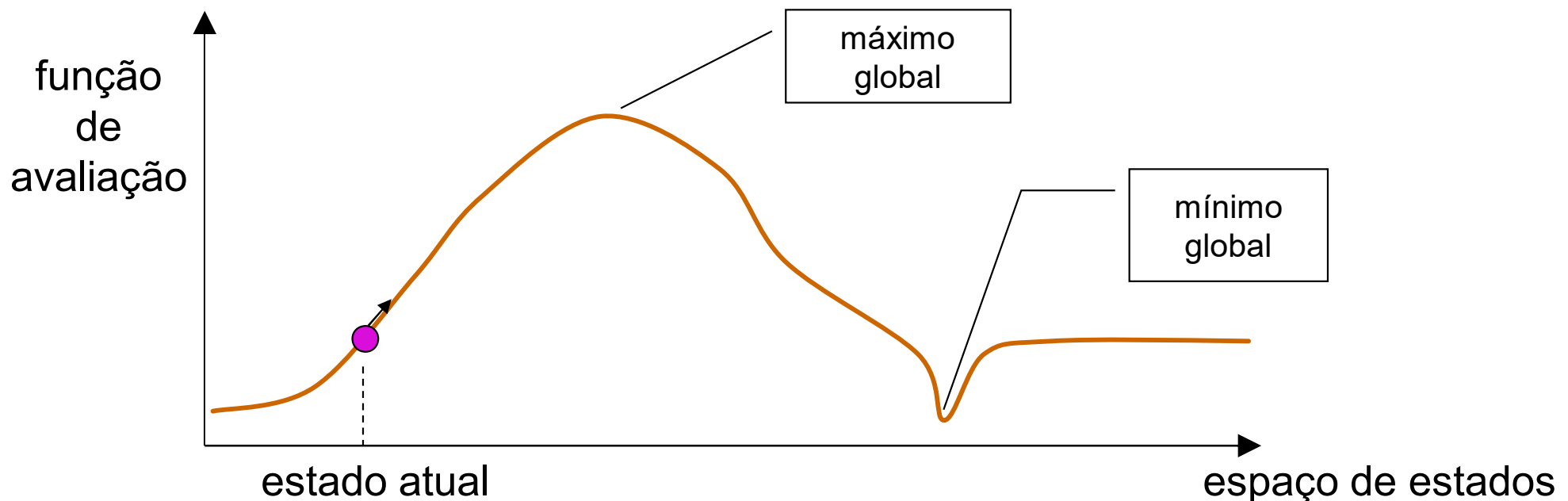
- ❑ Os algoritmos de busca (informada ou não) exploram sistematicamente o espaço de busca, mantendo um ou mais caminhos na memória
 - Quando um nó final é encontrado, o caminho até ele constitui uma solução para o problema
- ❑ Todavia, em algumas situações, o caminho até o nó final é irrelevante
 - No problema da 8-rainhas o que importa é a configuração final no tabuleiro e não a ordem em que as rainhas são acrescentadas
 - Outros exemplos incluem projetos de circuitos integrados, otimização de rede de comunicação, roteamento de veículos, etc

Algoritmos de Busca Local

- ❑ Dessa forma, se o caminho não é importante, podemos considerar uma classe diferente de algoritmos de busca
- ❑ Os algoritmos de **busca local** trabalham usando um **único estado corrente** (ao invés de vários caminhos) e, em geral, se movem apenas para os vizinhos desse estado
 - Normalmente, os caminhos não são armazenados
- ❑ Embora os algoritmos de busca local não sejam sistemáticos, eles apresentam vantagens:
 - Utilizam pouca memória (em geral, um valor constante)
 - Podem encontrar soluções razoáveis em espaços muito grande ou infinito, para os quais os algoritmos sistemáticos não são adequados
- ❑ Além de encontrar estados finais, os algoritmos de busca local são úteis para resolver **problemas de otimização**, nos quais o objetivo é encontrar o melhor estado de acordo com uma função objetivo

Algoritmos de Busca Local

- Uma forma de entender os algoritmos de busca local consiste em considerar a **topologia do espaço de estados** definida pela posição no espaço de estados versus a elevação, definida pelo valor da função de avaliação
 - Se a elevação corresponder ao custo, o objetivo será encontrar um **mínimo global** (vale)
 - Se a elevação corresponder a uma função objetivo que deve ser maximizada, então o objetivo será encontrar um **máximo global** (pico)



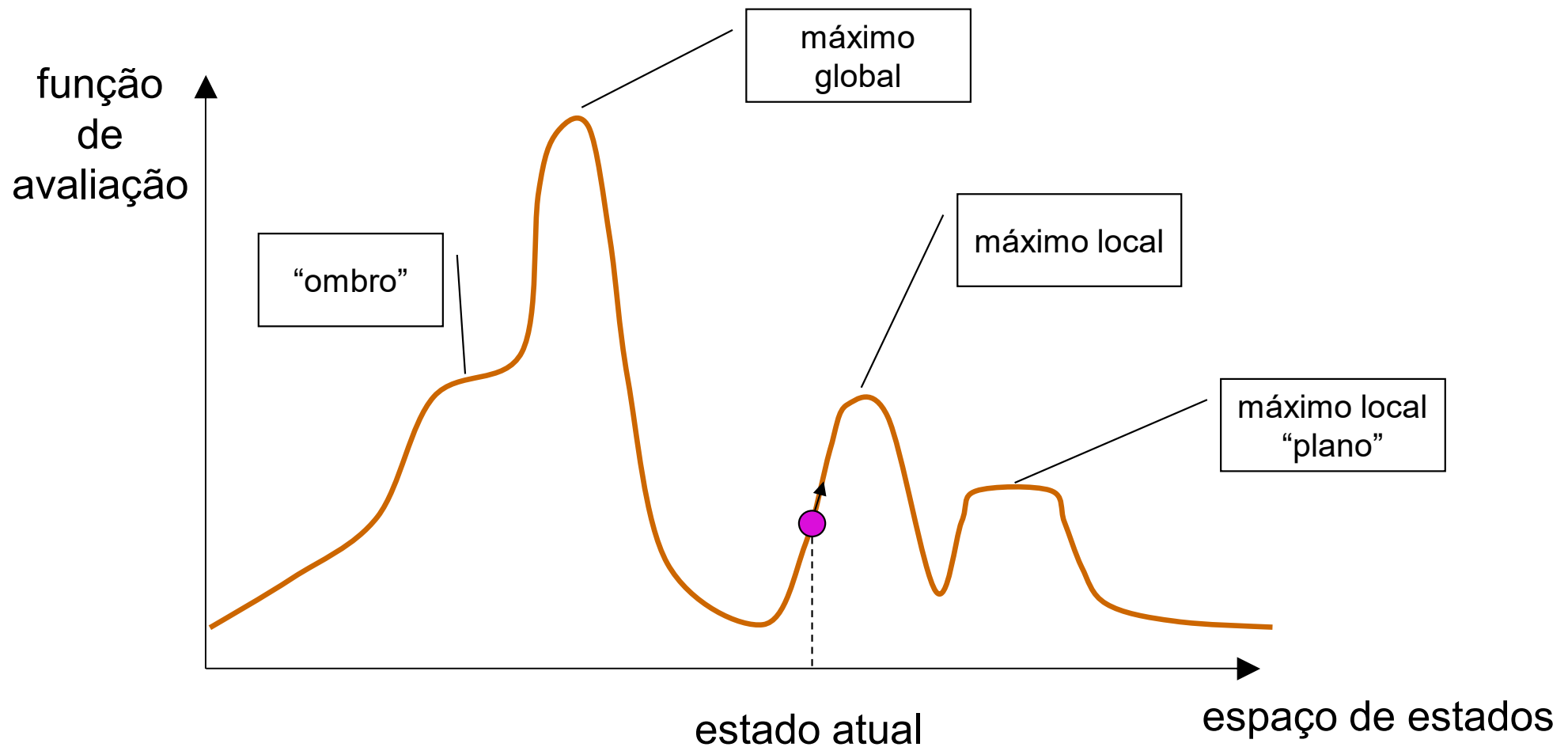
Algoritmo de Subida de Encosta

- ❑ O algoritmo de **subida de encosta** (*hill-climbing*) se move de forma contínua no sentido crescente da função de avaliação, ou seja, encosta acima; o algoritmo termina ao encontrar um pico (um máximo)
 - O algoritmo também é conhecido como **gradiente descendente**, se o objetivo é minimizar a função de avaliação
- ❑ O algoritmo não mantém uma árvore de busca mas somente o estado atual e o valor de sua função de avaliação
- ❑ O algoritmo não explora valores da função de avaliação além dos vizinhos imediatos ao estado atual
 - “É como escalar o monte Everest em um nevoeiro denso com amnésia”
 - “É como usar óculos que limitam sua visão a 3 metros”

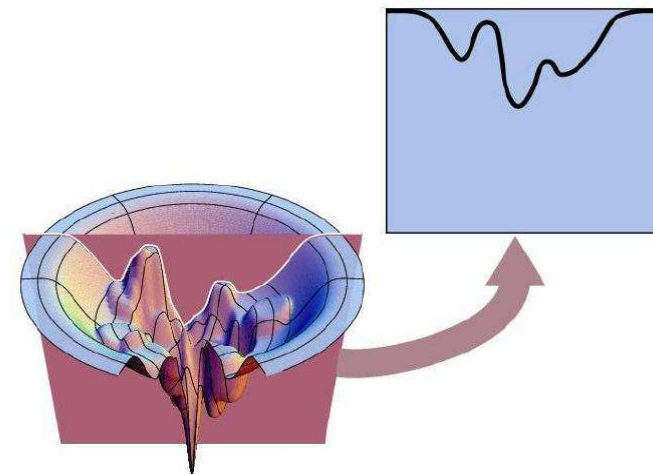
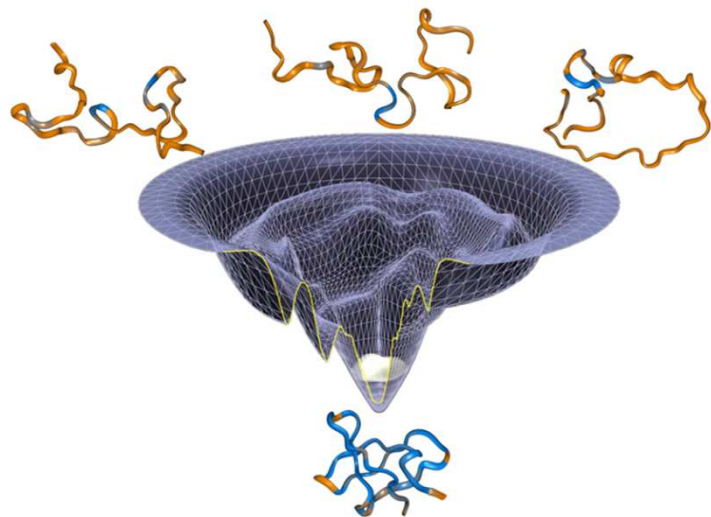
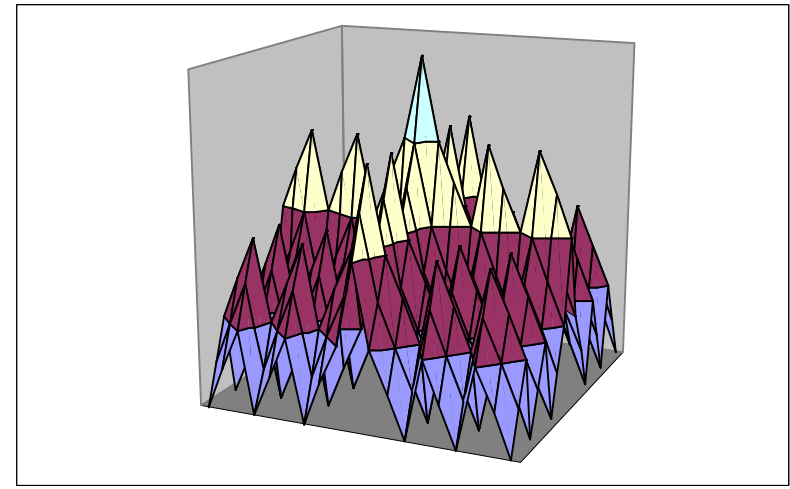
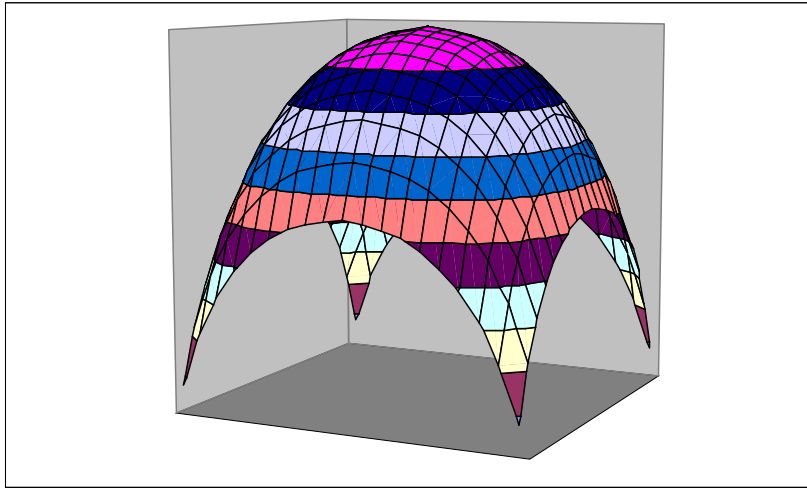
Algoritmo de Subida de Encosta

- ❑ O algoritmo também é chamado de busca gulosa local, pois captura um bom estado vizinho sem decidir com antecedência para onde irá em seguida
- ❑ O algoritmo tende a ser rápido ao encontrar uma solução pois é sempre fácil melhorar um estado ruim
- ❑ Problemas
 - Máximo local: uma vez atingido, o algoritmo termina mesmo que a solução esteja longe de ser satisfatória
 - Platôs (regiões planas): regiões onde a função de avaliação é essencialmente plana; isso pode impedir o algoritmo de encontrar uma saída do platô
 - Picos: vários máximos locais não interligados no espaço de estados, o que dificulta a navegação pelo algoritmo

Algoritmo de Subida de Encosta



Algoritmo de Subida de Encosta



Algoritmo de Subida de Encosta

- ❑ Algoritmo de Subida de Encosta (Puro/Original)
 1. Escolha um estado inicial do espaço de busca (pode ser escolhido de forma aleatória)
 2. Considere todos os vizinhos (sucessores) no espaço de busca
 3. Escolha o vizinho com a melhor qualidade (função de avaliação) e mova para aquele estado
 4. Repita os passos de 2 até 4 até que todos os estados vizinhos tenham qualidade menor ou igual ao estado atual
 5. Retorne o estado atual como sendo a solução
- ❑ Alternativas no caso de haver mais de um vizinho com a melhor qualidade:
 - Escolher o primeiro melhor
 - Escolher um dentre todos os melhores de forma aleatória

Variações da Busca Guiada por Heurística

- ❑ Os códigos a seguir ilustram três variações baseadas na avaliação heurística, evidenciando como pequenas mudanças na memória e no descarte de nós alteram completamente a classificação do algoritmo:
- 1. Abordagem Local Pura (Subida de Encosta / Hill-Climbing)
 - Foco em Configuração: Abandona o histórico do caminho, mantendo apenas o estado atual (Memória estritamente $O(1)$)
 - Parada Natural: Não requer o predicado `final(No)`. O algoritmo relata onde travou assim que o $h(\cdot)$ para de cair (seja na solução ótima ou em um ótimo local)
 - Sem Detecção de Ciclos: Como o movimento só é aceito se for estritamente melhor, é impossível voltar para o estado anterior, dispensando o uso do `member/2`
- 2. Abordagem Local com Rastreio ("Licença Poética" Didática)
 - Foco em Roteamento: É uma adaptação prática do Hill-Climbing que guarda o caminho na memória (`[No|Caminho]`) apenas para podermos visualizar a rota ao final
 - Forte Descarte: O retrocesso (backtracking) continua bloqueado. A extração `[MelhorCaminho|_]` joga os irmãos fora, forçando o algoritmo a seguir apenas o melhor sucessor
 - Uso da Fila: A fila de prioridade serve apenas para trazer o melhor vizinho para a cabeça da lista; o restante é imediatamente descartado
- 3. Abordagem Sistemática (DFS Ordenada por Heurística)
 - Foco em Completude: Atua como uma extensão estruturada da busca DFS básica, mantendo todos os caminhos gerados na memória
 - Prevenção de Loops: Possui detecção de ciclos embutida obrigatoriamente (via `member/2` no caminho atual)
 - Retenção de Alternativas: Permite o retrocesso (backtracking) natural do Prolog: se o vizinho mais promissor falhar mais à frente, o algoritmo retorna à lista de alternativas retida na memória e tenta o segundo melhor

Algoritmo de Subida de Encosta Puro

```
resolvaHcPuro(No, SolucaoFinal) :-
    h(No, H),
    hillClimbingPuro(No:H, SolucaoFinal).

% Passo Recursivo (A Escalada)
hillClimbingPuro(NoAtual:HAAtual, SolucaoFinal) :-
    estenderLocal(NoAtual, Vizinhos),
    Vizinhos \= [], % Tem vizinhos
    filaPrioridadeH(Vizinhos, [], [MelhorViz:HMV | _]), % Pega o melhor
    % So move se for estritamente melhor
    HMV < HAAtual, !, % O corte garante que não tem backtracking
    hillClimbingPuro(MelhorViz:HMV, SolucaoFinal).

% Condição de Parada: se a regra acima falhar (não tem vizinho ou nenhum vizinho melhora o H),
% Prolog cai nesta regra. Ele simplesmente devolve o Nó Atual e encerra. Note que ele não tem o
% predicado final(No). Ele para naturalmente quando o h() para de cair. Se o h() parou em zero,
% achou o objetivo global. Se o h() parou em 3, o algoritmo caiu num Ótimo Local e reportou o
% tabuleiro defeituoso
hillClimbingPuro(NoAtual:_, NoAtual).

% predicado auxiliar (Sem verificação de ciclos)
% Note que o estado é apenas 'No:H', não tem mais lista [No|Caminho]
% E o \+ member(...) sumiu, pois a regra HMV < HAAtual impede voltar
estenderLocal(No, Vizinhos) :-
    findall(NovoNo:H, ( s(No, NovoNo, _),
                        h(NovoNo, H) ),
            Vizinhos).
```

Hill-Climbing com Rastreio vs DFS Ordenada por Heurística

% BUSCA LOCAL: SUBIDA DE ENCOSTA (HILL-CLIMBING COM RASTREIO)

```
resolvaHcTrace(No, Solucao) :-  
    h(No, H),  
    hillClimbingTrace([No]:H, Solucao).  
  
hillClimbingTrace([No|Caminho]:_, [No|Caminho]) :-  
    final(No), !.  
hillClimbingTrace(Caminho:_, Solucao) :-  
    estenderH(Caminho, NovosCaminhos),  
    NovosCaminhos \= [],  
    filaPrioridadeH(NovosCaminhos, [],  
                    [MelhorCaminho | _]),  
  
    % Extrai o melhor e descarta o resto  
    hillClimbingTrace(MelhorCaminho, Solucao).
```

% BUSCA SISTEMÁTICA: DFS ORDENADA POR HEURÍSTICA

```
resolvaHdfs(No, Solucao) :-  
    h(No, H),  
    heuristicDfs([No]:H, Solucao).  
  
heuristicDfs([No|Caminho]:_, [No|Caminho]) :-  
    final(No).  
heuristicDfs(Caminho:_, Solucao) :-  
    estenderH(Caminho, NovosCaminhos),  
    NovosCaminhos \= [],  
    filaPrioridadeH(NovosCaminhos, [],  
                    CaminhosOrdenados),  
    member(MelhorCaminho, CaminhosOrdenados),  
    % Gerador de backtracking (built-in)  
    heuristicDfs(MelhorCaminho, Solucao).
```

Hill-Climbing com Rastreio vs DFS Ordenada por Heurística

```
% predicados auxiliares para os dois algoritmos anteriores (Sufixo 'H' - Foco na Heurística)
```

```
% Gera vizinhos e avalia usando apenas H
```

```
estenderH([No|Caminho], NovosCaminhos) :-
```

```
    findall([NovoNo, No|Caminho]:H,  
            ( s(No, NovoNo, _Custo),  
              \+ member(NovoNo, [No|Caminho]), % sem ciclos  
              h(NovoNo, H)  
            ),  
            NovosCaminhos).
```

```
% Fila de prioridade que ordena pelo valor H
```

```
filaPrioridadeH([], L, L).
```

```
filaPrioridadeH([Caminho:H|Caminhos], Existente, Final) :-
```

```
    insertH(Caminho:H, Existente, ExistenteAumentado),  
    filaPrioridadeH(Caminhos, ExistenteAumentado, Final).
```

```
% Inserção ordenada (Menor H fica na frente)
```

```
insertH(Caminho:H, [Path:Hs|Paths], [Path:Hs|NewPaths]) :-
```

```
    H > Hs, !,  
    insertH(Caminho:H, Paths, NewPaths).
```

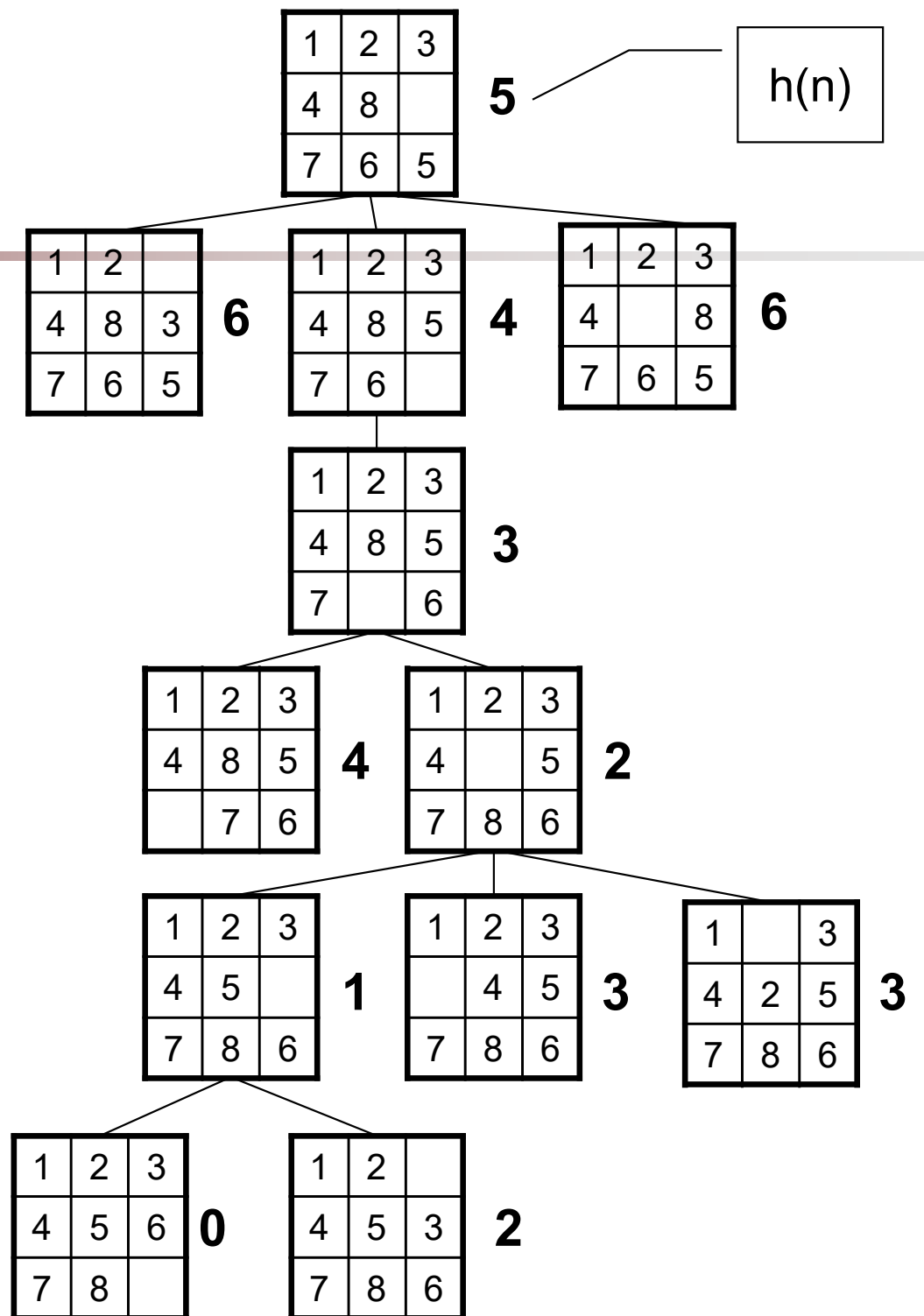
```
insertH(C, Paths, [C|Paths]).
```

Variações da Busca Guiada por Heurística

Característica	HC Puro (Teórico)	HC com Rastreio (Licença Poética)	DFS Heurística (Busca Sistemática)
Linha de Código Chave	$HMV < H_{Atual}$, ! (Avanço Estrito)	[MelhorCaminho _] (Descarte via Padrão)	member(Melhor, Caminhos) (Gerador)
Classificação	Busca Local Pura (Configuração)	Busca Local Adaptada (Roteamento)	Busca Sistemática
Uso de Memória	$O(1)$ (Guarda apenas o nó atual)	$O(m)$ (Guarda 1 única rota, que cresce com a profundidade)	$O(bd)$ (A pilha guarda os irmãos inexplorados)
Mecanismo de Parada / Falha	Para naturalmente e retorna o estado atual assim que o $h(\cdot)$ parar de cair	Se travar antes de alcançar o nó objetivo (final), a execução falha	Se o ramo falhar, Prolog ativa o Backtracking e tenta o próximo irmão
Garantia (Compleitude)	Nenhuma. Reporta o tabuleiro onde travou (Ótimo Local ou Global).	Nenhuma. Aborta sem devolver solução se cair num Ótimo Local	Completo (em grafos finitos). Se a solução existir, ele a encontrará

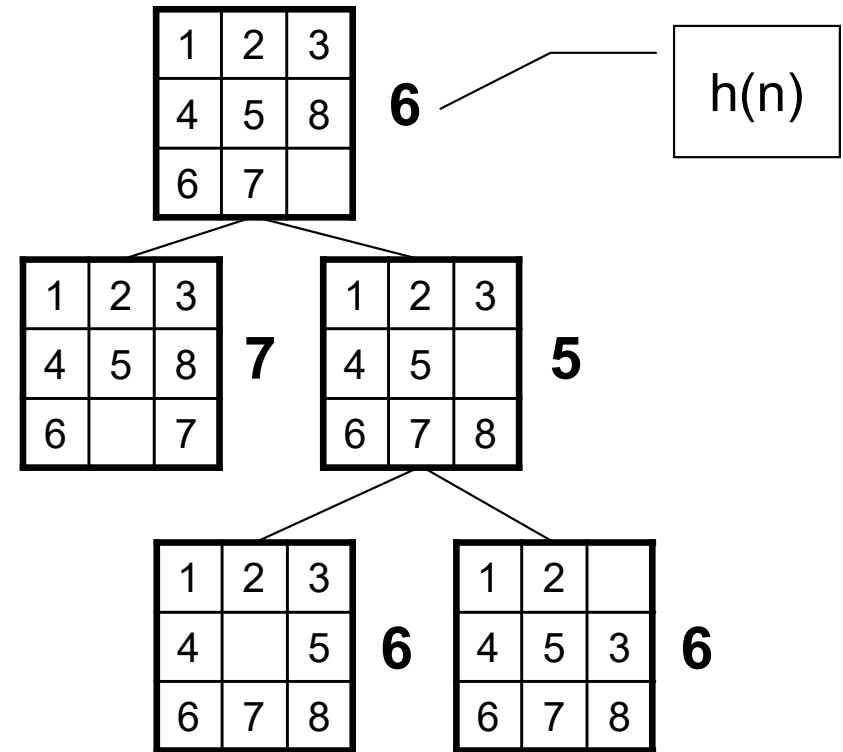
Exemplo 1

- ❑ Neste exemplo, a heurística distância Manhattan permite encontrar uma solução rapidamente por hill-climbing



Exemplo 2

- ❑ Neste exemplo, com a mesma heurística, hill-climbing não consegue encontrar uma solução
- ❑ Todos os nós vizinhos (sucessores) têm valores maiores (mínimo local)
- ❑ Observe que este jogo tem solução em apenas mais 12 movimentos



Hill-Climbing: Variações

❑ Hill-Climbing Estocástico

- Nem sempre escolha o melhor vizinho

❑ Hill-Climbing Primeira Escolha

- Escolha o primeiro bom vizinho que encontrar
 - ❖ Útil se é grande o número de sucessores de um nó

❑ Hill-Climbing Reinício Aleatório

- Conduz uma série de buscas hill-climbing a partir de estados iniciais gerados aleatoriamente, executando cada busca até terminar ou até que não exista progresso significativo
- O melhor resultado de todas as buscas é armazenado

Hill-Climbing Estocástico

```
resolvaHcPrimeiraEscolha(No, SolucaoFinal) :-  
    h(No, H),  
    hcPrimeiraEscolha(No:H, SolucaoFinal).
```

% Esqueleto Conceitual

```
hcPrimeiraEscolha(NoAtual:HAtual, SolucaoFinal) :-  
    % Não gere a lista inteira de sucessores.  
    % Escreva um predicado que gere UM vizinho por vez  
    % e faça um cut (!) no exato momento em que encontrar  
    % um HVizinho < HAtual.
```

```
    gerarPrimeiroBomVizinho(NoAtual, HAtual, VizinhoBom:HVizinho),  
    hcPrimeiraEscolha(VizinhoBom:HVizinho, SolucaoFinal).
```

```
% Se o gerarPrimeiroBomVizinho falhar (ou seja, testou todos os vizinhos  
% e nenhum tem o HVizinho menor que o HAtual), Prolog cai nesta regra,  
% encerra a busca e relata o nó onde travou  
hcPrimeiraEscolha(NoAtual:_, NoAtual).
```

Hill-Climbing Primeira Escolha

```
resolvaHcEstocastico(No, SolucaoFinal) :-  
    h(No, H),  
    hcEstocastico(No:H, SolucaoFinal).
```

% Esqueleto Conceitual

% Passo Recursivo: A Escalada Probabilística

```
hcEstocastico(NoAtual:HAtual, SolucaoFinal) :-
```

% Usa o estenderLocal para gerar os vizinhos no formato Vizinho:HViz (sem caminho)

```
estenderLocal(NoAtual, Vizinhos),
```

% Escreva um predicado para filtrar a lista 'Vizinhos',

% mantendo apenas aqueles onde HVizinho < HAtual (movimentos de subida)

```
filtrarVizinhosMelhores(Vizinhos, HAtual, VizinhosBons),
```

VizinhosBons \= [], % Se não sobrar nenhum vizinho melhor, falha e para

% Substitua a fila de prioridade por uma função que calcule a

% probabilidade de cada vizinho bom (baseado no quão melhor ele é)

% e aplique o método da Roleta (usando o random) para sortear um deles.

```
sortearVizinhoRoleta(VizinhosBons, VizinhoSorteado:HVizinho),
```

```
hcEstocastico(VizinhoSorteado:HVizinho, SolucaoFinal).
```

% Se a lista de VizinhosBons ficar vazia (ou seja, você está no topo da colina

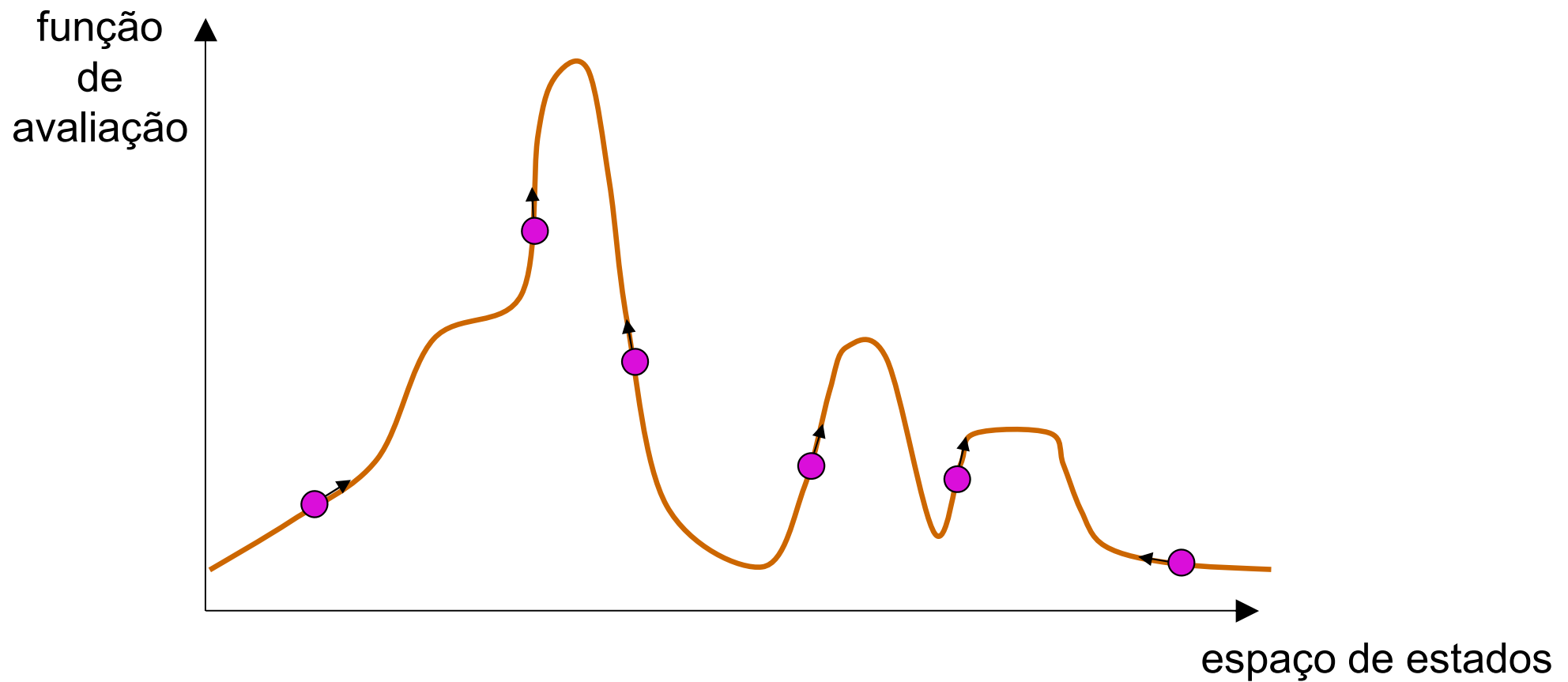
% e todos os lados descem), o Prolog cai nesta regra e devolve o Nó Atual.

```
hcEstocastico(NoAtual:_, NoAtual).
```

Hill-Climbing Reinício Aleatório

```
resolvaHcReinicioAleatorio(Solucao) :-  
    hcReinicioAleatorio(100, Solucao). % Define o numero de execucoes (k)  
  
hcReinicioAleatorio(MaxIteracoes, SolucaoFinal) :-  
    gerarEstadoAleatorio(NoInicial),  
    h(NoInicial, H),  
    loopReinicio(MaxIteracoes, NoInicial:H, SolucaoFinal).  
  
% Condição de Parada 1: Esgotaram-se as tentativas. Retorna o melhor histórico.  
loopReinicio(0, RecordeAtual, RecordeAtual) :- !.  
% Condição de Parada 2 (Otimização): Achou a solução global (H=0).  
% Interrompe as repetições e reporta o resultado  
loopReinicio(_, MelhorNo:0, MelhorNo:0) :- !.  
% Passo Recursivo do Reinício  
loopReinicio(IteracoesRestantes, RecordeAteAgora, SolucaoFinal) :-  
    gerarEstadoAleatorio(NovoNo),  
    h(NovoNo, H),  
    % Roda a Subida de Encosta Pura (sem caminhos)  
    hillClimbingPuro(NovoNo:H, SolucaoDestaTentativa),  
    % Compara o resultado desta escalada com o recorde global  
    atualizarRecorde(RecordeAteAgora, SolucaoDestaTentativa, NovoRecorde),  
    NovasIteracoes is IteracoesRestantes - 1,  
    loopReinicio(NovasIteracoes, NovoRecorde, SolucaoFinal).  
  
% Se o HNovo for estritamente menor que o HAntigo, temos um novo recorde  
atualizarRecorde(_:HAntigo, NovoNo:HNovo, NovoNo:HNovo) :-  
    HNovo < HAntigo, !.  
% Caso contrário (empatou ou piorou), o recorde antigo é mantido.  
atualizarRecorde(RecordeAntigo, _, RecordeAntigo).
```

Hill-Climbing Reinício Aleatório



Têmpera Simulada (Simulated Annealing)

- ❑ É uma variação estocástica da Subida de Encosta (Hill-Climbing) que permite, de forma controlada, movimentos “ruins” para evitar ficar preso em ótimos locais
- ❑ Analogia com a Metalurgia:
 - Quando um metal é aquecido e resfriado muito rápido, suas moléculas formam uma estrutura frágil (ótimo local)
 - Têmpera é o processo de aquecer o metal a altas temperaturas e deixá-lo resfriar lentamente
 - No calor (alta temperatura), as moléculas se agitam e exploram posições diversas
 - ❖ Conforme resfriam, o movimento diminui, permitindo que se acomodem em uma estrutura cristalina altamente estável e perfeita (ótimo global)

Algoritmo: Têmpera Simulada

```
% Esqueleto Conceitual
```

```
atual = escolha um estado inicial aleatório
```

```
i = 1
```

```
T = Temperatura(i)
```

```
while T > Tf    % Tf eh a temperatura final
```

```
    vizinho = escolha um sucessor aleatório de atual
```

```
    deltaQ = Qualidade(vizinho) - Qualidade(atual)
```

```
    if deltaQ > 0    % vizinho com melhor qualidade, aceitar
```

```
        atual = vizinho
```

```
    else % vizinho com pior qualidade, aceitar probabilisticamente
```

```
        atual = vizinho com probabilidade  $\exp(\text{deltaQ} / T)$ 
```

```
    endif
```

```
    i = i + 1
```

```
    T = Temperatura(i)
```

```
endwhile
```

```
return atual
```

Têmpera Simulada (Simulated Annealing)

- ❑ A probabilidade $e^{\Delta Q/T}$ (com $\Delta Q \leq 0$) garante que pioras pequenas são mais fáceis de aceitar do que pioras grandes e que, à medida que T cai, a chance de aceitar pioras tende a zero
- ❑ Altas Temperaturas (início da busca)
 - O valor de T é grande
 - Logo, $e^{\Delta Q/T} \approx 1$ (100%)
 - O algoritmo age como um Passeio Aleatório (*Random Walk*), aceitando quase qualquer movimento ruim, o que garante ampla exploração do espaço de estados
- ❑ Temperaturas Baixas (fim da busca)
 - O valor de $T \approx 0$
 - Logo, a probabilidade de aceitar movimentos ruins cai drasticamente
 - O algoritmo age como um Hill-Climbing Puro, parando de explorar e focando em escalar até o topo da colina onde está
- ❑ O Balanço Perfeito
 - Movimentos negativos controlados são essenciais para escapar de máximos locais
 - A chave para encontrar o máximo global está na lentidão do resfriamento (Temperatura(i))
 - ❖ se esfriar muito rápido, trava num ótimo local
 - ❖ se esfriar devagar o suficiente, a teoria prova que encontrará o ótimo global

Têmpera Simulada (Simulated Annealing)

```
resolvaTemperaSimulada(No, Solucao) :-  
    h(No, H),  
    TempInicial is 100.0, % Define a temperatura inicial de derretimento  
    temperaSimulada(No:H, TempInicial, Solucao).  
  
% 1. Achou o objetivo final (H = 0)  
temperaSimulada(NoAtual:0, _, NoAtual:0) :- !.  
% 2. O sistema congelou (Temperatura muito próxima de zero)  
% Retorna onde parou (pode ser um ótimo local se a têmpera falhou)  
temperaSimulada(NoAtual:HAtual, Temp, NoAtual:HAtual) :-  
    Temp < 0.001, !.  
% 3. Passo de busca e resfriamento  
temperaSimulada(NoAtual:HAtual, Temp, SolucaoFinal) :-  
    % Ele deve usar o s(NoAtual, Vizinho) e sortear apenas UM deles.  
    vizinhoAleatorio(NoAtual, VizinhoSorteado:HVizinho),  
    DeltaH is HVizinho - HAtual,  
    ( aceitaVizinho(DeltaH, Temp) ->  
        ProximoNo = VizinhoSorteado:HVizinho ;  
        ProximoNo = NoAtual:HAtual  
    ),  
    NovaTemp is Temp * 0.95, % Taxa de resfriamento (cooling rate)  
    temperaSimulada(ProximoNo, NovaTemp, SolucaoFinal).
```

Têmpera Simulada (Simulated Annealing)

```
% A logica da aceitacao com base na funcao h()

% 1. O vizinho é melhor ou igual (DeltaH <= 0)
% O custo caiu ou manteve, aceitar
aceitaVizinho(DeltaH, _) :-
    DeltaH <= 0, !.

% 2. O vizinho é pior (DeltaH > 0)
% O custo aumentou.
% Só aceita baseando-se na probabilidade da Têmpera Simulada
aceitaVizinho(DeltaH, Temp) :-
    % Probabilidade P = e^(-DeltaH / Temp)
    % Usamos o sinal de menos porque DeltaH eh positivo e queremos
    % uma probabilidade entre 0 e 1 (expoente negativo)
    Probabilidade is exp(-DeltaH / Temp),
    random(0.0, 1.0, Sorteio),
    Sorteio < Probabilidade.
```

Têmpera Simulada (Simulated Annealing)

- ❑ Na formulação clássica da física, a Têmpera tenta minimizar a Energia (que equivale maximizar a qualidade do material)
- ❑ Em Inteligência Artificial, nós minimizamos a Heurística $h(\cdot)$
 - $h(n) = 0$ indica a solução final (qualidade máxima)
 - $\text{delta}H = h(\text{novo}) - h(\text{atual})$
 - Se $\text{delta}H < 0$ custo diminuiu (a qualidade melhorou) e o movimento é aceito
 - Se $\text{delta}H > 0$ o custo aumentou (a qualidade piorou) e a aceitação depende da probabilidade $P = e^{-\frac{\text{delta}H}{T}}$
- ❑ No algoritmo:
 - Temperatura Inicial: **TempInicial is 100.0**
 - ❖ Se for muito baixa, o algoritmo vira um simples Hill-Climbing e trava no primeiro ótimo local
 - ❖ Se for muito alta, ele vira uma busca aleatória cega que demora muito
 - Taxa de Resfriamento (0.95) dita a velocidade do algoritmo, **NovaTemp is Temp * 0.95**
 - ❖ Se multiplicar por 0.5, esfria rápido demais
 - ❖ Se multiplicar por 0.99, esfria lentamente e aumenta muito a chance de achar a solução perfeita, mas consome mais CPU

Busca em Feixe (Beam Search)

- ❑ É uma variação da Busca em Largura (ou Best-First) com severa restrição de memória
- ❑ O limite k indica o tamanho do Feixe: Em vez de guardar todos os caminhos gerados na Fronteira, o algoritmo mantém no máximo k estados (caminhos) simultâneos na memória a cada momento
- ❑ O objetivo consiste em explorar as ramificações mais promissoras sem deixar a memória estourar (ideal para árvores de busca muito grandes, como reconhecimento de fala e processamento de linguagem)

Busca em Feixe (Beam Search)

1. O algoritmo começa com a Fronteira contendo apenas o estado inicial
2. Expansão: São gerados todos os sucessores dos nós que atualmente compõem o feixe na Fronteira
3. Se algum dos sucessores for a solução final, o algoritmo termina com sucesso
4. A poda: todos os sucessores recém-gerados são avaliados pela função Heurística
 - O algoritmo seleciona estritamente os k melhores sucessores e descarta todo o resto
5. A Fronteira é atualizada apenas com esses k sobreviventes e o algoritmo volta ao passo 2

Busca em Feixe (Beam Search)

- ❑ Vantagem (Memória Consistente):
 - Diferente da Busca em Largura, cuja Fronteira cresce exponencialmente, a Busca em Feixe usa uma quantidade de memória estritamente constante e previsível $O(k)$
- ❑ O Risco (Falta de Completude):
 - A poda é definitiva.
 - Se o caminho para a solução ótima (ou para a única solução existente) passar por um nó temporariamente ruim que foi cortado pela poda, o algoritmo nunca achará a resposta
- ❑ Variações:
 - Em problemas de otimização pura (como as N-Rainhas), utiliza-se a Busca em Feixe Local, que omite o estado inicial único e começa sorteando k estados aleatórios diferentes no espaço para varrê-lo de forma paralela

Busca em Feixe (Beam Search)

```
% Busca em Feixe (classica, com caminho) (Beam Search)
% O algoritmo começa com a Fronteira contendo apenas o estado inicial
resolvaBeamSearch(NoInicial, K, SolucaoFinal) :-
    h(NoInicial, H),
    FronteiraInicial = [[NoInicial]:H],
    beamSearch(FronteiraInicial, K, CaminhoInvertido),
    reverse(CaminhoInvertido, SolucaoFinal).

% Termina: Se algum sucessor (agora no feixe) for a solução final
beamSearch(Fronteira, _, [NoAtual | Caminho]) :-
    % Procura na fronteira se o nó da cabeça de algum caminho é o estado final
    member([NoAtual | Caminho]:_, Fronteira),
    final(NoAtual), !. % Achou, corta a busca e termina

% Expansão e poda
beamSearch(Fronteira, K, SolucaoFinal) :-
    % EXPANSÃO: Gera todos os sucessores dos nós que compõem o feixe
    findall([NovoNo, NoAtual | Restante]:NovoH,
        (member([NoAtual | Restante]:_, Fronteira), % Avalia cada caminho no feixe
         s(NoAtual, NovoNo, _), % Gera os filhos
         \+ member(NovoNo, [NoAtual | Restante]), % Evita ciclos no caminho
         h(NovoNo, NovoH) % Avalia a heurística
        ), TodosSucessores),

    % Se a lista vier vazia, o feixe inteiro entrou em beco sem saída
    TodosSucessores \= [],

    % PODA: Ordena pela Heurística e seleciona os K melhores
    ordenarPorHeuristica(TodosSucessores, SucessoresOrdenados),
    pegarK(K, SucessoresOrdenados, NovoFeixe),

    % ATUALIZAÇÃO: A Fronteira é atualizada e volta ao início
    beamSearch(NovoFeixe, K, SolucaoFinal).
```

Busca em Feixe (Beam Search)

```
% Selecionar os K primeiros da lista
pegarK(0, _, []) :- !.                                % Já pegou K elementos
pegarK(_, [], []) :- !.                                % Acabaram os elementos (tinha menos que K)
pegarK(K, [X|Xs], [X|Ys]) :-
    K > 0,
    K1 is K - 1,
    pegarK(K1, Xs, Ys).

% quicksort
ordenarPorHeuristica([], []).
ordenarPorHeuristica([Pivo | Resto], ListaOrdenada) :-
    dividir(Resto, Pivo, Menores, Maiores),
    ordenarPorHeuristica(Menores, MenoresOrdenados),
    ordenarPorHeuristica(Maiores, MaioresOrdenados),
    append(MenoresOrdenados, [Pivo | MaioresOrdenados], ListaOrdenada).

dividir([], _, [], []).
% Se o H do item atual for MENOR OU IGUAL ao H do Pivô, vai para a lista de Menores
dividir([Caminho:H | Resto], PivoCaminho:PivoH, [Caminho:H | Menores], Maiores) :-
    H =< PivoH, !,
    dividir(Resto, PivoCaminho:PivoH, Menores, Maiores).
% Caso contrário (H maior), vai para a lista de Maiores
dividir([Caminho:H | Resto], Pivo, Menores, [Caminho:H | Maiores]) :-
    dividir(Resto, Pivo, Menores, Maiores).
```

Busca em Feixe Local (Local Beam Search)

- ❑ Enquanto a versão clássica (beam search) constrói uma árvore a partir de um nó inicial único, a versão Local descarta a ideia de "caminho"
- ❑ O foco é achar uma configuração final válida
- ❑ O início é descentralizado: em vez de começar no nó inicial, o algoritmo inicia espalhando k estados aleatórios pelo espaço de busca
- ❑ A Mecânica Darwinista (Sobrevivência dos Mais Aptos)
 - Gera-se todos os sucessores dos k estados atuais simultaneamente
 - Avalia-se a heurística de toda essa massa gerada
 - Seleciona-se estritamente os k melhores vizinhos do grupo inteiro, descartando o resto
- ❑ Busca Cooperativa vs. Independente
 - A Busca em Feixe Local não é o mesmo que rodar k Subidas de Encosta isoladas
 - Existe um efeito manada: se apenas um dos estados descobrir uma região excelente no mapa, os filhos dele vão dominar o pódio e roubar todas as k vagas na próxima etapa, extinguindo rapidamente as buscas paralelas que estavam presas em platôs e becos sem saída
 - Analogia: imagine k pessoas procurando ouro em montanhas separadas
 - No Hill-Climbing com Reinício, elas não se comunicam
 - Na Busca em Feixe Local, no fim de cada passo, elas avisam como está o terreno. Se uma delas gritar que achou ouro, todas as outras $k - 1$ pessoas abandonam suas montanhas ruins e vão imediatamente explorar os arredores da montanha promissora

Busca em Feixe Local (Local Beam Search)

```
% Busca em Feixe Local (Local Beam Search)
resolvaLocalBeamSearch(K, SolucaoFinal) :-
    % 1. Início descentralizado: gera K estados aleatórios
    gerarKEstados(K, FeixeInicial), % Retorna ex: [No1:H1, No2:H2, ...]
    localBeamSearch(FeixeInicial, K, SolucaoFinal).

% Termina: Se algum estado do feixe atingiu a perfeição (H=0)
localBeamSearch(Feixe, _, NoAtual:0) :-
    member(NoAtual:0, Feixe),
    !. % Achou e termina

% Expansão e seleção competitiva
localBeamSearch(Feixe, K, SolucaoFinal) :-
    % EXPANSÃO GLOBAL: Gera todos os sucessores de todos os K estados
    findall(NovoNo:NovoH,
        (member(NoAtual:_, Feixe),
         s(NoAtual, NovoNo, _),
         h(NovoNo, NovoH)
        ),
        TodosSucessores),
    TodosSucessores \= [], % Se todos morrerem, falha
    % SELEÇÃO DARWINISTA: Ordena tudo e pega apenas os K melhores absolutos
    % Os estados ruins morrem, os bons dominam as vagas do novo feixe
    ordenarPorHeuristica(TodosSucessores, SucessoresOrdenados),
    pegarK(K, SucessoresOrdenados, NovoFeixe),
    % Nova geração
    localBeamSearch(NovoFeixe, K, SolucaoFinal).
```

Resumo

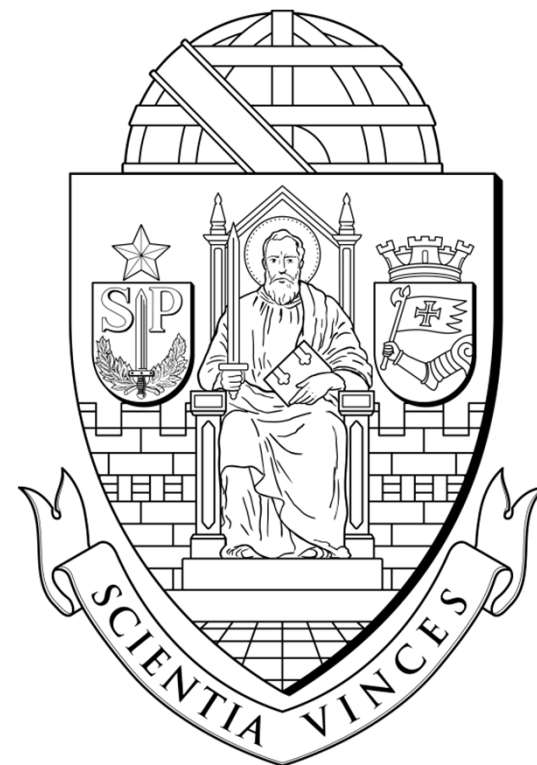
- ❑ Buscas Sistemáticas Tradicionais (Caminho Ótimo)
 - Greedy Best-First (Gulosa): Foca apenas na heurística (h). É extremamente rápida, mas míope: não garante a solução ótima e pode sofrer com *loops* se não usar Lista Fechada
 - A*: O "padrão ouro". Equilibra o custo passado e a estimativa futura ($f = g + h$). Garante a rota ótima, mas o consumo de memória pode ser exponencial na Fronteira
- ❑ Buscas Sistemáticas com Memória Limitada
 - IDA* e RBFS: Foram criados para salvar a memória RAM. Ambos garantem a solução ótima utilizando uma quantidade de espaço linear $O(bd)$ em relação à profundidade da busca
 - Diferentemente do IDA* (que usa limites iterativos rígidos), o RBFS consegue expandir os nós na ordem exata do best-first (igual ao A*), mantendo o rastreo da melhor alternativa na recursão, mesmo se a heurística for inconsistente (função f não monotônica)
- ❑ Buscas Locais (Foco na Configuração e Memória Restrita)
 - Subida de Encosta (Hill-Climbing): Descarta o caminho e o *backtracking*. Memória estrita de tamanho 1. Extremamente rápido, mas vulnerável a travar em ótimos locais
 - Têmpera Simulada (Simulated Annealing): Evolução da Subida de Encosta que consegue escapar de ótimos locais permitindo "pioras" temporárias, controladas por uma "temperatura" que vai esfriando
 - Busca em Feixe Local (Beam Search): Mantém k estados simultâneos trocando informações. Equilibra o paralelismo da busca com o uso constante de memória $O(k)$

*Copyright© Apresentação 2026 por
José Augusto Baranauskas
Universidade de São Paulo*

*Professores são convidados a utilizarem esta
apresentação da maneira que lhes for conveniente para
uso não comercial, desde que esta nota de copyright
permaneça intacta.*

Slides baseados nos livros:

- ❑ *Bratko, I.; Prolog Programming for Artificial Intelligence, 3rd Edition, Pearson Education, 2001.*
- ❑ *Clocksin, W.F.; Mellish, C.S.; Programming in Prolog, 5th Edition, Springer-Verlag, 2003.*



USP